UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

NAILSON FRANCISCO DA SILVA CHAGAS

DESENVOLVIMENTO DE UMA PLATAFORMA HIL PARA VALIDAÇÃO DE CONTROLADORES DIGITAIS PARA CONVERSORES CC-CC

PATO BRANCO

2026

versão 11.0 (abr. 25)

NAILSON FRANCISCO DA SILVA CHAGAS

DESENVOLVIMENTO DE UMA PLATAFORMA HIL PARA VALIDAÇÃO DE CONTROLADORES DIGITAIS PARA CONVERSORES CC-CC

DEVELOPMENT OF A HIL PLATFORM FOR VALIDATION OF DIGITAL CONTROLLERS FOR DC-DC CONVERTERS

Proposta de Trabalho de Conclusão de Curso de Graduação como requisito para obtenção do título de Bacharel em Engenharia De Computação da Universidade Tecnológica Federal do Paraná.
Orientador: Rafael Cardoso

PATO BRANCO

2026

4.0 Internacional

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

NAILSON FRANCISCO DA SILVA CHAGAS

DESENVOLVIMENTO DE UMA PLATAFORMA HIL PARA VALIDAÇÃO DE CONTROLADORES DIGITAIS PARA CONVERSORES CC-CC

Proposta de Trabalho de Conclusão de Curso de Graduação como requisito para obtenção do título de Bacharel em Engenharia De Computação da Universidade Tecnológica Federal do Paraná.

Data de aprovação: dia / mês por extenso / ano

Nome completo e por extenso do Membro 1 (de acordo com o Currículo Lattes)
Titulação (Especialização, Mestrado, Doutorado)
Nome completo e por extenso da instituição a qual possui vínculo

---

Nome completo e por extenso do Membro 2 (de acordo com o Currículo Lattes)
Titulação (Especialização, Mestrado, Doutorado)
Nome completo e por extenso da instituição a qual possui vínculo

---

Nome completo e por extenso do Membro 3 (de acordo com o Currículo Lattes)
Título (Titulação (Especialização, Mestrado, Doutorado)
Nome completo e por extenso da instituição a qual possui vínculo

PATO BRANCO

2026

Dedico este trabalho à minha família, por todo o apoio ao longo da minha vida.

AGRADECIMENTOS

Ao longo da minha vida, muitas pessoas contribuíram, direta ou indiretamente, para que eu chegasse até aqui. Embora não seja possível mencionar todos em uma única página, carrego comigo a gratidão por cada ensinamento, apoio e incentivo recebidos durante essa trajetória.

Agradeço, acima de tudo, aos meus pais, por todo o amor, dedicação e apoio incondicional ao longo da minha vida. Sou eternamente grato por todos os esforços realizados para que eu pudesse me dedicar aos estudos, inclusive pelo apoio financeiro nos momentos em que precisei priorizar minha formação acadêmica. Vocês foram meus primeiros e maiores professores, e grande parte de quem sou hoje devo aos valores e ensinamentos que recebi dentro de casa.

À minha irmã, agradeço por tudo o que me ensinou ao longo da minha vida, pelos conselhos, apoio e por sempre estar presente nos momentos importantes.

Gostaria também de agradecer especialmente aos meus amigos Anny Dias, João Manoel, Luiz Kramer, Matheus Deodato, Robert Hartkop e Taís Manica. Vocês não fazem ideia do quanto conhecer cada um de vocês transformou minha vida. Levarei para sempre comigo todas as conversas, aprendizados, incentivos e toda a ajuda que recebi ao longo dessa caminhada.

Aos meus primos Lucas Pedrosa e Jean Pangioni, agradeço por terem me apresentado, ainda na infância, ao universo dos MMORPGs, especialmente *Tibia* e *Ragnarok Online*. Muito da minha curiosidade por tecnologia e por compreender como sistemas computacionais funcionam surgiu a partir dessas experiências. Muito provavelmente, sem isso, eu não teria desenvolvido o interesse que me trouxe até a Engenharia da Computação.

Ao Professor Dr. Jefferson Tales, agradeço pelos ensinamentos que vão além do conteúdo acadêmico. Com ele, aprendi muito sobre disciplina, dedicação aos estudos e sobre como me portar dentro do ambiente acadêmico.

Ao Professor Dr. Rafael Cardoso, agradeço pela orientação, disponibilidade e apoio no desenvolvimento deste trabalho. Além disso, agradeço pelos excelentes ensinamentos transmitidos ao longo da graduação. Suas aulas despertaram grande parte do meu interesse pelas áreas de Controle e Processamento de Sinais, servindo como inspiração para minha trajetória acadêmica. Tenho grande admiração pela forma como conduz o ensino e compartilha seu conhecimento, sendo, para mim, uma das maiores referências da UTFPR nessas áreas.

Enfim, agradeço a todos que, de alguma forma, contribuíram para minha formação acadêmica, profissional e pessoal.

"It is possible to commit no mistakes and still lose. That is not a weakness. That is life." (Picard, 1989)

RESUMO

O desenvolvimento de sistemas de controle digital aplicados à eletrônica de potência demanda métodos de validação que reduzam riscos, custos e tempo de desenvolvimento, especialmente nas etapas iniciais de projeto. Nesse contexto, a simulação *Hardware-in-the-Loop* apresenta-se como uma alternativa capaz de integrar controladores reais a modelos computacionais executados em tempo real, permitindo a realização de testes em ambiente seguro e controlado. Este trabalho tem como objetivo desenvolver uma plataforma baseada nessa abordagem para a validação de controladores digitais aplicados a conversores de corrente contínua. Como exemplo de aplicação será considerado um conversor Buck. Inicialmente, foi realizada uma revisão bibliográfica sobre Desenvolvimento Baseado em Modelos, metodologia *X-in-the-Loop* e simulação *Hardware-in-the-Loop*, abordando seus fundamentos, aplicações, vantagens e requisitos computacionais. Com base nesse referencial teórico, foi proposta uma arquitetura composta por um computador para supervisão e parametrização, um simulador responsável pela execução em tempo real do modelo matemático da planta e um controlador destinado à execução da estratégia de controle. A metodologia prevê a modelagem e discretização do conversor CC-CC em estudo, a implementação do modelo em microcontrolador dedicado, o desenvolvimento do controlador digital e a integração dos módulos de *hardware* e *software* da plataforma. Também foi definido um procedimento de validação estruturado em etapas progressivas, contemplando a verificação do modelo matemático, do simulador, do controlador e da operação integrada do sistema. Como resultado esperado, pretende-se obter uma plataforma experimental baseada em *Hardware-in-the-Loop* capaz de reproduzir, em tempo real, a dinâmica de um conversor de corrente contínua e permitir a validação de estratégias de controle digital embarcado. Espera-se que o desenvolvimento da plataforma contribua para a aplicação integrada dos conhecimentos adquiridos ao longo da graduação, envolvendo conceitos de teoria de controle, modelagem de sistemas dinâmicos, sistemas embarcados e programação de microcontroladores, e que, além de possibilitar o estudo e a validação de estratégias de controle digital em ambiente experimental, sua implementação favoreça a documentação e a disseminação das técnicas associadas à abordagem *Hardware-in-the-Loop*, ampliando o acesso a essa tecnologia em atividades de ensino e pesquisa.

Palavras-chave: controle digital; eletrônica de potência; hardware-in-the-loop; sistemas embarcados; conversor buck.

ABSTRACT

The development of digital control systems applied to power electronics requires validation methods capable of reducing risks, costs, and development time, especially during the early stages of design. In this context, Hardware-in-the-Loop simulation emerges as an alternative capable of integrating real controllers with computational models executed in real time, enabling tests to be carried out in a safe and controlled environment. This work aims to develop a platform based on this approach for the validation of digital controllers applied to direct current converters. A Buck converter is considered as an application example. Initially, a literature review was conducted on Model-Based Development, the X-in-the-Loop methodology, and Hardware-in-the-Loop simulation, addressing their fundamentals, applications, advantages, and computational requirements. Based on this theoretical background, an architecture composed of a computer for supervision and parameterization, a simulator responsible for the real-time execution of the mathematical model of the plant, and a controller dedicated to the execution of the control strategy was proposed. The methodology includes the modeling and discretization of the DC-DC converter under study, the implementation of the model on a dedicated microcontroller, the development of the digital controller, and the integration of the platform's hardware and software modules. A validation procedure structured in progressive stages was also defined, encompassing the verification of the mathematical model, the simulator, the controller, and the integrated operation of the system. As an expected outcome, an experimental Hardware-in-the-Loop platform capable of reproducing, in real time, the dynamics of a direct current converter and enabling the validation of embedded digital control strategies is intended to be obtained. It is expected that the development of the platform will contribute to the integrated application of the knowledge acquired throughout the undergraduate program, involving concepts from control theory, dynamic systems modeling, embedded systems, and microcontroller programming. Furthermore, besides enabling the study and validation of digital control strategies in an experimental environment, its implementation is expected to promote the documentation and dissemination of techniques associated with the Hardware-in-the-Loop approach, expanding access to this technology in teaching and research activities.

Keywords: embedded digital control; power electronics; hardware-in-the-loop; embedded systems; buck converter.

LISTA DE ILUSTRAÇÕES

Figura 1 – Comparação entre o Ciclo em V tradicional e sua evolução baseada em MBD **20**
Figura 2 – Topologia típica de um simulador HiL **21**
Quadro 1 – Síntese das vantagens e limitações da simulação *Hardware-in-the-Loop* (HiL) **22**
Figura 3 – Diagrama simplificado da arquitetura proposta do sistema HIL **25**
Quadro 2 – Cronograma de desenvolvimento das atividades do projeto **31**
Figura 4 – Diagrama de blocos do sistema de controle em malha fechada do sistema **32**
Figura 5 – Topologia do circuito conversor Buck **33**
Figura 6 – Circuito equivalente com a chave fechada **33**
Figura 7 – Circuito equivalente com a chave aberta **34**
Figura 8 – Conversor Buck com teste de degrau na razão cíclica, simulado no software LTspice **38**
Figura 9 – Comparação da corrente do indutor: LTspice vs. modelo médio linearizado com offset CC **39**
Figura 10 – Comparação da tensão de saída: LTspice vs. modelo médio linearizado com offset CC **39**
Figura 11 – Comparação da corrente no indutor (i_L) entre os métodos de Euler e Trapézio **44**
Figura 12 – Comparação da tensão no capacitor (v_C) entre os métodos de Euler e Trapézio **44**
Figura 13 – Comparação da corrente no indutor (i_L) entre os métodos de Euler e Trapézio em regime transitório (0 a 0,12 ms) **45**
Figura 14 – Comparação da tensão no capacitor (v_C) entre os métodos de Euler e Trapézio em regime transitório (0 a 0,12 ms) **45**
Figura 15 – Comparação da corrente no indutor (i_L) entre os métodos de Euler e Trapézio em regime permanente (0,2 a 0,225 ms) **46**
Figura 16 – Comparação da tensão no capacitor (v_C) entre os métodos de Euler e Trapézio em regime permanente (0,2 a 0,225 ms) **46**
Figura 17 – Erro relativo percentual médio da corrente (i_L) e da tensão (v_C) em função do fator n, tendo o QSpice como referência **47**
Figura 18 – Diagrama de Bode do filtro passa-baixas antialiasing **48**
Figura 19 – Topologia de um filtro passa-baixa Sallen-Key de segunda ordem **49**
Figura 20 – Posição dos polos e zeros do sistema em função da variação da potência nominal **52**
Figura 21 – Resposta transitória do sistema em função da variação da potência nominal **53**
Figura 22 – Circuito do conversor Buck em malha fechada implementado no QSPICE **55**
Figura 23 – Comparação do Sinal v_C: Simulação Numérica ($n = 10$) vs. QSPICE **56**
Figura 24 – Comparação do Sinal i_L: Simulação Numérica ($n = 10$) vs. QSPICE **56**
Figura 25 – Erro relativo percentual médio em malha fechada da corrente (i_L), da tensão (v_C) e *duty cycle* (D) em função do fator n, tendo o QSPICE como referência **57**
Figura 26 – Comparação do Sinal v_C: Simulação Numérica ($n = 100$) vs. QSPICE **58**
Figura 27 – Comparação do Sinal i_L: Simulação Numérica ($n = 100$) vs. QSPICE **58**
Figura 28 – Comparação do Sinal v_C: Simulação Numérica ($n = 1000$) vs. QSPICE **59**
Figura 29 – Comparação do Sinal i_L: Simulação Numérica ($n = 1000$) vs. QSPICE **59**
Figura 30 – Interface gráfica do simulador configurada no modo de entrada por parâmetros de projeto. **61**
Figura 31 – Interface gráfica do simulador configurada no modo de entrada por componentes físicos. **61**
Figura 32 – Janela de monitoramento e recepção dos sinais de telemetria serial em tempo real. **62**
Figura 33 – Comparação do sinal de corrente no indutor i_L: Precisão Simples vs. Dupla **67**
Figura 34 – Comparação do sinal de tensão no capacitor v_C: Precisão Simples vs. Dupla **67**
Figura 35 – Circuito com simulação do simulador do Buck implementado em microcontrolador no QSPICE. **68**
Figura 36 – Comportamento completo da corrente no indutor e tensão no capacitor para diferentes resoluções do DAC. **71**
Figura 37 – Detalhamento da corrente no indutor (i_L) em diferentes etapas de operação em função da resolução do DAC. **72**
Figura 38 – Detalhamento da tensão no capacitor (v_C) em diferentes etapas de operação em função da resolução do DAC. **72**
Figura 39 – Erro relativo percentual médio em malha fechada da corrente (i_L) e da tensão (v_C) em função do número de bits do DAC, tendo uma simulação analógica QSPICE como referência. **73**
Figura 40 – Diagrama de transição de estados do firmware do simulador do conversor Buck. **78**
Figura 41 – Interface de configuração com os parâmetros da simulação do conversor Buck. **86**
Figura 42 – Sinais de saída da corrente no indutor e tensão do capacitor obtidos na simulação. **87**
Figura 43 – Simulação QSPICE da corrente no indutor e tensão no condensador do conversor Buck projetado. **95**

Figura 44 – Comparação entre a corrente no indutor do conversor Buck alternativo obtida no QSPICE e na simulação numérica para $n = 100$. **97**
Figura 45 – Comparação entre a tensão no capacitor do conversor Buck alternativo obtida no QSPICE e na simulação numérica para $n = 100$. **97**
Figura 46 – Diagrama de Bode da resposta em frequência do filtro antialiasing do conversor Buck alternativo. **98**
Figura 47 – Posição dos polos e zeros do sistema de controle do projeto alternativo para diferentes níveis de potência de operação. **99**
Figura 48 – Resposta transitória do sistema de controle do projeto alternativo para diferentes níveis de potência de operação. **99**
Figura 49 – Erro relativo percentual médio em malha fechada alternativo da corrente (i_L), da tensão (v_C) e *duty cycle* (D) em função do fator n, tendo o QSPICE como referência. **100**
Figura 50 – Comparação do Sinal i_L do conversor Buck alternativo: Simulação Numérica (n = 1000) vs. QSPICE. **100**
Figura 51 – Comparação do Sinal v_C do conversor Buck alternativo: Simulação Numérica (n = 1000) vs. QSPICE. **101**
Figura 52 – Comparação do Sinal i_L do conversor Buck alternativo: Simulação Numérica (n = 100) vs. QSPICE. **101**
Figura 53 – Comparação do Sinal v_C do conversor Buck alternativo: Simulação Numérica (n = 100) vs. QSPICE. **102**

LISTA DE TABELAS

Tabela 1 – Comparação da capacidade computacional e dos periféricos analógicos de microcontroladores STM32 para o projeto principal e o projeto alternativo. **75**
Tabela 2 – Dicionário de cabeçalhos do protocolo binário USB. **77**
Tabela 3 – Estrutura de campos da mensagem de configuração (`CONFIG`). **77**
Tabela 4 – Estrutura de campos da mensagem de telemetria (`DATA`). **78**
Tabela 5 – Resumo das transições de estado do firmware do simulador. **79**
Tabela 6 – Parâmetros de projeto e especificações do conversor Buck. **94**
Tabela 7 – Parâmetros de projeto e especificações do conversor Buck alternativo. **97**

LISTAGEM DE CÓDIGOS FONTE

Código 1 – Implementação do controlador no QSPICE. **54**
Código 2 – Configuração do temporizador do controlador PI. **63**
Código 3 – Rotina de interrupção do ADC para execução do controlador PI. **64**
Código 4 – Inicialização dos periféricos do controlador PI. **64**
Código 5 – Configuração do DAC e função de quantização do modelo do Buck no bloco Simulador-Buck no QSPICE. **69**
Código 6 – Implementação do algoritmo do controle PI no bloco Controlador no QSPICE. **70**
Código 7 – Código em C estruturado para simulação e estimativa de ciclos do conversor Buck. **74**
Código 8 – Laço principal de execução do firmware com abordagem sequencial (*bare-metal*). **79**
Código 9 – Rotina de recepção de dados USB por meio de um *callback* de interrupção. **80**
Código 10 – Função de processamento da máquina de estados do simulador. **80**
Código 11 – Função de configuração dinâmica das frequências dos temporizadores no *hardware*. **81**
Código 12 – Rotina de execução do passo de simulação computacional do conversor Buck. **82**
Código 13 – Rotina de interrupção responsável pelo enfileiramento dos dados de telemetria. **83**
Código 14 – Função de enfileiramento de dados de transmissão USB com proteção de seção crítica. .. **83**
Código 15 – Processamento assíncrono e transmissão dos dados enfileirados pela interface USB CDC. **84**
Código 16 – Rotina de execução do passo de simulação modificada para medição de ciclos de clock. . **85**
Código 17 – Configuração do dispositivo serial virtual. **104**
Código 18 – Processamento dos parâmetros de configuração no dispositivo virtual. **105**
Código 19 – Atualização do modelo discreto do conversor Buck. **105**
Código 20 – Quantização dos sinais para representação das saídas analógicas. **106**
Código 21 – Formação e transmissão da telemetria do dispositivo virtual. **106**

LISTA DE ABREVIATURAS E SIGLAS

A/D	Analógico-Digital
ADC	*Analog-to-Digital Converter*
CC	Corrente Contínua
CCM	*Continuous Conduction Mode*
D/A	Digital-Analógico
DAC	*Digital-to-Analog Converter*
DCM	*Discontinuous Conduction Mode*
ECU	*Electronic Control Unit*
FPGA	*Field Programmable Gate Array*
FPU	*Floating Point Unit*
GUI	*Graphical User Interface*
HIL	*Hardware-in-the-Loop*
I/O	*Input/Output*
IAG	Inteligência Artificial Generativa
LCK	Lei das Correntes de Kirchhoff
LTK	Lei das Tensões de Kirchhoff
MBD	*Model-Based Design*
PI	*Proportional-Integral*
PID	*Proportional-Integral-Derivative*
PWM	*Pulse Width Modulation*
RTDS	*Real-Time Digital Simulator*
RTOS	*Real-Time Operating System*
RX	*Receive*
SMPS	*Switched-Mode Power Supplies*
TX	*Transmit*
USB	*Universal Serial Bus*
VSCode	*Visual Studio Code*

SUMÁRIO

1 INTRODUÇÃO 15
1.1 Considerações Iniciais 15
1.2 Contextualização 15
1.3 Objetivos 16
1.3.1 Objetivo geral 16
1.3.2 Objetivos específicos 16
1.4 Justificativa 16
1.5 Estrutura do Trabalho 16
2 REVISÃO BIBLIOGRÁFICA 18
2.1 Desenvolvimento Baseado em Modelos e o Ciclo em V 18
2.2 Simulação Hardware-in-the-Loop 20
2.2.1 Definição e Histórico 20
2.2.2 Objetivos e Vantagens 21
2.2.3 Requisitos de Hardware 22
2.2.4 Plataformas HiL Comerciais 23
3 ARQUITETURA E METODOLOGIA PROPOSTA 25
3.1 Estrutura Proposta do Sistema 25
3.2 Componentes do Sistema 26
3.2.1 O Computador 26
3.2.2 O Controlador 26
3.2.3 O Simulador 27
3.3 Materiais e Ferramentas a Serem Utilizados 28
3.3.1 Plataformas de Hardware 28
3.3.2 Ferramentas de Desenvolvimento de Software 29
3.4 Procedimentos para Validação 29
3.5 Cronograma de Desenvolvimento 30
4 DESENVOLVIMENTO 32
4.1 Modelagem Matemática e Discretização do Conversor Buck CCM 32
4.1.1 Modelagem Matemática do Conversor Buck CCM 32
4.1.1.1 Primeiro Estágio: Chave Fechada ($0 < t \leq DT_s$) 33
4.1.1.2 Segundo Estágio: Chave Aberta ($DT_s < t \leq T_s$) 34
4.1.1.3 Modelo Médio Contínuo e Equação de Saída 35
4.1.1.3.1 Linearização e Funções de Transferência 35
4.1.1.3.2 Validação das Funções de Transferência 38
4.1.2 Representação Discreta dos Estágios Topológicos do Conversor Buck 39
4.1.2.1 Método de Euler (Forward) 40
4.1.2.2 Método do Segurador de Ordem Zero (ZOH) 41
4.1.2.3 Método do Trapézio 42
4.1.2.4 Validação da Representação Discreta 43
4.2 Projeto do filtro antialiasing 47
4.3 Projeto e discretização do controlador 50
4.3.1 Projeto do controlador 50
4.3.2 Discretização do controlador 53
4.4 Validação do Modelo em Malha Fechada 55
4.5 Plataforma de Monitoramento e Configuração do Simulador 60
4.6 Implementação do Firmware do Controlador 63
4.7 Implementação do Simulador Digital em Tempo Real (RTDS) 66
4.7.1 Análise do Impacto da Resolução da FPU no Desempenho do Simulador 67
4.7.2 Análise do Impacto da Resolução do DAC no Desempenho do Simulador 68
4.7.3 Dimensionamento Computacional e Seleção do Microcontrolador 74
4.7.4 Arquitetura e Projeto do Firmware do Simulador 77
4.7.5 Implementação do firmware do simulador 79
4.7.6 Testes do firmware do simulador 85
5 RESULTADOS 88
6 CONCLUSÃO 89
REFERÊNCIAS 90

APÊNDICES **92**
APÊNDICE A – Projeto do Conversor Buck **93**
APÊNDICE B – Alternativa de Projeto para o Conversor Buck e Controlador **96**
APÊNDICE C – Dispositivo serial virtual **103**

1 INTRODUÇÃO

O desenvolvimento de sistemas de controle digital embarcados tem se tornado cada vez mais relevante em aplicações de eletrônica de potência, automação e sistemas industriais. Em particular, conversores eletrônicos de potência, como os conversores CC-CC (Corrente Contínua para Corrente Contínua), são amplamente utilizados em diversas aplicações que exigem regulação eficiente de tensão e corrente. No entanto, o projeto e a validação de algoritmos de controle para esses sistemas podem envolver riscos e custos elevados quando realizados diretamente em protótipos físicos, especialmente nas etapas iniciais de desenvolvimento.

Nesse contexto, técnicas de HIL (*Hardware-in-the-Loop*) surgem como uma alternativa para teste e validação de controladores em ambientes seguros e controlados. A simulação HIL é caracterizada pela integração entre componentes reais de *hardware* e componentes simulados executados em tempo real, permitindo que o sistema de controle interaja com modelos computacionais que representam o processo físico. Nessa abordagem, o controlador é implementado em *hardware* real, enquanto a planta é representada por um modelo computacional, possibilitando a realização de testes mesmo quando o sistema físico não está disponível ou quando experimentos diretos seriam muito caros ou demorados (Isermann; Schaffnit; Sinsel, 1999).

Diante disso, o presente trabalho propõe o desenvolvimento de uma plataforma de teste baseada em HIL para validação de controladores digitais embarcados aplicados a conversores CC-CC, sendo utilizado como estudo de caso o conversor Buck. A escolha desse conversor deve-se à sua estrutura e funcionamento relativamente simples, além de apresentar um modelo matemático de baixa complexidade, o que facilita o desenvolvimento inicial da plataforma e a verificação do seu funcionamento.

1.1 Considerações Iniciais

Este trabalho contou com o suporte de ferramentas de Inteligência Artificial Generativa (IAG) em diferentes etapas. O modo *Web Search* das plataformas *Gemini* e *DeepSeek* foi utilizado na etapa de pesquisa bibliográfica, auxiliando na localização e identificação de fontes acadêmicas relevantes, como artigos científicos, periódicos e livros relacionados ao tema abordado. O *ChatGPT* foi empregado como ferramenta de apoio à escrita, especificamente na revisão ortográfica, adequação de estilo, melhoria da coesão textual e auxílio na tradução e compreensão de termos técnicos em língua inglesa presentes nas referências consultadas.

Ressalta-se que todo o conteúdo produzido com o auxílio dessas ferramentas foi submetido à revisão crítica e validação final pelo autor, que assume integral responsabilidade pela originalidade, precisão e veracidade do trabalho, em conformidade com o Art. 9º da Portaria CNPq nº 2.664/2026 (Brasil. Conselho Nacional de Desenvolvimento Científico e Tecnológico, 2026).

1.2 Contextualização

Diversos trabalhos na literatura têm explorado o uso de técnicas de HIL no desenvolvimento e validação de sistemas de controle aplicados à eletrônica de potência. De modo geral, essas abordagens utilizam modelos matemáticos de conversores implementados em tempo real, permitindo a emulação do comportamento da planta e a validação de algoritmos de controle sem a necessidade de protótipos físicos completos (Kirei *et al.*, 2023).

Além disso, é observado o uso de diferentes plataformas para implementação de sistemas HIL, desde soluções de baixo custo até plataformas comerciais de simulação em tempo real (Gaviria-cano *et al.*, 2025; Cha *et al.*, 2012).

De maneira geral, esses estudos demonstram que o uso de HIL contribui para a redução de custos, riscos e tempo de desenvolvimento, ao permitir a realização de testes em tempo real antes da implementação em sistemas físicos. Dessa forma, essa abordagem tem se consolidado como uma ferramenta relevante no desenvolvimento de sistemas de controle para conversores eletrônicos de potência.

1.3 Objetivos

Nesta seção são apresentados os objetivos deste trabalho, divididos em objetivo geral 1.3.1 e objetivos específicos 1.3.2.

1.3.1 Objetivo geral

Desenvolver uma plataforma de teste baseada em HIL para validação de controladores digitais embarcados aplicados a conversores CC-CC, utilizando o conversor do tipo Buck como caso de estudo para implementação e validação.

1.3.2 Objetivos específicos

- Modelar e discretizar matematicamente um conversor CC-CC do tipo Buck para utilização como planta de referência na simulação em tempo real;
- Definir e selecionar um microcontrolador adequado para a execução do modelo da planta em tempo real;
- Projetar a arquitetura de software da plataforma de teste baseada em HIL;
- Implementar um protótipo da plataforma HIL utilizando microcontroladores para simulação da planta em tempo real e para execução do controlador;
- Validar o funcionamento da plataforma por meio de testes com uma estratégia de controle digital aplicada a um conversor CC-CC, utilizando um conversor Buck como exemplo de aplicação.

1.4 Justificativa

O desenvolvimento de sistemas de controle digital embarcados envolve a integração de diferentes áreas da engenharia, como teoria de controle, modelagem de sistemas dinâmicos e programação de microcontroladores. Nesse contexto, a realização deste trabalho possibilita aplicar de forma integrada conhecimentos adquiridos ao longo do curso em uma aplicação prática relacionada ao controle de conversores eletrônicos de potência.

A utilização de uma plataforma de testes baseada em HIL permite aprofundar conceitos relacionados à simulação em tempo real e à validação de controladores digitais embarcados. Além disso, o desenvolvimento desse sistema favorece o estudo e a documentação das técnicas envolvidas, contribuindo para a disseminação desse conhecimento em ambientes acadêmicos e de pesquisa.

Essa proposta torna-se particularmente relevante diante do fato de que as plataformas HIL disponíveis comercialmente são predominantemente importadas, o que pode limitar seu emprego em atividades de ensino e pesquisa. Assim, a investigação de alternativas baseadas em componentes de ampla disponibilidade e menor custo busca ampliar o acesso a essa tecnologia e fortalecer sua utilização em aplicações acadêmicas voltadas à validação de estratégias de controle digital em conversores CC-CC.

1.5 Estrutura do Trabalho

Este trabalho está organizado em três capítulos principais, além desta introdução.

O Capítulo 2 apresenta a revisão bibliográfica que fundamenta o desenvolvimento da plataforma proposta. Inicialmente, a Seção 2.1 discute os conceitos de Desenvolvimento Baseado em Modelos (MBD), o Ciclo em V e a metodologia *X-in-the-Loop*. Na sequência, a Seção 2.2 aborda a simulação *Hardware-in-the-Loop* (HIL),

destacando seus objetivos, vantagens, requisitos de hardware e as principais plataformas comerciais utilizadas na área.

No Capítulo 3, é apresentada a modelagem do sistema proposto. A Seção 3.1 descreve a arquitetura geral da plataforma HIL e a interação entre seus componentes. Em seguida, a Seção 3.2 detalha a modelagem funcional do computador, do controlador e do simulador. Por fim, a Seção 3.3 apresenta os materiais e as ferramentas de hardware e software previstos para a implementação da plataforma.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo apresenta os fundamentos teóricos que embasam o desenvolvimento deste trabalho. Inicialmente, na Seção 2.1, são discutidos os conceitos de Desenvolvimento Baseado em Modelos (MBD), sua integração ao Ciclo em V e as etapas da metodologia *X-in-the-Loop*. Em seguida, a Seção 2.2 aborda a simulação HiL, apresentando sua definição, evolução histórica, objetivos, vantagens, requisitos de hardware e as principais plataformas comerciais utilizadas na validação de sistemas de controle em tempo real, incluindo aplicações em eletrônica de potência.

2.1 Desenvolvimento Baseado em Modelos e o Ciclo em V

A crescente complexidade dos sistemas de controle nas diversas áreas da engenharia exige métodos de projeto capazes de garantir maior confiabilidade, rastreabilidade e redução de falhas operacionais. Nesse contexto, metodologias estruturadas tornam-se fundamentais para organizar o fluxo de desenvolvimento e permitir a validação do comportamento do sistema desde as etapas iniciais do projeto.

O Desenvolvimento Baseado em Modelos (MBD) é uma metodologia consolidada que utiliza modelos computacionais como elemento central do processo de engenharia. Essa abordagem permite representar matematicamente o comportamento do sistema para análise, simulação e validação ainda na fase digital do projeto, possibilitando que funcionalidades, desempenho e confiabilidade sejam avaliados ao longo de todo o desenvolvimento (Samiee *et al.*, 2018). Dessa forma, estratégias de controle podem ser desenvolvidas e testadas sem a necessidade imediata do sistema físico real, reduzindo riscos de danos e desgaste dos equipamentos durante as fases iniciais de validação (Lambert *et al.*, 2025).

Na prática, a engenharia baseada em MBD utiliza modelos que representam não apenas a unidade de controle, mas também os elementos que interagem com ela, como sensores, atuadores, redes de comunicação e a própria planta do sistema (Brooks, 2020). Essa integração permite antecipar a identificação de falhas arquiteturais e problemas de integração ainda nas primeiras etapas do desenvolvimento. Como consequência, reduz-se significativamente o retrabalho e o custo de correção de erros quando comparado aos métodos tradicionais, nos quais muitas inconsistências são descobertas apenas durante a integração física do sistema (Brooks, 2020).

No contexto de conversores eletrônicos e eletrônica de potência, o MBD apresenta benefícios particularmente relevantes, principalmente na implementação digital de malhas de controle para fontes chaveadas (SMPS). A possibilidade de desenvolver e validar o controle digital sem depender inicialmente do estágio de potência físico acelera o processo de desenvolvimento, reduz custos experimentais e amplia a segurança durante os testes iniciais (Kirei *et al.*, 2023).

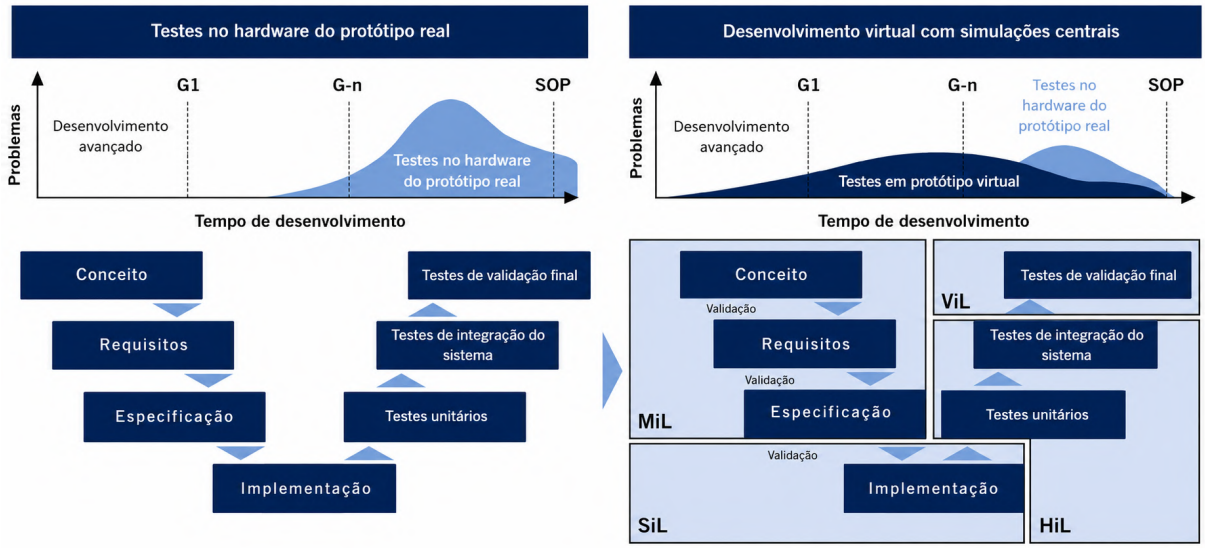
Para estruturar o fluxo de trabalho proporcionado pelo MBD e aplicá-lo de forma sistemática no desenvolvimento de sistemas embarcados, a engenharia tradicionalmente utiliza o modelo do Ciclo em V (Pestana; Miranda, 2024). Nesse modelo, o lado esquerdo do “V” concentra as etapas de especificação e projeto do sistema, enquanto o lado direito reúne as atividades de integração, verificação e validação. Na abordagem tradicional, o desenvolvimento normalmente não é orientado a modelos, fazendo com que a implementação de software e a integração com hardware ocorram predominantemente nas fases finais do projeto (Yang *et al.*, 2026). Como consequência, muitos problemas de integração e inconsistências funcionais acabam sendo identificados apenas durante os testes finais, aumentando retrabalho, custos de validação e dependência de protótipos físicos.

Com a incorporação do MBD, o Ciclo em V evoluiu para uma abordagem orientada a modelos, na qual diferentes níveis de validação são distribuídos ao longo de todo o desenvolvimento por meio da metodologia *X-in-the-Loop* (XiL). As primeiras verificações ocorrem em *Model-in-the-Loop* (MiL), etapa em que os algoritmos de controle são avaliados diretamente no ambiente de modelagem, permitindo validar o comportamento funcional ainda em nível matemático e conceitual (Brooks, 2020). Em seguida, no *Software-in-the-Loop* (SiL), o código gerado automaticamente ou implementado manualmente é executado em ambiente computacional para verificar se a implementação em software preserva o comportamento previamente validado no modelo (Brooks, 2020).

Posteriormente, a validação pode avançar para o *Hardware-in-the-Loop* (HiL), no qual o software embarcado interage com hardware real ou parcialmente real em tempo de execução, permitindo testes mais próximos das condições físicas de operação do sistema (Brooks, 2020; Yang *et al.*, 2026). Após essas etapas em ambiente controlado, o desenvolvimento pode seguir para testes em condições reais de operação, nos quais planta e controlador passam a atuar conjuntamente no sistema físico completo. Um exemplo dessa etapa final é o *Vehicle-in-the-Loop* (ViL), utilizado principalmente em aplicações automotivas, onde os algoritmos de controle são executados diretamente no veículo real para validar o comportamento global do sistema em condições reais de uso (Yang *et al.*, 2026; Kochem; Kemper; Istoc, 2023).

Essa evolução pode ser observada na Figura 1. No modelo tradicional, ilustrado à esquerda, grande parte da integração entre software e hardware ocorre apenas nas etapas finais do desenvolvimento, concentrando os testes próximos à conclusão do projeto e aumentando a dependência de protótipos físicos (Yang *et al.*, 2026). Em contraste, o modelo orientado a MBD, apresentado à direita da figura, distribui testes virtuais e validações contínuas ao longo de todo o processo de desenvolvimento, incorporando etapas MiL, SiL, HiL e ViL desde as fases iniciais até a integração final do sistema (Yang *et al.*, 2026; Samiee *et al.*, 2018; Brooks, 2020).

Figura 1 – Comparação entre o Ciclo em V tradicional e sua evolução baseada em MBD



Fonte: Adaptado de Kochem, Kemper e Istoc (2023, p. 75) e traduzido pelo autor.

2.2 Simulação Hardware-in-the-Loop

Esta seção apresenta os conceitos fundamentais da simulação HiL. Inicialmente, são abordadas sua definição e evolução histórica na Subseção 2.2.1. Em seguida, a Subseção 2.2.2 discute os objetivos, as vantagens e as limitações dessa técnica. Os requisitos de hardware necessários para a execução de simulações em tempo real são apresentados na Subseção 2.2.3, enquanto a Subseção 2.2.4 descreve as principais plataformas HiL comerciais utilizadas atualmente.

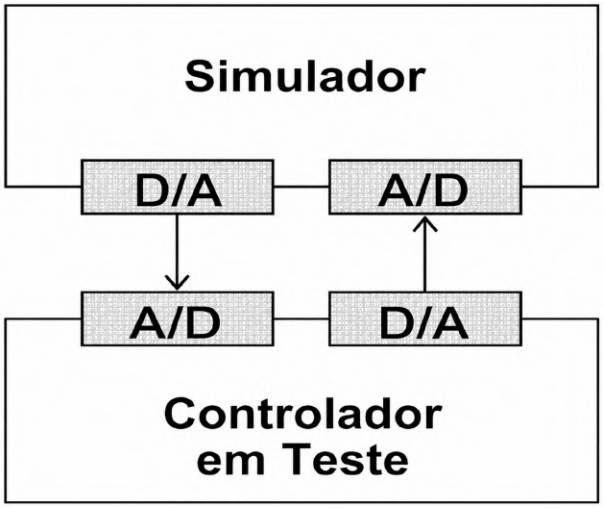
2.2.1 Definição e Histórico

A simulação HiL é uma técnica metodológica caracterizada pela operação conjunta de componentes físicos reais conectados a componentes simulados em tempo real (Isermann; Schaffnit; Sinsel, 1999). Em sua aplicação mais comum, o hardware e o software do sistema de controle representam a parte real do sistema, enquanto o processo a ser controlado, incluindo atuadores, sensores e as dinâmicas físicas envolvidas, é simulado de forma total ou parcial (Isermann; Schaffnit; Sinsel, 1999).

O princípio fundamental do HiL consiste em inserir uma réplica física e fiel de um subsistema, geralmente o controlador, em uma simulação virtual de malha fechada que representa os demais subsistemas (Mihalič; Truntič; Hren, 2022). Nesse arranjo bidirecional, o controlador recebe sinais físicos de entrada e envia comandos ao ambiente virtual como se estivesse operando a planta real. Essa separação permite submeter o equipamento a condições extremas de operação e a cenários de falha de forma segura, reprodutível e sem os custos associados a possíveis danos ao sistema físico completo (Mihalič; Truntič; Hren, 2022; Isermann; Schaffnit; Sinsel, 1999). A Figura 2 apresenta, de forma esquemática, a interação entre a unidade de controle e a planta simulada, destacando o fluxo de sinais e as principais interfaces de hardware presentes em um simulador HiL clássico.

Historicamente, as origens da simulação HiL remontam às indústrias aeroespacial e de defesa. As primeiras aplicações da técnica foram desenvolvidas para simulações de voo, inicialmente voltadas à emulação de painéis e instrumentos em cabines fixas de treinamento ainda na década de 1930 (Isermann; Schaffnit; Sinsel, 1999). Nas décadas seguintes, instituições como a NASA expandiram o uso dessa tecnologia no desenvolvimento de subsistemas de aeronaves de alta manobrabilidade e em testes de segurança para sistemas críticos de orientação de mísseis (Mihalič; Truntič; Hren, 2022).

Figura 2 – Topologia típica de um simulador HiL



Fonte: Adaptado de Cha *et al.* (2012, p. 1) e traduzido pelo autor.

Com a evolução e a redução do custo do poder computacional, o HiL passou gradualmente a ser adotado em outros setores da engenharia. Sua difusão para as indústrias automotiva e de eletrônica de potência foi impulsionada principalmente pelo aumento da complexidade dos controladores modernos e pelas rigorosas exigências de segurança associadas a esses sistemas (Lambert *et al.*, 2025). Atualmente, a ampla utilização de semicondutores de comutação rápida e de arquiteturas embarcadas distribuídas tornou os testes físicos de integração cada vez menos eficientes diante de cronogramas de desenvolvimento mais curtos (Pestana; Miranda, 2024; Mihalič; Truntič; Hren, 2022).

Nesse contexto, o HiL consolidou-se como um elemento central dentro da metodologia MBD (Lambert *et al.*, 2025). Inserido no ramo ascendente de verificação do Ciclo em V, ele representa a etapa de consolidação das abordagens *X-in-the-Loop*. Enquanto as fases anteriores validam modelos (MiL) e códigos de software (SiL), o HiL adiciona o nível final de abstração ao integrar o código executando diretamente no hardware alvo da Unidade de Controle Eletrônico (ECU), em comunicação, em tempo real, com as dinâmicas elétricas e mecânicas da planta modelada (Brooks, 2020).

2.2.2 Objetivos e Vantagens

O principal objetivo da simulação HiL é fornecer um ambiente rigoroso e altamente realista para a validação de sistemas de controle embarcados. Essa abordagem baseia-se na substituição da planta física por modelos matemáticos executados em tempo real, permitindo que a ECU interaja com sensores e atuadores simulados sob diferentes condições operacionais de forma segura, repetível e economicamente viável (Cha *et al.*, 2012).

Ao transferir o ambiente de testes da planta física para o laboratório, a simulação HiL consolida-se como uma ferramenta estratégica no ciclo de desenvolvimento, oferecendo vantagens significativas em termos de custo, tempo e flexibilidade. Entre os principais benefícios destacam-se a redução substancial de custos e a aceleração do processo de desenvolvimento, uma vez que o *design* e os testes do *hardware* e do *software* de controle podem ser conduzidos paralelamente ao desenvolvimento da planta, sem a necessidade de operar o sistema físico definitivo (Isermann; Schaffnit; Sinsel, 1999).

Outro aspecto relevante é a possibilidade de validação precoce. Inserida no contexto do MBD, a técnica HiL contribui para uma integração mais eficiente entre *hardware* e *software*, especialmente pela reutilização sistemática de modelos e casos de teste. Nesse cenário, os mesmos cenários de falha e testes funcionais empregados nas etapas iniciais de simulação, como MiL e SiL, podem ser reaproveitados diretamente na validação final da ECU de produção. Como consequência, defeitos arquiteturais e incompatibilidades de código tendem a ser identificados em fases mais iniciais do projeto, reduzindo custos e prazos associados ao retrabalho (Brooks, 2020).

No contexto da eletrônica de potência, as vantagens da simulação HiL tornam-se ainda mais relevantes devido às dinâmicas extremamente rápidas e aos elevados níveis de energia envolvidos. O uso de plataformas HiL em tempo real reduz significativamente os riscos financeiros e operacionais normalmente presentes durante a fase experimental em laboratório, evitando situações em que falhas na lógica de controle possam causar danos físicos à etapa de potência e aos dispositivos semicondutores (Cha *et al.*, 2012; Isermann; Schaffnit; Sinsel, 1999).

Para sintetizar de forma comparativa os principais benefícios e limitações dessa abordagem, o Quadro 1 apresenta um resumo das características gerais das plataformas HiL modernas.

Quadro 1 – Síntese das vantagens e limitações da simulação HiL

Vantagens e Benefícios	Desafios e Limitações
Natureza não destrutiva e segurança: Permite testar condições severas, falhas críticas e situações anormais sem comprometer a integridade de equipamentos ou operadores (Mihalič; Truntič; Hren, 2022).	**Efeito caixa preta:** O emulador normalmente não fornece acesso direto às variáveis internas ou ao estado detalhado da memória do controlador em teste (Mihalič; Truntič; Hren, 2022).
Repetibilidade e estabilidade: Possibilita a repetição de ensaios sob as mesmas condições iniciais e estímulos virtuais, garantindo maior consistência experimental (Mihalič; Truntič; Hren, 2022).	**Tempo de configuração:** A preparação de cenários complexos de teste pode demandar mais tempo do que a própria execução dos ensaios (Mihalič; Truntič; Hren, 2022).
Prototipagem rápida e redução de custos: Acelera o ciclo de validação *shift-left*, reduzindo a necessidade de sucessivos protótipos físicos (Mihalič; Truntič; Hren, 2022; Brooks, 2020).	**Limitações de hardware em tempo real:** Há um compromisso entre os recursos computacionais disponíveis e o nível de precisão numérica exigido para manter a estabilidade da simulação (Guo *et al.*, 2018).

Fonte: Elaborado pelo autor com base em Mihalič, Truntič e Hren (2022), Brooks (2020) e Guo *et al.* (2018).

Em síntese, embora a implementação da simulação HiL exija esforço computacional significativo e enfrente limitações relacionadas aos recursos de *hardware* em tempo real, essa abordagem oferece o rigor necessário para validar arquiteturas complexas de controle antes de sua aplicação definitiva em sistemas de potência.

2.2.3 Requisitos de Hardware

Para que a metodologia HiL consiga reproduzir com fidelidade o comportamento de um sistema físico, a plataforma computacional utilizada deve atender a requisitos rigorosos de processamento em tempo real, determinismo temporal e baixa latência. Em essência, um simulador HiL é caracterizado pela interação bidirecional em malha fechada entre os modelos virtuais da planta e os controladores físicos conectados ao sistema (Mihalič; Truntič; Hren, 2022).

Nesse contexto, a comunicação entre o ambiente virtual e o hardware físico depende diretamente da presença de conversores Analógico-Digitais (A/D) e Digital-Analógicos (D/A) capazes de operar sem introduzir gargalos no processamento. Sinais físicos, como comandos de *duty cycle* gerados por modulação PWM, precisam ser capturados, processados pelo modelo matemático e posteriormente convertidos novamente em sinais analógicos de tensão ou corrente. Para garantir coerência temporal entre o modelo computacional e os estímulos elétricos aplicados ao controlador, a atualização dos conversores D/A deve permanecer rigidamente sincronizada com o período de amostragem da simulação (Gaviria-cano *et al.*, 2025).

Na eletrônica de potência, o desempenho de uma plataforma HiL está fortemente relacionado à razão entre a frequência de chaveamento do controlador físico e a frequência de simulação, correspondente ao inverso do passo de integração (*time step*). De modo geral, a frequência de simulação precisa ser significativamente superior à frequência de controle. A literatura estabelece como referência prática a utilização de, no mínimo, dez passos de integração por período de chaveamento, de modo a preservar a fidelidade das formas de onda simuladas e evitar oscilações sub-harmônicas causadas por *aliasing* (Gaviria-cano *et al.*, 2025).

Essa exigência impõe restrições severas ao hardware. Em um conversor operando a 100 kHz, por exemplo, cujo período de chaveamento é de 10 μs, o processador dispõe de apenas 1 μs para resolver todo o sistema de equações do modelo e atualizar as interfaces de entrada e saída (Gaviria-cano *et al.*, 2025).

Quando a discretização é inadequada, os impactos sobre a estabilidade tornam-se significativos. Caso a taxa de amostragem se aproxime demasiadamente do período de chaveamento do conversor, o simulador deixa de reconstruir corretamente o *duty cycle*. Esse problema tende a se intensificar durante transitórios rápidos de carga, resultando em oscilações sub-harmônicas que comprometem a validade experimental dos ensaios de controle (Gaviria-cano *et al.*, 2025).

A definição do passo de integração envolve, portanto, um importante compromisso computacional. Embora passos menores reduzam os erros de truncamento associados à discretização numérica, eles também elevam substancialmente a quantidade de operações executadas por segundo. Em plataformas com limitação de precisão numérica, isso pode intensificar erros de quantização relacionados ao comprimento de palavra do hardware (Guo *et al.*, 2018).

Para atender a essas restrições de paralelismo e latência, as plataformas HiL comerciais tradicionalmente utilizam arquiteturas baseadas em matrizes de portas programáveis (FPGA) e Processadores Digitais de Sinais (DSP) (Mihalič; Truntič; Hren, 2022). Em particular, os FPGAs oferecem elevado paralelismo computacional, permitindo a execução de simulações em tempo real na escala de nanossegundos mesmo em sistemas elétricos complexos (Guo *et al.*, 2018).

Como alternativa mais acessível, microcontroladores modernos com arquiteturas avançadas, como dispositivos ARM Cortex-M7 operando entre 600 MHz e 1 GHz, têm demonstrado capacidade suficiente para executar cálculos em ponto flutuante e gerenciar periféricos em passos fixos da ordem de 1 μs. Dessa forma, tornam-se soluções viáveis para plataformas HiL voltadas a aplicações de pesquisa e ensino (Gaviria-cano *et al.*, 2025).

2.2.4 Plataformas HiL Comerciais

No contexto do desenvolvimento e validação de sistemas de controle aplicados à eletrônica de potência, a implementação da metodologia HiL está associada ao uso de um Simulador Digital em Tempo Real (RTDS). Enquanto o HiL define a natureza do ensaio em malha fechada, o RTDS corresponde à infraestrutura computacional responsável por emular a planta em tempo real. Diferentes controladores podem ser avaliados por meio de sua conexão ao RTDS, sendo comum que os algoritmos de controle sejam implementados em linguagem C e embarcados em DSPs ou microcontroladores dedicados (Cha *et al.*, 2012).

Atualmente, empresas como OPAL-RT, Typhoon HIL, dSPACE, RT Box, Speedgoat e National Instruments oferecem soluções comerciais baseadas nesse princípio (Mihalič; Truntič; Hren, 2022; Gaviria-cano *et al.*, 2025). Esses ecossistemas combinam hardware e software especializados para aplicações de pesquisa e desenvolvimento, destacando-se pela precisão e pela capacidade de simulação em tempo real (Gaviria-cano *et al.*, 2025).

A integração dessas plataformas com ferramentas como o MATLAB Simulink Real-Time possibilita a transição direta entre a modelagem virtual e os testes em hardware, com elevado nível de automação (Mihalič; Truntič; Hren, 2022). A partir de modelos matemáticos e diagramas de blocos, essas ferramentas geram e executam códigos em tempo real, reduzindo a necessidade de programação manual em baixo nível (Mihalič; Truntič; Hren, 2022). Além disso, oferecem arquiteturas escaláveis e interfaces de entrada e saída (I/O) de fácil configuração, mantendo baixa latência e bom desempenho computacional (Mihalič; Truntič; Hren, 2022).

Apesar da robustez dessas soluções, sua adoção ainda enfrenta limitações importantes. O elevado custo de aquisição restringe sua utilização em ambientes acadêmicos e grupos de pesquisa com orçamento reduzido (Gaviria-cano *et al.*, 2025). Soma-se a isso a curva de aprendizado associada à configuração dessas plataformas e à programação de FPGAs proprietários, frequentemente exigindo treinamento especializado (Gaviria-cano *et al.*, 2025).

Em aplicações acadêmicas, entretanto, nem sempre são necessárias resoluções temporais na faixa de nanossegundos, típicas de plataformas industriais de alto desempenho. Muitos estudos voltados à emulação de conversores estáticos podem ser adequadamente representados com passos de simulação na ordem de microssegundos (Gaviria-cano *et al.*, 2025).

Nesse cenário, pesquisas recentes têm explorado plataformas HiL modulares, de menor custo e maior flexibilidade, baseadas em componentes comerciais amplamente disponíveis. Soluções baseadas em microcontroladores de arquitetura avançada, como o ARM Cortex-M7, capaz de operar com frequências próximas de 1 GHz, têm se destacado como alternativas computacionais promissoras (Gaviria-cano *et al.*, 2025).

Com uma arquitetura mais enxuta, essas plataformas reduzem significativamente as barreiras financeiras e técnicas associadas à implementação de sistemas HiL. Como consequência, tornam-se particularmente atrativas para estudantes e pesquisadores, facilitando a integração em ambientes de ensino e em projetos experimentais preliminares, sem comprometer a capacidade de representar, em tempo real, o comportamento dinâmico de conversores CC-CC (Gaviria-cano *et al.*, 2025).

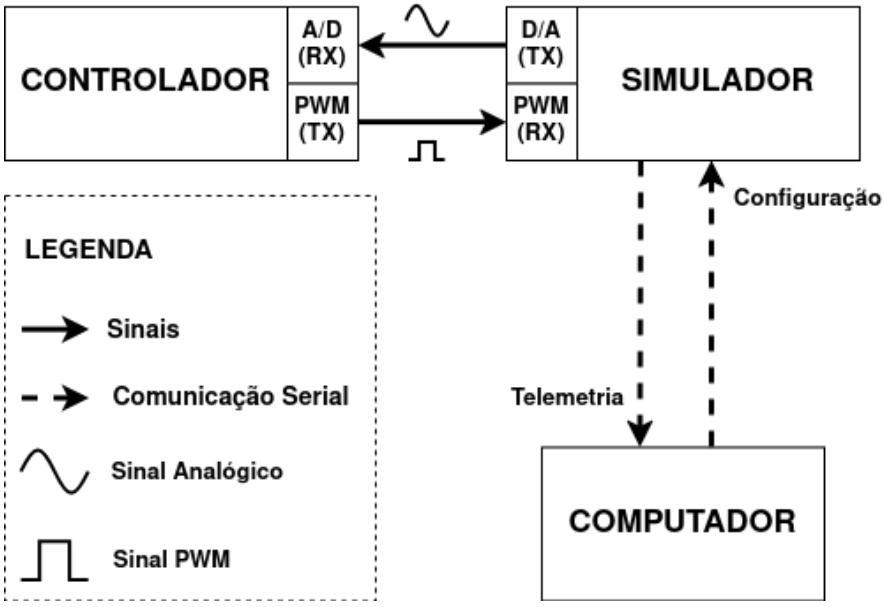
3 ARQUITETURA E METODOLOGIA PROPOSTA

Este capítulo apresenta a arquitetura proposta para a plataforma HIL, descrevendo sua estrutura geral e a interação entre os módulos que compõem o sistema. Inicialmente, na Seção 3.1, são apresentados a organização da plataforma e o fluxo de comunicação entre os elementos envolvidos na emulação. Em seguida, a Seção 3.2 descreve o funcionamento dos principais componentes do sistema, detalhando as funções e responsabilidades do computador hospedeiro, do controlador digital e do simulador em tempo real. A Seção 3.3 apresenta os materiais e as ferramentas de hardware e software previstos para a implementação da plataforma. A Seção 3.4 descreve a metodologia de validação, detalhando os procedimentos que serão empregados para verificar o correto funcionamento do sistema. Por fim, a Seção 3.5 apresenta o cronograma das atividades a serem realizadas.

3.1 Estrutura Proposta do Sistema

A arquitetura proposta para a plataforma HIL fundamenta-se nos conceitos de MBD previamente discutidos e é composta por três elementos principais: o computador, o simulador e o controlador. A plataforma foi desenvolvida para a validação e o teste de controladores aplicados a conversores CC-CC, possibilitando a emulação, em tempo real, da dinâmica da planta e sua interação direta com o hardware de controle. Em especial, para validação neste trabalho, será considerado um conversor do tipo Buck. A Figura 3 apresenta o diagrama que ilustra a organização estrutural da plataforma e a interação entre seus componentes.

Figura 3 – Diagrama simplificado da arquitetura proposta do sistema HIL



Fonte: Elaborado pelo autor.

No nível de supervisão da plataforma encontra-se o **computador**, responsável pela configuração e parametrização do modelo matemático da planta, bem como pela disponibilização de ferramentas para monitoramento contínuo do sistema. A comunicação entre o computador e o simulador ocorre por meio de uma interface serial implementada sobre o barramento *Universal Serial Bus* (USB), permitindo tanto o envio dos parâmetros de configuração quanto a recepção dos dados de telemetria gerados durante os ensaios.

O segundo elemento da arquitetura é o **simulador**, correspondente ao núcleo computacional do RTDS. Sua função consiste em executar, em tempo real, o modelo matemático da planta em estudo, preservando sua dinâmica ao longo da simulação. A partir dos parâmetros elétricos fornecidos pelo computador e do *duty cycle* proveniente do controlador, o simulador atualiza iterativamente as variáveis de estado, reproduzindo o comportamento do conversor e enviando os dados de telemetria ao computador hospedeiro por meio da interface USB.

Fechando a malha de operação, o **controlador** representa a ECU submetida à validação. Esse módulo, geralmente implementado por meio de um microcontrolador ou DSP, é responsável pela execução nativa dos algoritmos de controle. Durante os testes, o controlador opera de maneira equivalente à empregada em uma aplicação física real, recebendo as variáveis simuladas da planta e gerando sinais de Modulação por Largura de Pulso (PWM) utilizados no acionamento do conversor.

A comunicação bidirecional entre o simulador e o controlador ocorre por meio da troca de sinais analógicos e digitais. As variáveis de estado calculadas pelo simulador são convertidas em sinais analógicos por meio dos conversores Digital-Analógico (D/A) e adquiridas pelo controlador através de seus conversores Analógico-Digital (A/D). Em sentido contrário, o controlador envia ao simulador os comandos de acionamento por meio de sinais PWM, os quais são capturados pelos temporizadores do simulador para a determinação do *duty cycle* aplicado ao modelo matemático. Dessa forma, a interação entre os módulos reproduz a troca de sinais observada em um sistema físico real.

3.2 Componentes do Sistema

A implementação da plataforma HIL proposta requer a integração de diferentes elementos responsáveis pela configuração do sistema, execução da estratégia de controle e emulação da planta em tempo real. Nesse contexto, esta seção apresenta a arquitetura funcional adotada para o desenvolvimento da plataforma. A Subseção 3.2.1 descreve as funções do computador hospedeiro, a Subseção 3.2.2 apresenta o controlador digital empregado na malha de controle e a Subseção 3.2.3 aborda a estrutura prevista para o simulador em tempo real.

3.2.1 O Computador

Como mostrado na Figura 3, a arquitetura da plataforma HIL proposta adotará o computador como principal interface entre o usuário e o sistema. Durante a operação, este desempenhará funções centrais relacionadas à parametrização inicial do simulador e à recepção contínua dos dados de telemetria.

Na etapa inicial, o computador realizará a configuração do modelo virtual por meio de uma interface serial via USB, enviando ao microcontrolador responsável pela simulação os parâmetros elétricos da planta, tais como tensão de entrada, resistência de carga, indutância e capacitância. Essa parametrização prévia tornará o processo de teste mais flexível, permitindo a modificação das características do sistema diretamente via *software*, sem a necessidade de alterações no *firmware* do simulador a cada novo ensaio.

Durante a execução da malha fechada, o computador passará a atuar também na aquisição dos dados de telemetria enviados pelo simulador via USB. Esses dados poderão ser armazenados continuamente e utilizados para a geração de gráficos e análises do comportamento dinâmico do sistema. Dessa forma, será possível avaliar tanto o desempenho transitório quanto o regime permanente da malha de controle, fornecendo base para a validação da estratégia de controle digital implementada.

3.2.2 O Controlador

Para a implementação da malha de regulação do conversor CC-CC neste trabalho, será utilizado um controlador digital de malha fechada, cujo algoritmo será definido durante a etapa de desenvolvimento, considerando as alternativas Proporcional-Integral (PI) e Proporcional-Integral-Derivativo (PID). Ambos são amplamente utilizados na engenharia de sistemas de controle, destacando-se pela versatilidade e aplicabilidade em diferentes classes de plantas dinâmicas (Ogata, 2009).

Neste projeto, o controlador digital será embarcado diretamente no microcontrolador sob teste e operará sincronizado à frequência de controle e chaveamento de $F_c = 50\,\text{kHz}$. A escolha dessa frequência justifica-se pela necessidade de manter o chaveamento acima da faixa de audição humana, evitando a geração de ruídos acústicos indesejáveis durante a operação do conversor (Hart, 2010). Além disso, frequências mais elevadas reduzem a

necessidade de armazenamento de energia por ciclo, permitindo a utilização de componentes passivos menores, como indutores e capacitores, o que contribui para a redução do tamanho e do peso do circuito (Hart, 2010).

Embora a planta de potência não seja construída fisicamente no escopo deste trabalho, essa definição de frequência permanece relevante, pois possibilita que, em possíveis etapas futuras do desenvolvimento, o sistema possa ser migrado do ambiente de simulação para uma implementação física destinada à validação experimental.

Conforme ilustrado na Figura 3, o controlador receberá do simulador o valor da tensão de saída do conversor, utilizado como variável de realimentação da malha de controle. A partir dessa informação, o sinal de erro será calculado pela diferença entre a referência desejada e a saída medida da planta (Ogata, 2009).

O algoritmo de controle processará continuamente o sinal de erro para determinar a ação de regulação necessária. A estrutura adotada poderá combinar as ações proporcional, integral e derivativa, conforme a configuração selecionada durante o desenvolvimento. A ação proporcional responderá à magnitude do erro atual, enquanto a ação integral acumulará o erro ao longo do tempo, contribuindo para eliminar desvios em regime permanente. Caso seja utilizada, a ação derivativa considerará a taxa de variação do erro, contribuindo para o ajuste da resposta transitória do sistema (Ogata, 2009).

A atuação do controlador sobre a planta será realizada por meio de um sinal PWM, caracterizado por uma forma de onda retangular de frequência constante cuja largura do pulso é ajustada dinamicamente. O ciclo ativo corresponde à razão entre o intervalo em que o sinal permanece em nível lógico alto e o período total da forma de onda, permitindo controlar diretamente a tensão média aplicada ao conversor (Angélico; Neves, 2023). Assim, a cada período de amostragem, o algoritmo de controle calculará o esforço de regulação necessário para atualizar a razão cíclica do PWM enviado ao simulador. Dessa maneira, será possível regular a transferência de energia da planta e conduzir a tensão de saída ao valor de referência, buscando desempenho adequado tanto em regime permanente quanto durante os transitórios do sistema.

3.2.3 O Simulador

O desenvolvimento de um simulador computacional requer, inicialmente, uma representação matemática consistente do sistema físico a ser emulado (Angélico; Neves, 2023). Neste trabalho, o simulador tem como objetivo representar o comportamento dinâmico de conversores CC-CC, sendo adotado o conversor Buck como caso de estudo para a implementação e validação da plataforma HIL em tempo real.

O conversor Buck é uma topologia de conversor CC-CC do tipo abaixador de tensão, o que significa que a tensão média em sua saída será sempre menor ou igual à tensão de entrada (Rashid, 2014; Hart, 2010). Devido à sua estrutura simples e à elevada eficiência no processo de conversão de energia, essa topologia é amplamente empregada em aplicações de eletrônica de potência, fontes de alimentação chaveadas e sistemas que demandam regulação de tensão (Rashid, 2014).

No contexto da teoria de sistemas de controle, a representação matemática do conversor será expressa por meio de sua função de transferência, a qual descreverá a relação dinâmica entre as variáveis de entrada e saída da planta. Nesse sentido, a etapa inicial da concepção do simulador HIL consistirá na obtenção e posterior discretização da função de transferência do conversor Buck, com ênfase no modelo linear associado às variações da razão cíclica (Mehl, 2026), permitindo a transição do comportamento contínuo do circuito de potência para uma representação discreta no tempo (Angélico; Neves, 2023).

Em seguida, o modelo discretizado será implementado na forma de equações de diferenças no *firmware* de um microcontrolador dedicado à execução da simulação, o qual atuará como núcleo computacional responsável por reproduzir, em tempo real e de forma determinística, a dinâmica do conversor.

A partir da arquitetura apresentada na Figura 3, o simulador receberá como entrada o sinal PWM proveniente do controlador, além dos parâmetros elétricos de configuração do conversor, como resistência, indutância e capacitância (Hart, 2010). Com base nessas informações, as variáveis de estado serão atualizadas iterativamente a cada passo de simulação, permitindo a emulação contínua da resposta dinâmica da planta.

Para garantir a fidelidade da simulação e reduzir erros associados à discretização numérica, o período de integração numérica T_{sim} deverá ser, no mínimo, dez vezes menor que o período de chaveamento do conversor T_s (Gaviria-cano *et al.*, 2025). Como o valor de F_s adotado é de $50\,\text{kHz}$, a taxa mínima de atualização da simulação deve ser de $500\,\text{kHz}$, correspondente a um período de atualização $T_{\text{sim}} = 1/F_{\text{sim}} = 2\,\mu\text{s}$.

Por fim, o simulador realizará a transmissão das variáveis de estado relevantes ao computador hospedeiro por meio da conexão USB, permitindo o monitoramento contínuo, a aquisição de dados e a análise do comportamento do sistema durante a execução da emulação.

3.3 Materiais e Ferramentas a Serem Utilizados

A implementação da plataforma HIL proposta requer a integração de diferentes recursos de hardware e software capazes de atender às restrições de tempo real impostas pela emulação da planta física e pela execução da malha de controle digital. Nesse contexto, esta seção apresenta os principais componentes previstos para utilização no desenvolvimento do sistema, descritos na Subseção 3.3.1, e as ferramentas de desenvolvimento de software, apresentadas na Subseção 3.3.2, bem como as características técnicas que motivaram sua escolha.

3.3.1 Plataformas de Hardware

A emulação de uma planta física em tempo real exige recursos computacionais capazes de executar cálculos matemáticos em intervalos extremamente reduzidos, enquanto a implementação da malha de controle demanda periféricos flexíveis para a aquisição e a geração contínua e precisa de sinais. Para o atendimento a esses requisitos, a seleção dos microcontroladores que comporão o sistema será realizada durante o desenvolvimento do projeto, por meio de análises de viabilidade e de capacidade computacional.

Para atuar como controlador digital em teste, será avaliado e selecionado um dispositivo adequado às demandas do algoritmo. Durante cada ciclo de controle, o microcontrolador deverá adquirir a tensão de saída proveniente do simulador por meio de conversores A/D integrados, cuja taxa de amostragem deve ser suficiente para favorecer a aquisição rápida e precisa do estado da planta. Em seguida, o algoritmo de controle calculará o esforço de regulação e atualizará continuamente o sinal PWM responsável pelo acionamento do simulador da planta. A necessidade de uma Unidade de Ponto Flutuante (FPU) integrada também será analisada, visto que esse recurso favorece a execução eficiente de algoritmos de controle em ponto flutuante.

Já a simulação da planta física em tempo real exigirá um microcontrolador com arquitetura compatível com a elevada carga matemática do modelo digital do conversor. A presença de uma FPU, associada a uma alta frequência de operação e à capacidade de memória adequada, será um critério central de seleção para garantir a execução dos cálculos numéricos nos intervalos exigidos pela plataforma HIL. Além disso, o hardware deverá dispor de conversores D/A integrados com taxas de atualização capazes de permitir a geração contínua dos sinais analógicos necessários para representar as variáveis da planta simulada. Em conjunto, esses recursos possibilitarão a implementação do modelo matemático da planta e sua integração com o controlador embarcado por meio da troca de sinais analógicos e digitais.

As análises a serem conduzidas ao longo do projeto determinarão os componentes definitivos, visando assegurar que os dispositivos selecionados atendam rigorosamente aos requisitos temporais da aplicação nas frequências de controle e simulação propostas. Além do desempenho técnico, a escolha dos equipamentos também poderá considerar a sua disponibilidade imediata, fator que contribui para a montagem ágil do protótipo e para a redução dos custos de aquisição.

3.3.2 Ferramentas de Desenvolvimento de Software

A infraestrutura lógica da plataforma compreende tanto o *firmware* embarcado, responsável pela execução determinística das malhas de simulação e controle, quanto o aplicativo computacional executado no hospedeiro, destinado à parametrização e supervisão do sistema.

O *firmware* dos microcontroladores STM32 será desenvolvido em linguagem C, utilizando o ambiente de desenvolvimento integrado *STM32CubeIDE* para compilação, organização e depuração do código. Para a configuração e o gerenciamento dos periféricos, será utilizada a biblioteca HAL (*Hardware Abstraction Layer*) (STMicroelectronics, 2026a).

Visando atender aos requisitos temporais da aplicação, não será utilizado um sistema operacional de tempo real (RTOS), uma vez que as operações críticas seguem um fluxo sequencial e determinístico, composto pela leitura das entradas, resolução das equações e atualização das saídas. Dessa forma, o gerenciamento direto das rotinas por temporizadores e interrupções de *hardware* permite simplificar a arquitetura do *software* e evitar a sobrecarga associada ao escalonamento de tarefas.

Paralelamente, o aplicativo computacional executado no computador hospedeiro será desenvolvido em linguagem *Python*, devido à ampla disponibilidade de bibliotecas e pacotes de código aberto voltados ao desenvolvimento científico, à comunicação de dados e à construção de interfaces gráficas de usuário (GUI). Essa escolha contribui para simplificar e acelerar o desenvolvimento da interface destinada à configuração remota dos parâmetros físicos da planta simulada e à visualização da telemetria do sistema.

Adicionalmente, a codificação estrutural, o gerenciamento dos ambientes virtuais e a depuração interativa do aplicativo serão realizados por meio do editor de código *Visual Studio Code* (*VSCode*) (Microsoft, 2026). A ferramenta foi selecionada em virtude da elevada produtividade proporcionada por sua arquitetura modular baseada em extensões, bem como pelos recursos integrados de depuração e identificação de erros de *software*.

3.4 Procedimentos para Validação

Para assegurar o correto funcionamento da plataforma HIL desenvolvida, será estabelecido um roteiro sistemático de validação dividido em quatro etapas progressivas. Essa abordagem permite isolar variáveis e garantir que eventuais falhas sejam identificadas e corrigidas em nível unitário antes da integração completa do sistema.

A primeira etapa consistirá na validação do modelo matemático discretizado do conversor Buck. Após a obtenção das equações de diferenças, será verificado se a resposta do modelo numérico reproduz adequadamente o comportamento dinâmico esperado da planta. Para isso, o modelo discretizado, implementado em ambiente computacional como *MATLAB* ou *Python*, terá suas respostas nos domínios do tempo e da frequência comparadas com os resultados obtidos por meio de uma simulação de referência desenvolvida no *QSPICE*. Ambas as simulações utilizarão os mesmos parâmetros elétricos, incluindo tensão de entrada, frequência de chaveamento e razão cíclica. A concordância entre os resultados permitirá verificar a exatidão das equações matemáticas utilizadas na emulação da planta. Além disso, a utilização do *QSPICE* possibilitará a futura integração do modelo da planta com a implementação do controlador digital, permitindo a validação do sistema completo em malha fechada em um ambiente único.

A segunda etapa terá como objetivo validar o *firmware* do simulador da planta. Após a validação computacional do modelo, as equações discretizadas serão embarcadas no microcontrolador responsável pela simulação. Inicialmente, os ensaios serão realizados em malha aberta mediante a aplicação de uma razão cíclica constante e previamente conhecida. O correto processamento das equações será avaliado por meio de ferramentas de depuração em hardware, da análise dos dados de telemetria transmitidos ao computador hospedeiro e da medição direta dos sinais analógicos gerados pelos conversores D/A utilizando um osciloscópio. O propósito desta etapa é verificar se as variáveis de estado e as saídas produzidas pelo *firmware* correspondem aos resultados teóricos obtidos durante a validação do modelo matemático.

A terceira etapa contemplará a validação isolada do *firmware* do controlador digital implementado no microcontrolador selecionado para essa finalidade. De maneira semelhante ao simulador, o algoritmo de controle adotado (PI ou PID) será inicialmente testado em malha aberta com o objetivo de verificar o funcionamento correto dos periféricos e das rotinas computacionais. Serão avaliadas a precisão e a periodicidade da aquisição dos sinais analógicos pelos conversores A/D, a execução correta da equação de diferenças da lei de controle e a geração adequada dos sinais PWM correspondentes ao esforço de controle calculado.

Por fim, a quarta etapa consistirá na validação integrada da plataforma HIL. Após a certificação individual dos módulos, o simulador e o controlador serão interconectados fisicamente para operar em malha fechada. Nessa condição, o desempenho global da plataforma será comparado aos resultados obtidos por meio de uma simulação de referência do sistema completo desenvolvida no *QSPICE*, na qual tanto o conversor Buck quanto o controlador digital serão implementados. A resposta do sistema será analisada com o objetivo de verificar a correta operação da estratégia de controle adotada e sua capacidade de manter a tensão de saída regulada. Para isso, serão avaliados parâmetros de desempenho como o comportamento transitório, o erro em regime permanente e a estabilidade da malha. Resultados compatíveis com aqueles obtidos na simulação de referência indicarão que a plataforma é capaz de emular o conversor Buck e validar estratégias de controle digital em tempo real.

3.5 Cronograma de Desenvolvimento

Para o desenvolvimento e a validação do projeto, conforme proposto nas Seções 3.2 e 3.4, as atividades foram organizadas em sete etapas principais. As principais tarefas associadas a cada etapa são descritas a seguir.

1. **Obtenção do modelo matemático do conversor CC-CC Buck:** compreende a modelagem e discretização da planta, além da avaliação da capacidade do hardware em atender aos requisitos temporais do sistema.

2. **Validação computacional do modelo:** consiste na implementação e verificação do modelo discretizado em ambiente computacional.

3. **Desenvolvimento e validação do simulador em tempo real:** envolve a implementação do modelo no microcontrolador e a validação do simulador em malha aberta.

4. **Desenvolvimento e validação do controlador digital:** contempla a implementação do algoritmo de controle e a verificação do funcionamento do *firmware*.

5. **Desenvolvimento do software do computador:** consiste na criação da interface para configuração do sistema, aquisição e visualização dos dados.

6. **Integração e validação da plataforma HIL:** compreende a operação do sistema em malha fechada e a avaliação do desempenho da plataforma.

7. **Análise e documentação dos resultados:** envolve a avaliação dos resultados obtidos e a elaboração do documento final do Trabalho de Conclusão de Curso.

A execução dessas etapas foi planejada para um período de seis meses, conforme apresentado no Quadro 2.

Quadro 2 – Cronograma de desenvolvimento das atividades do projeto

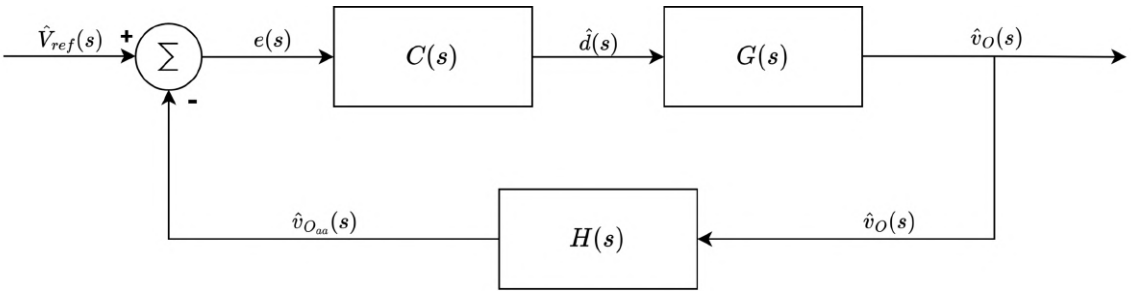
Atividade	Jul	Ago	Set	Out	Nov	Dez
1. Obtenção do modelo matemático do conversor CC-CC Buck	●					
2. Validação computacional do modelo	●	●				
3. Desenvolvimento e validação do simulador em tempo real		●	●			
4. Desenvolvimento e validação do controlador digital			●	●		
5. Desenvolvimento do software do computador			●	●		
6. Integração e validação da plataforma HIL					●	
7. Análise e documentação dos resultados	●	●	●	●	●	●

Fonte: Elaborado pelo autor.

4 DESENVOLVIMENTO

Neste capítulo são apresentadas as etapas de concepção, modelagem, dimensionamento e implementação da plataforma de simulação em tempo real HIL. A arquitetura do sistema em malha fechada, desenvolvida para a validação de algoritmos de controle a partir da emulação da planta física, é apresentada na Figura 4. A notação com o símbolo ˆ, utilizada nas variáveis representadas na figura, será definida posteriormente em 4.1.1.3.1.

Figura 4 – Diagrama de blocos do sistema de controle em malha fechada do sistema



Fonte: Elaborado pelo autor.

A estrutura em malha fechada da plataforma HIL é composta por três elementos principais: o controlador sob teste (C), responsável por calcular a ação de controle a partir do sinal de erro; a planta simulada em tempo real (G), que representa numericamente a dinâmica do conversor chaveado no simulador digital; e o filtro antialiasing analógico (H), utilizado na interface física para atenuar componentes de alta frequência e reduzir o risco de *aliasing* antes da digitalização pelo conversor analógico-digital (A/D). Esses elementos operam de forma integrada em um ciclo de realimentação, no qual o controlador C envia os comandos de acionamento para a planta G, enquanto os estados simulados são convertidos em sinais elétricos, passam pelo filtro H e são então digitalizados e realimentados ao controlador, fechando a malha de controle.

Para a realização das etapas de modelagem, projeto e validação da plataforma HIL, é adotado como caso de teste um conversor Buck com os parâmetros apresentados na Tabela 6, cujo projeto detalhado está demonstrado no Apêndice A. Esse mesmo conversor é utilizado ao longo do desenvolvimento, desde a obtenção e discretização do modelo matemático até o projeto do filtro antialiasing e do controlador, permitindo que os resultados obtidos nas diferentes etapas sejam avaliados sob condições de operação consistentes. As especificações consideradas incluem os parâmetros elétricos do conversor e as condições nominais de operação, definidas de modo a estabelecer uma referência comum para as análises e validações realizadas neste trabalho.

Para estruturar o desenvolvimento e a validação da plataforma, a Seção 4.1 trata da modelagem matemática contínua do conversor Buck e de sua respectiva formulação discreta em equações a diferenças, viabilizando a execução determinística em tempo real no microcontrolador da bancada. Em seguida, a Seção 4.2 aborda a especificação, a topologia e a resposta em frequência do filtro analógico antialiasing (H). Por fim, a Seção 4.3 apresenta a síntese, o ajuste de ganhos e a discretização do controlador (C), utilizado como caso de teste para demonstrar a fidelidade e a eficácia operacional da plataforma HIL desenvolvida.

4.1 Modelagem Matemática e Discretização do Conversor Buck CCM

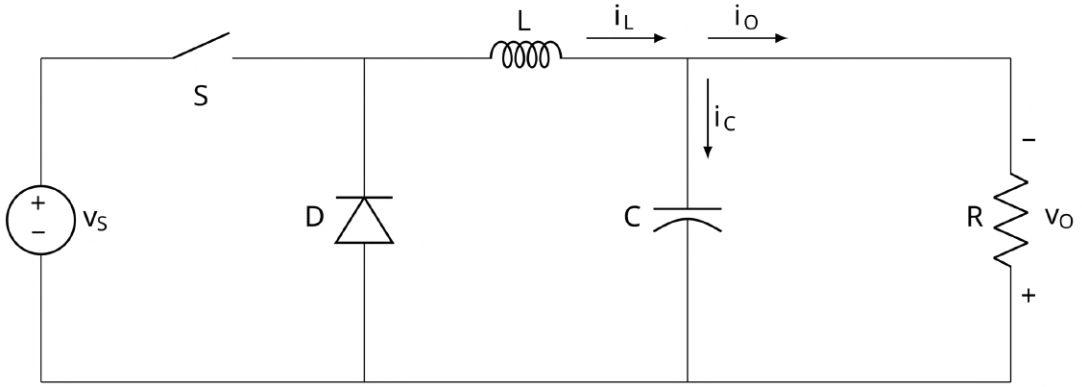
4.1.1 Modelagem Matemática do Conversor Buck CCM

A Figura 5 apresenta a topologia de um conversor Buck ideal, no qual a tensão de entrada $v_S(t)$ fornece energia ao circuito por meio do chaveamento em alta frequência da chave eletrônica (S). O filtro passa-baixa, constituído pelo indutor (L) e pelo capacitor (C), atenua as componentes de alta frequência geradas pelo chaveamento, fornecendo à carga (R) uma tensão com baixa ondulação. Como o capacitor está conectado em paralelo com a carga, a tensão de saída do conversor é igual à tensão sobre o capacitor, isto é, $v_O(t) = v_C(t)$.

Para a modelagem do conversor, é necessário distinguir as grandezas instantâneas das suas componentes médias. Dessa forma, ao longo deste trabalho, as variáveis representadas por letras minúsculas representam valores instantâneos dependentes do tempo, enquanto as letras maiúsculas indicam os respectivos valores médios em regime permanente. Por exemplo, v_O e i_L representam a tensão de saída e a corrente no indutor instantâneas, enquanto V_O e I_L correspondem aos seus valores médios.

Com essa definição, a análise é realizada considerando o conversor operando no Modo de Condução Contínua (CCM). Essa condição garante que a corrente no indutor permaneça diferente de zero durante todo o período de chaveamento, permitindo representar o funcionamento do conversor por meio de duas etapas topológicas distintas. Assim, o modelo médio em espaço de estados pode ser obtido pela combinação ponderada dessas duas etapas, utilizando os intervalos de condução d e $1 - d$.

Figura 5 – Topologia do circuito conversor Buck



Fonte: Elaborado pelo autor.

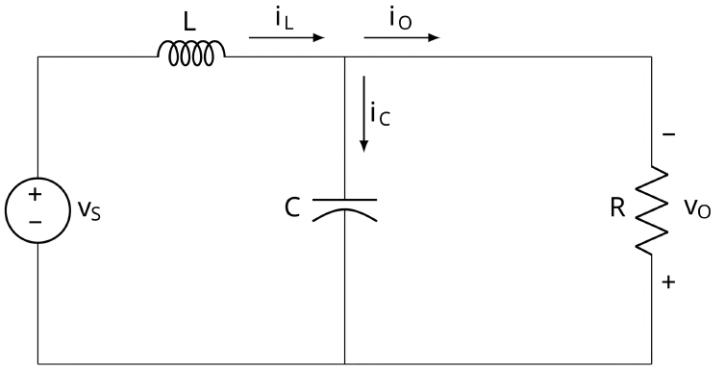
Para a modelagem temporal, o circuito é analisado em seus dois estágios topológicos de operação, correspondentes à chave fechada e à chave aberta (Barbi, 2015). Define-se o vetor de variáveis de estado $\mathbf{x}(t)$ e o vetor de entrada $\mathbf{u}(t)$ como:

$$\mathbf{x}(t) = \begin{bmatrix} i_L \\ i_C \end{bmatrix}, \quad \mathbf{u}(t) = \begin{bmatrix} v_S \end{bmatrix}. \tag{1}$$

4.1.1.1 Primeiro Estágio: Chave Fechada ($0 < t \leq DT_s$)

Durante este intervalo, a chave S conduz e a fonte v_S alimenta o indutor e a carga (Figura 6). A duração desta etapa é dada pelo produto da razão cíclica D pelo período de chaveamento T_s.

Figura 6 – Circuito equivalente com a chave fechada



Fonte: Elaborado pelo autor.

Para extrair o modelo matemático, aplica-se a Lei das Tensões de Kirchhoff (LTK) à malha de entrada. A tensão sobre o indutor (v_L) é dada pela diferença entre a tensão da fonte e a tensão do capacitor, logo (Barbi, 2015):

$$L\dot{i}_L = v_S - v_C \quad \Rightarrow \quad \dot{i}_L = -\frac{1}{L}v_C + \frac{1}{L}v_S. \tag{2}$$

Em seguida, aplica-se a Lei das Correntes de Kirchhoff (LCK) ao nó de saída. A corrente do indutor se divide entre o capacitor e a carga resistiva ($i_L = i_C + i_R$). Substituindo $i_C = C\dot{v}_C$ e $i_R = \frac{v_C}{R}$, tem-se (Barbi, 2015):

$$C\dot{v}_C = i_L - \frac{v_C}{R} \quad \Rightarrow \quad \dot{v}_C = \frac{1}{C}i_L - \frac{1}{RC}v_C. \tag{3}$$

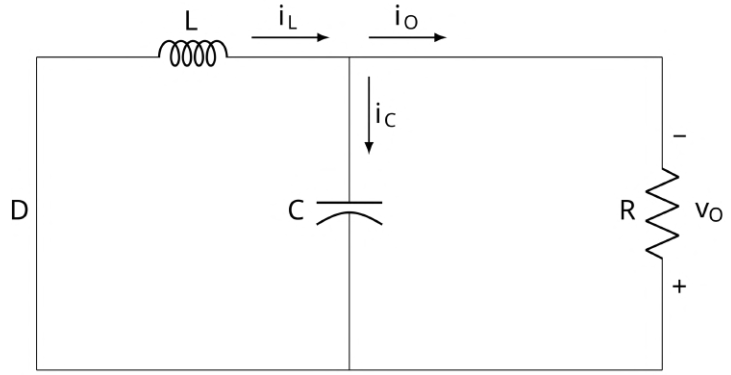
Organizando (2) e (3) na representação matricial em espaço de estados $\dot{\mathbf{x}} = \mathbf{A}_1\mathbf{x} + \mathbf{B}_1 u$ (Middlebrook; Cuk, 1976), obtém-se o modelo para o intervalo de chave fechada :

$$\begin{bmatrix} \dot{i}_L \\ \dot{v}_C \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & -\frac{1}{L} \\ \frac{1}{C} & -\frac{1}{RC} \end{bmatrix}}_{\mathbf{A}_1} \begin{bmatrix} i_L \\ v_C \end{bmatrix} + \underbrace{\begin{bmatrix} \frac{1}{L} \\ 0 \end{bmatrix}}_{\mathbf{B}_1} v_S \ . \tag{4}$$

4.1.1.2 Segundo Estágio: Chave Aberta ($DT_s < t \leq T_s$)

No tempo restante do período, a chave abre e a fonte v_S é desconectada. Para garantir a continuidade da corrente no indutor, o diodo de roda livre entra em condução (Figura 7).

Figura 7 – Circuito equivalente com a chave aberta



Fonte: Elaborado pelo autor.

Aplicando a LTK para esta nova malha, o indutor fica conectado diretamente em paralelo com o bloco RC, mas com polaridade invertida em relação à tensão de saída ($v_L = -v_C$). Logo:

$$L\dot{i}_L = -v_C \quad \Rightarrow \quad \dot{i}_L = -\frac{1}{L}v_C. \tag{5}$$

A LCK no nó de saída não sofre alteração em relação ao estágio anterior, mantendo a mesma dinâmica para o capacitor, como mostrado em (3). Agrupando (5) e (3) no formato matricial do segundo estado $\dot{\mathbf{x}} = \mathbf{A}_2\mathbf{x} + \mathbf{B}_2 u$ (Middlebrook; Cuk, 1976), tem-se:

$$\begin{bmatrix} \dot{i}_L \\ \dot{v}_C \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & -\frac{1}{L} \\ \frac{1}{C} & -\frac{1}{RC} \end{bmatrix}}_{\mathbf{A}_2} \begin{bmatrix} i_L \\ v_C \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \\ 0 \end{bmatrix}}_{\mathbf{B}_2} v_S \ . \tag{6}$$

4.1.1.3 Modelo Médio Contínuo e Equação de Saída

Assumindo que a frequência de chaveamento é significativamente superior à frequência característica do filtro LC, as componentes de alta frequência (*ripple*) decorrentes da comutação são fortemente atenuadas. Nessas condições, a dinâmica do conversor pode ser descrita por um modelo médio em espaço de estados (*State-Space Averaging*), obtido pela ponderação das matrizes correspondentes aos intervalos de chave fechada e chave aberta pelos respectivos tempos de permanência em cada estado durante um período de chaveamento, representados por d e $1-d$, respectivamente (Barbi, 2015; Middlebrook; Cuk, 1976).

$$\dot{\mathbf{x}} = \underbrace{[\mathbf{A}_1 d + \mathbf{A}_2(1-d)]}_{\mathbf{A}_M}\mathbf{x} + \underbrace{[\mathbf{B}_1 d + \mathbf{B}_2(1-d)]}_{\mathbf{B}_M}\mathbf{u}. \tag{7}$$

Substituindo as matrizes de (4) e (6) no conceito de valor médio contínuo mostrado em (7), define-se o modelo médio para o conversor Buck:

$$\begin{bmatrix} \dot{i}_L \\ \dot{v}_C \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & -\frac{1}{L} \\ \frac{1}{C} & -\frac{1}{RC} \end{bmatrix}}_{\mathbf{A}_M} \begin{bmatrix} i_L \\ v_C \end{bmatrix} + \underbrace{\begin{bmatrix} \frac{d}{L} \\ 0 \end{bmatrix}}_{\mathbf{B}_M} v_S \ . \tag{8}$$

4.1.1.3.1 Linearização e Funções de Transferência

Para a obtenção das funções de transferência do conversor, aplica-se o método da perturbação e linearização de pequenos sinais sobre o modelo médio contínuo apresentado na Equação 8. Cada variável do sistema é decomposta no somatório de seu valor médio em regime permanente (composto por componentes CC, representadas por letras maiúsculas) e uma pequena perturbação alternada em torno desse ponto de operação (componentes CA, representadas por letras minúsculas com acento circunflexo) (Middlebrook; Cuk, 1976):

$$i_L = I_L + \hat{\imath}_L, \quad v_O = V_O + \hat{v}_O, \quad v_S = V_S + \hat{v}_S, \quad d = D + \hat{d}. \tag{9}$$

Reescrevendo as equações diferenciais de estado a partir do modelo médio presente em (8):

$$\begin{aligned} \dot{i}_L &= -\frac{v_C}{L} + \frac{d}{L}v_S \\ \dot{i}_C &= \frac{i_L}{C} - \frac{v_C}{RC} \end{aligned} \tag{10}$$

Assumindo que a tensão no capacitor equivale à tensão na carga ($v_C = v_O$), substituem-se as componentes definidas em (9) nas equações diferenciais (10), resultando em:

$$\begin{aligned} L\frac{d(I_L + \hat{\imath}_L)}{dt} &= -(V_O + \hat{v}_O) + (D + \hat{d})(V_S + \hat{v}_S) \\ C\frac{d(V_O + \hat{v}_O)}{dt} &= (I_L + \hat{\imath}_L) - \frac{(V_O + \hat{v}_O)}{R} \end{aligned} \tag{11}$$

Como pode ser observado em (11), ao expandir o produto algébrico presente na equação do indutor, como mostrado em (12), surgem componentes puramente contínuas, componentes lineares de pequenos sinais e um termo não linear de segunda ordem. A partir deste ponto, a simplificação do modelo divide-se conceitualmente em duas etapas: a linearização e a eliminação das componentes de regime permanente (Middlebrook; Cuk, 1976; Barbi, 2015).

$$(D+\hat{d})(V_S+\hat{v}_S) = \underbrace{DV_S}_{\text{Nível CC}} + \underbrace{D\hat{v}_S + V_S\hat{d}}_{\text{Termos Lineares}} + \underbrace{\hat{d}\hat{v}_S}_{\text{Termo não linear}} \tag{12}$$

Cabe ressaltar que a necessidade de linearização restringe-se exclusivamente à dinâmica do indutor, visto que a equação do capacitor já é naturalmente linear por não possuir produtos entre variáveis. O princípio da linearização baseia-se na premissa de que as perturbações são significativamente menores que os valores nominais ($\hat{d} \ll D$, $\hat{v}_S \ll V_S$). Portanto, o produto de duas variáveis de pequenos sinais ($\hat{d}\hat{v}_S$) é desprezível e eliminado do modelo (Barbi, 2015).

Por outro lado, a etapa de separação e eliminação das parcelas contínuas (CC) aplica-se a ambas as equações do sistema. Sabe-se que as derivadas das componentes em regime permanente são nulas ($\frac{dI_L}{dt} = 0$ e $\frac{dV_O}{dt} = 0$) e que, no ponto de equilíbrio, a soma isolada das parcelas contínuas de cada equação resulta em zero ($DV_S - V_O = 0$ no indutor e $I_L - \frac{V_O}{R} = 0$ no capacitor).

Ao aplicar essas duas premissas algébricas, segregando os termos e mantendo apenas as componentes dinâmicas alternadas, obtêm-se as equações diferenciais linearizadas que descrevem o modelo de pequenos sinais do conversor Buck no domínio do tempo:

$$\begin{aligned} L\frac{d\hat{\imath}_L}{dt} &= V_S\hat{d} + D\hat{v}_S - \hat{v}_O \\ C\frac{d\hat{v}_O}{dt} &= \hat{\imath}_L - \frac{\hat{v}_O}{R} \end{aligned} \tag{13}$$

Embora (13) considere perturbações tanto na razão cíclica quanto na tensão de entrada, para fins de modelagem assume-se que a fonte de alimentação é ideal, apresentando tensão contínua constante e sem ondulações (*ripple*). Dessa forma, a componente de pequenos sinais da tensão de entrada é nula, ou seja, $\hat{v}_S = 0$. Consequentemente, o termo $D\hat{v}_S$ é eliminado do modelo, resultando em (14)

$$\begin{aligned} L\frac{d\hat{\imath}_L}{dt} &= V_S\hat{d} - \hat{v}_O \\ C\frac{d\hat{v}_O}{dt} &= \hat{\imath}_L - \frac{\hat{v}_O}{R} \end{aligned} \tag{14}$$

Para prosseguir com a análise no domínio da frequência, aplica-se a Transformada de Laplace ao sistema de equações diferenciais lineares apresentado em (14). Considerando condições iniciais nulas, visto que as variáveis representam pequenas perturbações em torno de um ponto de operação já estabelecido, obtêm-se as equações algébricas no domínio complexo s mostradas em (15) e (16).

$$sL\hat{\imath}_L(s) = V_S\hat{d}(s) - \hat{v}_O(s) \tag{15}$$

$$sC\hat{v}_O(s) = \hat{\imath}_L(s) - \frac{\hat{v}_O(s)}{R} \tag{16}$$

Isolando a tensão $\hat{v}_O$ em (16), tem-se a relação fundamental entre as variáveis de estado (17):

$$\hat{v}_O(s) = \hat{\imath}_L(s)\frac{R}{RCs+1} \tag{17}$$

A função de transferência da corrente do indutor em relação à razão cíclica é obtida substituindo-se (17) em (15) e isolando as variáveis de interesse:

$$sL\hat{i}_L(s) = V_S\hat{d}(s) - \left[\hat{i}_L(s)\frac{R}{RCs+1}\right] \tag{18}$$

Agrupando os termos dependentes de $\hat{i}_L(s)$ no lado esquerdo da igualdade e colocando a variável em evidência:

$$\hat{i}_L(s)\left[sL + \frac{R}{RCs+1}\right] = V_S\hat{d}(s) \tag{19}$$

Desenvolvendo o mínimo múltiplo comum para unificar os termos entre colchetes, obtém-se o polinômio característico do sistema:

$$\hat{i}_L(s)\left[\frac{s^2RLC + sL + R}{RCs+1}\right] = V_S\hat{d}(s) \tag{20}$$

Isolando a relação $\frac{\hat{i}_L(s)}{\hat{d}(s)}$, obtém-se a função de transferência corrente-razão cíclica, definida por:

$$G_{id}(s) = \frac{\hat{i}_L(s)}{\hat{d}(s)} = \frac{V_S(RCs+1)}{s^2RLC + sL + R} = \frac{\frac{V_S}{L}\left(s + \frac{1}{RC}\right)}{s^2 + \frac{1}{RC}s + \frac{1}{LC}} \tag{21}$$

De maneira análoga, a função de transferência da tensão de saída em relação à razão cíclica é obtida isolando $\hat{i}_L$ a partir de (17) e substituindo a expressão resultante em (15):

$$sL\left[\hat{v}_O(s)\left(\frac{RCs+1}{R}\right)\right] = V_S\hat{d}(s) - \hat{v}_O(s) \tag{22}$$

Transferindo o termo isolado da tensão de saída para o lado esquerdo da igualdade e colocando a variável $\hat{v}_O(s)$ em evidência:

$$\hat{v}_O(s)\left[\frac{s^2RLC + sL}{R} + 1\right] = V_S\hat{d}(s) \tag{23}$$

Desenvolvendo o mínimo múltiplo comum para unificar os termos entre colchetes:

$$\hat{v}_O(s)\left[\frac{s^2RLC + sL + R}{R}\right] = V_S\hat{d}(s) \tag{24}$$

Isolando a relação $\frac{\hat{v}_O(s)}{\hat{d}(s)}$, obtém-se a função de transferência tensão de saída-razão cíclica, definida por:

$$G_{vd}(s) = \frac{\hat{v}_O(s)}{\hat{d}(s)} = \frac{V_SR}{s^2RLC + sL + R} = \frac{\frac{V_S}{LC}}{s^2 + \frac{1}{RC}s + \frac{1}{LC}} \tag{25}$$

Por fim, a relação direta entre a tensão de saída e a corrente do indutor pode ser obtida pela razão entre as funções de transferência $G_{vd}(s)$ e $G_{id}(s)$:

$$G_{vi}(s) = \frac{G_{vd}(s)}{G_{id}(s)} = \frac{\frac{\hat{v}_O(s)}{\hat{d}(s)}}{\frac{\hat{i}_L(s)}{\hat{d}(s)}} = \frac{\hat{v}_O(s)}{\hat{i}_L(s)} \tag{26}$$

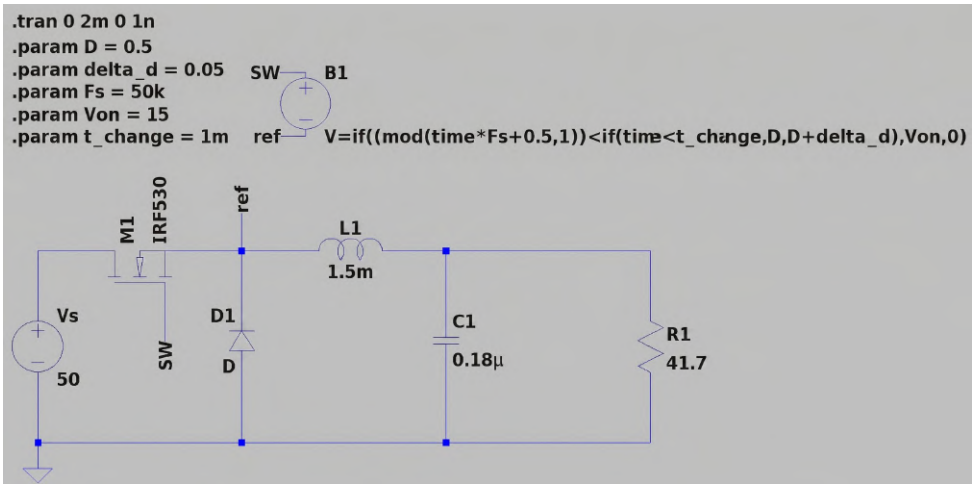
Substituindo as funções de transferência (25) e (21) previamente deduzidas e simplificando os termos comuns, obtém-se:

$$G_{vi}(s) = \frac{\left(\frac{V_SR}{s^2RLC+sL+R}\right)}{\left(\frac{V_S(RCs+1)}{s^2RLC+sL+R}\right)} = \frac{R}{RCs+1} \tag{27}$$

4.1.1.3.2 Validação das Funções de Transferência

O ensaio de validação consistiu em simular o conversor Buck descrito na Tabela 6 no software LTspice (Figura 8) e, no instante $t = 1$ ms, aplicar uma perturbação na razão cíclica. Especificamente, o valor nominal de D sofreu um acréscimo em formato de degrau de 5% absoluto, passando de 50% para 55%, sendo os dados dinâmicos do circuito extraídos para análise.

Figura 8 – Conversor Buck com teste de degrau na razão cíclica, simulado no software LTspice



Fonte: Elaborado pelo autor.

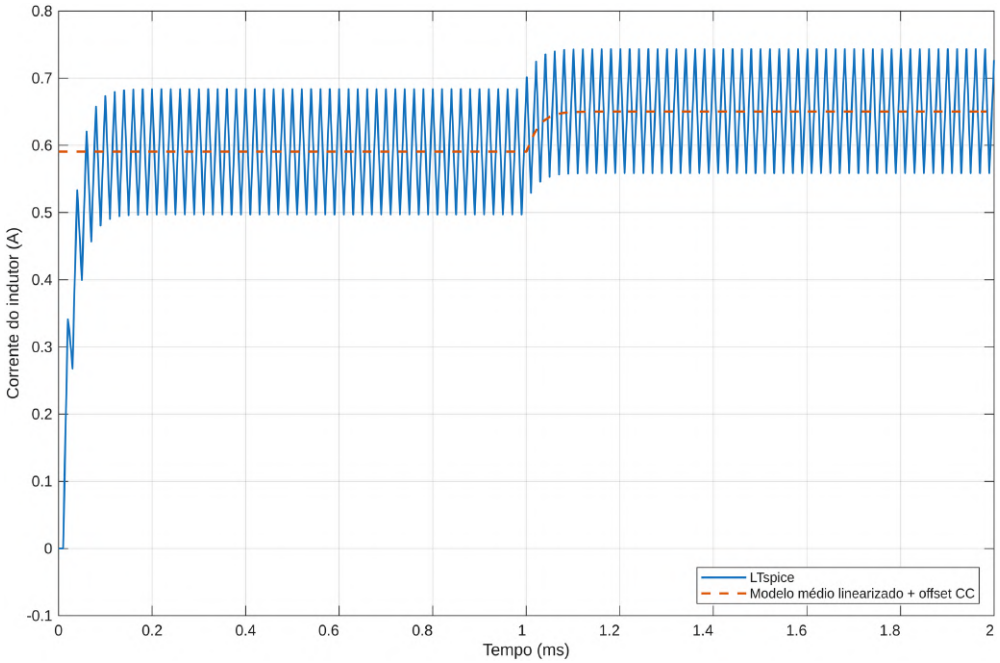
Substituindo os valores dimensionados nas funções de transferência literais deduzidas em (21) e (25), obtêm-se as equações que descrevem a dinâmica de pequenos sinais da corrente do indutor e da tensão de saída, apresentadas em (28) e (29):

$$G_{id}(s) = \frac{3{,}571 \cdot 10^4 s + 4{,}758 \cdot 10^9}{s^2 + 1{,}332 \cdot 10^5 s + 3{,}968 \cdot 10^9} \tag{28}$$

$$G_{vd}(s) = \frac{1{,}984 \cdot 10^{11}}{s^2 + 1{,}332 \cdot 10^5 s + 3{,}968 \cdot 10^9} \tag{29}$$

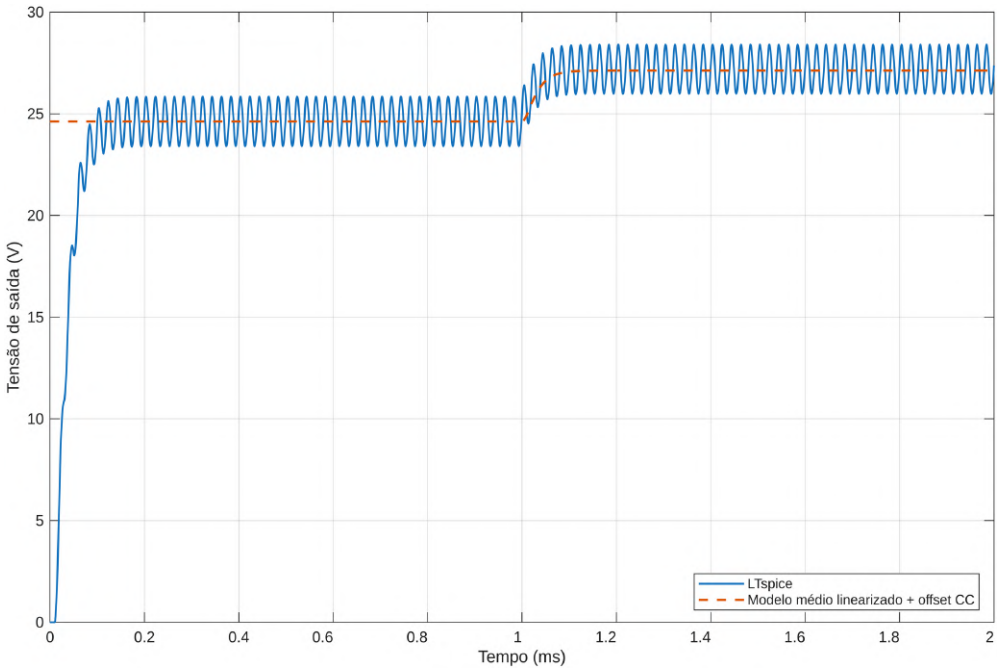
Ao submeter tanto o circuito elétrico não linear quanto as funções de transferência a esse mesmo degrau de 5% na razão cíclica, foi possível comparar diretamente as respostas dinâmicas. Cabe ressaltar que foi aplicado um *offset* correspondente ao valor médio dos sinais (nível CC de regime permanente) nas saídas das funções de transferência. Esse procedimento foi realizado para facilitar a visualização, permitindo a sobreposição correta das respostas do modelo de pequenos sinais com os dados da simulação integral do circuito. Os resultados dessa comparação para a variação da corrente no indutor e da tensão na carga são ilustrados nas Figuras 9 e 10, confirmando a precisão do modelo médio obtido.

Figura 9 – Comparação da corrente do indutor: LTspice vs. modelo médio linearizado com offset CC



Fonte: Elaborado pelo autor.

Figura 10 – Comparação da tensão de saída: LTspice vs. modelo médio linearizado com offset CC



Fonte: Elaborado pelo autor.

4.1.2 Representação Discreta dos Estágios Topológicos do Conversor Buck

A representação em tempo discreto do conversor Buck operando em CCM pode ser derivada a partir das equações diferenciais (4) e (6), que descrevem, respectivamente, as topologias do Estado 1 (chave fechada) e do Estado 2 (chave aberta). Para a discretização, considera-se que o sistema é amostrado a uma frequência F_{sim}, cujo período correspondente é definido por:

$$T_{sim} = \frac{1}{F_{sim}}. \tag{30}$$

Com o objetivo de avaliar o impacto da escolha do método de discretização no modelo, este trabalho aborda três técnicas de integração numérica: o método de Euler, o método do segurador de ordem zero e a regra do trapézio.

4.1.2.1 Método de Euler (Forward)

O método de Euler baseia-se em uma aproximação de primeira ordem que utiliza diferenças finitas avançadas (Süli; Mayers, 2003). Nesse caso, a derivada da variável de estado é substituída pela razão entre a variação do estado e o período de amostragem. Aplicando esse conceito, a representação em espaço de estados no domínio discreto é dada por:

$$A_d = I + T_{sim} \cdot A, \quad B_d = T_{sim} \cdot B \tag{31}$$

Isso resulta no modelo discreto fundamental $X[k+1] = A_d X[k] + B_d U[k]$. Realizando as multiplicações matriciais, obtêm-se as equações de diferenças para cada estágio topológico. Para o Estado 1, o modelo discreto é dado por:

$$\begin{aligned} i_L[k+1] &= i_L[k] - \frac{T_{sim}}{L} v_C[k] + \frac{T_{sim}}{L} v_S[k] \\ v_C[k+1] &= \frac{T_{sim}}{C} i_L[k] - \left(\frac{T_{sim}}{CR} - 1 \right) v_C[k] \end{aligned} \tag{32}$$

Por outro lado, quando o conversor opera no Estado 2, a dinâmica passa a ser descrita por:

$$\begin{aligned} i_L[k+1] &= i_L[k] - \frac{T_{sim}}{L} v_C[k] \\ v_C[k+1] &= \frac{T_{sim}}{C} i_L[k] - \left(\frac{T_{sim}}{CR} - 1 \right) v_C[k] \end{aligned} \tag{33}$$

Visando otimizar a implementação computacional do modelo, é conveniente agrupar os termos recorrentes em constantes pré-calculadas:

$$k_{e1} = \frac{T_{sim}}{L}, \quad k_{e2} = \frac{T_{sim}}{C}, \quad k_{e3} = 1 - \frac{T_{sim}}{RC} \tag{34}$$

Dessa forma, as equações de diferença assumem um formato reduzido e mais eficiente. Para o Estado 1, o sistema simplificado pode ser reescrito como:

$$\begin{aligned} i_L[k+1] &= i_L[k] - k_{e1} v_C[k] + k_{e1} v_S[k] \\ v_C[k+1] &= k_{e2} i_L[k] + k_{e3} v_C[k] \end{aligned} \tag{35}$$

Já para o Estado 2, a dinâmica do modelo assume a seguinte estrutura reduzida:

$$\begin{aligned} i_L[k+1] &= i_L[k] - k_{e1} v_C[k] \\ v_C[k+1] &= k_{e2} i_L[k] + k_{e3} v_C[k] \end{aligned} \tag{36}$$

Apesar de sua notável simplicidade computacional, o método de Euler possui erro de truncamento de primeira ordem, o que pode comprometer a precisão e a estabilidade da simulação quando comparado a métodos de ordens superiores.

4.1.2.2 Método do Segurador de Ordem Zero (ZOH)

O método do segurador de ordem zero (ZOH, *Zero-Order Hold*) considera que a entrada do sistema, $u(t)$, permanece constante durante cada período de amostragem T. Assim, para $t \in [kT,(k+1)T)$, assume-se que $u(t) = u[k]$. Com essa hipótese, o ZOH permite obter uma representação discreta exata de sistemas lineares invariantes no tempo (LTI). Em espaço de estados, essa representação é dada por (Angélico; Neves, 2023):

$$A_d = e^{AT}, \quad B_d = \left(\int_0^T e^{A\eta} d\eta \right) B. \tag{37}$$

Como as matrizes de estado A do conversor, definidas em (4) e (6), são não singulares, pois $\det(A) = \frac{1}{LC} \neq 0$, a matriz de entrada discreta B_d pode ser obtida de forma simplificada por (Angélico; Neves, 2023):

$$B_d = A^{-1}(e^{AT} - I)B. \tag{38}$$

No conversor Buck analisado, as matrizes de estado são iguais nos dois estágios de comutação, isto é, $A_1 = A_2 = A$. Consequentemente, a matriz de transição de estados discreta, $A_d = e^{AT_{sim}}$, é a mesma para as duas topologias. A diferença entre os estágios está, portanto, apenas nas matrizes de entrada, uma vez que $B_1 \neq B_2$.

Aplicando as relações anteriores aos dois estados de operação do conversor, para a chave fechada, definida por (4), obtém-se:

$$x[k+1] = \underbrace{\left[e^{AT_{sim}}\right]}_{A_d} x[k] + \underbrace{\left[A^{-1}(e^{AT_{sim}} - I)B_1\right]}_{B_{d_1}} u[k]. \tag{39}$$

De maneira análoga, para a chave aberta, definida por (6), tem-se:

$$x[k+1] = \underbrace{\left[e^{AT_{sim}}\right]}_{A_d} x[k] + \underbrace{\left[A^{-1}(e^{AT_{sim}} - I)B_2\right]}_{B_{d_2}} u[k]. \tag{40}$$

Dessa forma, as matrizes A_d, B_{d_1} e B_{d_2} obtidas pelo método ZOH permitem representar, em tempo discreto, a dinâmica do conversor em cada estado de comutação. Sob a hipótese de que a entrada permanece constante durante cada período de amostragem, essa representação reproduz a evolução do modelo contínuo nos instantes de amostragem.

Em comparação com o método de Euler, apresentado em 4.1.2.1, o método ZOH apresenta a vantagem de fornecer uma solução exata nos instantes de amostragem, sem os erros de truncamento decorrentes da aproximação utilizada pelo método de Euler. Dessa forma, evita-se também a introdução de efeitos numéricos artificiais nas variáveis de estado, como i_L e v_C. Além disso, a discretização pelo ZOH preserva as características de estabilidade do sistema contínuo, sem a mesma restrição sobre o passo de amostragem encontrada no método de Euler.

Por outro lado, a obtenção das matrizes discretas pelo ZOH envolve cálculos mais elaborados, como a exponencial de matriz e, na forma utilizada neste trabalho, a inversão da matriz A. No entanto, essa maior complexidade fica restrita à etapa de cálculo das matrizes, que pode ser realizada previamente com o auxílio de ferramentas computacionais. Assim, depois de obtidos os valores de A_d, B_{d_1} e B_{d_2}, essas matrizes podem ser utilizadas diretamente na simulação, sem a necessidade de realizar os cálculos matriciais durante a execução em tempo real. A hipótese de entrada constante também é compatível com o modelo adotado, uma vez que a tensão de entrada v_S é considerada constante durante cada período de amostragem.

4.1.2.3 Método do Trapézio

A regra do trapézio (ou método trapezoidal) é uma técnica de integração numérica que aproxima a integral de uma função pela área de um trapézio sob a curva (Süli; Mayers, 2003). Diferentemente do método de Euler, que avalia a função apenas no início do intervalo, a regra trapezoidal utiliza os valores em ambos os extremos. Assim, para um período T_{sim}, a integral de uma função $f(t)$ no intervalo de t_k a t_{k+1} é aproximada por:

$$\int_{t_k}^{t_{k+1}} f(t)\,dt \approx \frac{T_{sim}}{2}\left[f[k] + f[k+1]\right]. \tag{41}$$

Aplicando essa aproximação à equação diferencial genérica de um sistema em espaço de estados, $\dot{X}(t) = AX(t) + BU(t)$, obtém-se:

$$\frac{X[k+1] - X[k]}{T_{sim}} = A\frac{X[k+1] + X[k]}{2} + B\frac{U[k+1] + U[k]}{2}. \tag{42}$$

Essa formulação evidencia uma característica fundamental da regra do trapézio: o estado futuro, $X[k+1]$, aparece em ambos os lados da equação. Por essa razão, o método é classificado como *implícito*.

A presença simultânea de $X[k]$ e $X[k+1]$ permite considerar as informações dos dois limites do intervalo de integração, conferindo ao método uma aproximação de segunda ordem. Para encontrar a forma explícita do modelo em tempo discreto, é necessário isolar o estado futuro. Multiplicando a equação por T_{sim} e agrupando os termos dependentes de $X[k+1]$, tem-se:

$$\left(I - \frac{T_{sim}}{2}A\right)X[k+1] = \left(I + \frac{T_{sim}}{2}A\right)X[k] + \frac{T_{sim}}{2}B\left(U[k+1] + U[k]\right). \tag{43}$$

Consequentemente, o estado no instante $k+1$ é determinado pela inversão matricial:

$$X[k+1] = \left(I - \frac{T_{sim}}{2}A\right)^{-1}\left[\left(I + \frac{T_{sim}}{2}A\right)X[k] + \frac{T_{sim}}{2}B\left(U[k+1] + U[k]\right)\right]. \tag{44}$$

Após o processamento simbólico para as topologias do conversor Buck, obtêm-se as expressões expandidas. Especificamente para o Estado 1, a representação discreta assume a seguinte forma:

$$\begin{aligned} i_L[k+1] = \frac{\begin{aligned} &T_{sim}^2 v_S[k] + T_{sim}^2 v_S[k+1] + 2LT_{sim} i_L[k] \\ &- RT_{sim}^2 i_L[k] + 4CLR i_L[k] - 4CRT_{sim} v_C[k] \\ &+ 2CRT_{sim} v_S[k] + 2CRT_{sim} v_S[k+1] \end{aligned}}{RT_{sim}^2 + 2LT_{sim} + 4CLR} \\ v_C[k+1] = \frac{\begin{aligned} &RT_{sim}^2 v_S[k] - RT_{sim}^2 v_C[k] - 2LT_{sim} v_C[k] \\ &+ RT_{sim}^2 v_S[k+1] + 4CLR v_C[k] + 4LRT_{sim} i_L[k] \end{aligned}}{RT_{sim}^2 + 2LT_{sim} + 4CLR} \end{aligned} \tag{45}$$

Por sua vez, quando o conversor opera no Estado 2, as expressões correspondentes são dadas por:

$$\begin{aligned} i_L[k+1] &= \frac{2LT_{sim} i_L[k] - RT_{sim}^2 i_L[k] + 4CLR i_L[k] - 4CRT_{sim} v_C[k]}{RT_{sim}^2 + 2LT_{sim} + 4CLR} \\ v_C[k+1] &= \frac{4CLR v_C[k] - 2LT_{sim} v_C[k] - RT_{sim}^2 v_C[k] + 4LRT_{sim} i_L[k]}{RT_{sim}^2 + 2LT_{sim} + 4CLR} \end{aligned} \tag{46}$$

Para simplificar essas expressões algorítmicas, define-se um denominador comum Δ associado à inversão da matriz:

$$\Delta = 4CLR + 2LT_{sim} + RT_{sim}^2 \tag{47}$$

Assim, agrupam-se os termos nas seguintes constantes pré-calculadas:

$$\begin{aligned} k_{t1} &= \frac{4CLR + 2LT_{sim} - RT_{sim}^2}{\Delta} & k_{t4} &= \frac{4LRT_{sim}}{\Delta} \\ k_{t2} &= \frac{4CRT_{sim}}{\Delta} & k_{t5} &= \frac{4CLR - 2LT_{sim} - RT_{sim}^2}{\Delta} \\ k_{t3} &= \frac{T_{sim}^2 + 2CRT_{sim}}{\Delta} & k_{t6} &= \frac{RT_{sim}^2}{\Delta} \end{aligned} \tag{48}$$

Substituindo essas constantes, o modelo se torna direto e computacionalmente enxuto. Desse modo, para o Estado 1, o sistema simplificado é reescrito como:

$$\begin{aligned} i_L[k+1] &= k_{t1} i_L[k] - k_{t2} v_C[k] + k_{t3}\big(v_S[k] + v_S[k+1]\big) \\ v_C[k+1] &= k_{t4} i_L[k] + k_{t5} v_C[k] + k_{t6}\big(v_S[k] + v_S[k+1]\big) \end{aligned} \tag{49}$$

De forma análoga, para o Estado 2, as equações assumem a seguinte estrutura reduzida:

$$\begin{aligned} i_L[k+1] &= k_{t1} i_L[k] - k_{t2} v_C[k] \\ v_C[k+1] &= k_{t4} i_L[k] + k_{t5} v_C[k] \end{aligned} \tag{50}$$

Apesar de apresentar maior precisão (erro de truncamento de segunda ordem) e propriedades superiores de estabilidade em relação ao método de Euler, o método do trapézio possui como principal desvantagem o seu maior custo computacional. Devido à sua natureza implícita, o cálculo do passo seguinte exige a resolução de um sistema algébrico refletido na necessidade de calcular a inversa da matriz $\left(I - \frac{T_{sim}}{2} A\right)$, como demonstrado em (44). Isso aumenta a carga de processamento exigida a cada período de amostragem.

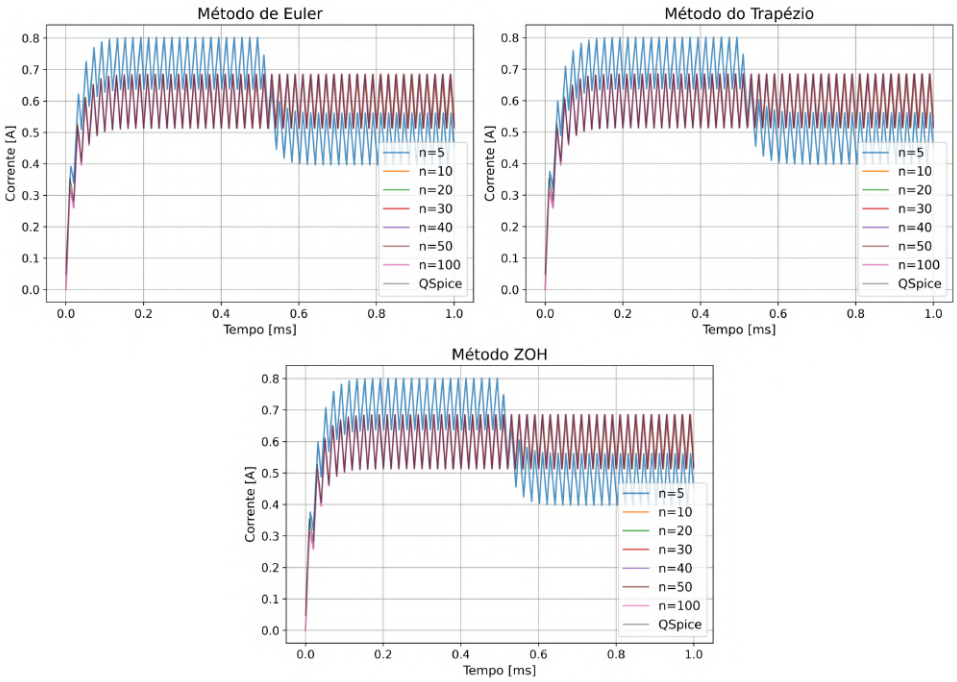
4.1.2.4 Validação da Representação Discreta

As equações diferenciais discretizadas apresentadas nas Subseções 4.1.2.1, 4.1.2.2 e 4.1.2.3 foram validadas utilizando o conversor Buck descrito pelos parâmetros apresentados na Tabela 6. A validação consistiu na comparação entre a resposta do modelo matemático discreto, implementado em MATLAB, e a resposta do modelo elétrico de referência desenvolvido no QSpice. Ambas as simulações foram realizadas com duração total de 10 ms. Para obter uma referência com elevada resolução temporal, adotou-se no QSpice um passo de integração fixo de 20 ns, correspondente a uma frequência de amostragem de 50 MHz.

Quanto à simulação discreta, (Gaviria-cano *et al.*, 2025) recomenda que o período de integração T_{sim} seja pelo menos dez vezes menor que o período de chaveamento T_s, mitigando assim os erros numéricos. Na prática, isso significa que a frequência de simulação deve ser no mínimo dez vezes maior que a de chaveamento ($F_{sim} \geq 10 \cdot F_s$).

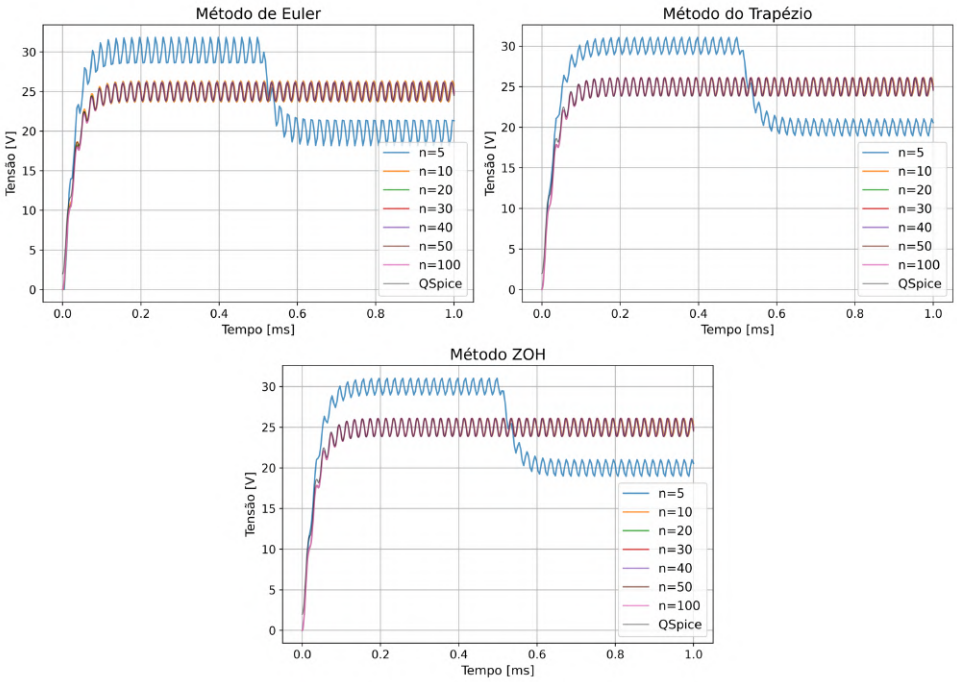
Para avaliar o impacto dessa restrição teórica, o modelo matemático foi simulado no MATLAB sob diferentes frequências F_{sim}, utilizando a relação $F_{sim} = F_s \cdot n$. Sendo $F_s = 50$ kHz, testaram-se os multiplicadores $n \in \{5, 10, 20, 30, 40, 50, 100\}$. Essa varredura contemplou desde cenários abaixo do limite recomendado (250 kHz para $n = 5$) e no limite teórico (500 kHz para $n = 10$), até taxas elevadas de amostragem, chegando a 5 MHz ($n = 100$). Essa abordagem permitiu analisar detalhadamente o comportamento numérico sob resoluções temporais crescentes.

Figura 11 – Comparação da corrente no indutor (i_L) entre os métodos de Euler e Trapézio



Fonte: Elaborado pelo autor.

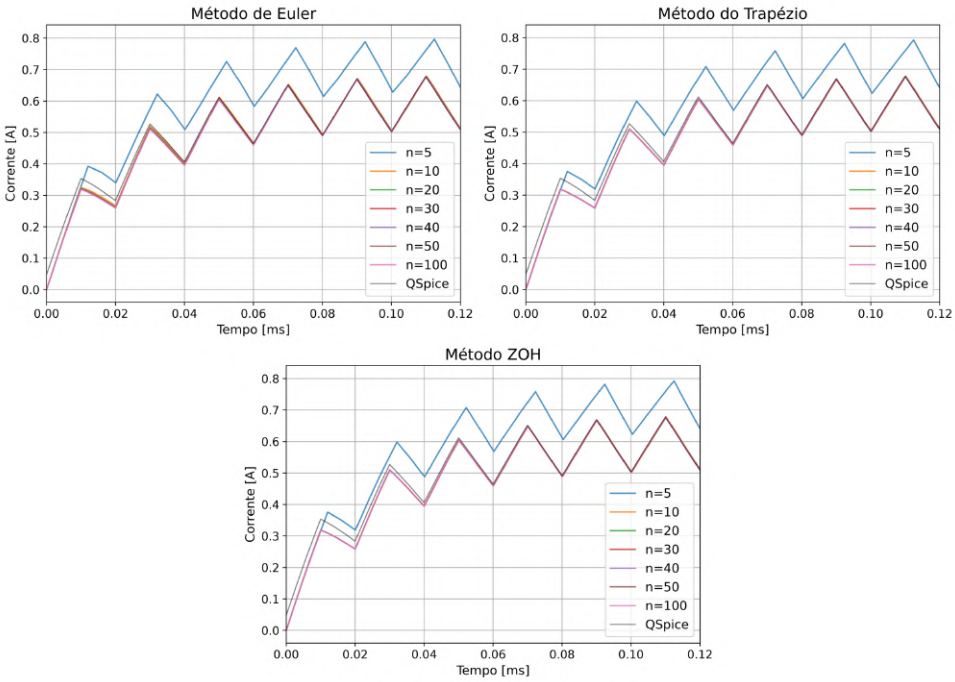
Figura 12 – Comparação da tensão no capacitor (v_C) entre os métodos de Euler e Trapézio



Fonte: Elaborado pelo autor.

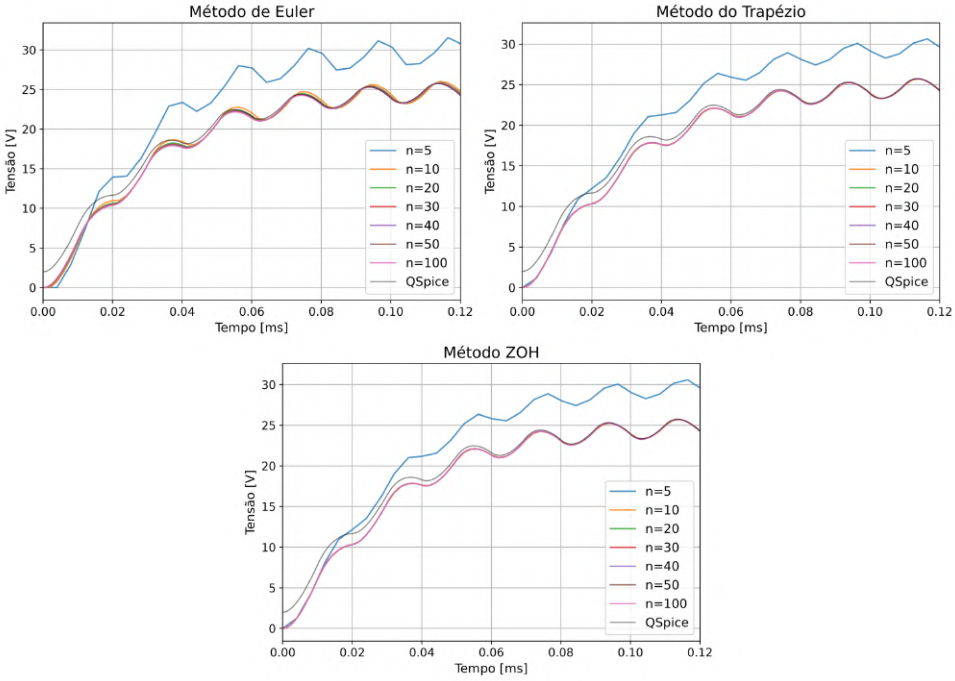
Analisando inicialmente as formas de onda completas (Figuras 11 e 12), os resultados corroboram as recomendações da literatura. Fica evidente que usar um fator $n < 10$ (como no cenário $n = 5$) gera uma representação imprecisa e com alto desvio da referência, inadequada para simular o conversor. Em contrapartida, para $n \geq 10$, as curvas de ambos os modelos discretos acompanham de perto a resposta do QSpice.

Figura 13 – Comparação da corrente no indutor (i_L) entre os métodos de Euler e Trapézio em regime transitório



Fonte: Elaborado pelo autor.

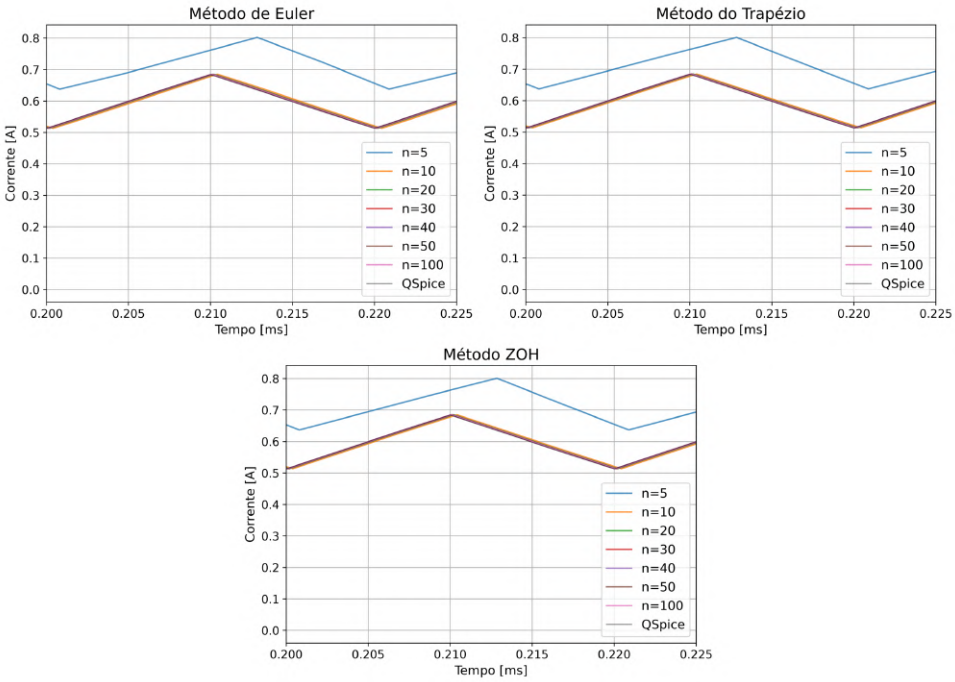
Figura 14 – Comparação da tensão no capacitor (v_C) entre os métodos de Euler e Trapézio em regime transitório



Fonte: Elaborado pelo autor.

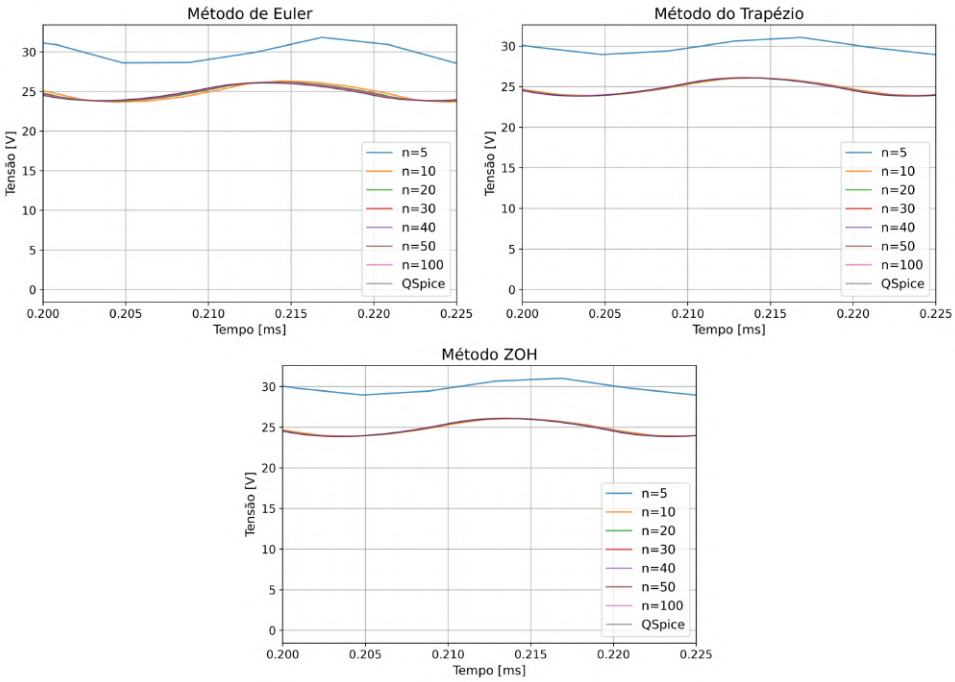
Apesar da boa concordância geral para $n \geq 10$, uma observação detalhada do regime transitório (Figuras 13 e 14) revela uma leve diferença visual entre os métodos discretos e a curva do QSpice. Esse desvio inicial ocorre tanto na corrente do indutor quanto na tensão do capacitor, indicando que a partida do conversor é o momento mais sensível à escolha do passo de integração e do método numérico.

Figura 15 – Comparação da corrente no indutor (i_L) entre os métodos de Euler e Trapézio em regime permanente



Fonte: Elaborado pelo autor.

Figura 16 – Comparação da tensão no capacitor (v_C) entre os métodos de Euler e Trapézio em regime permanente

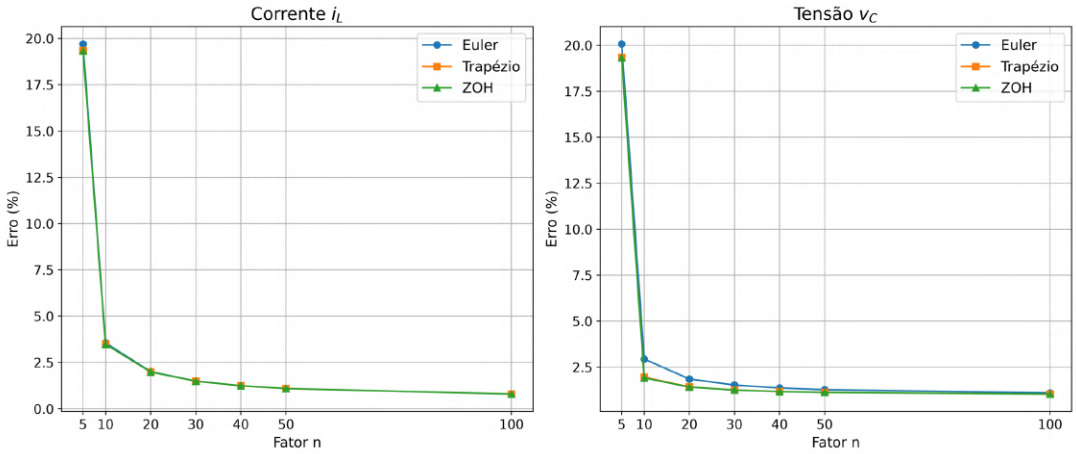


Fonte: Elaborado pelo autor.

Já no regime permanente (Figuras 15 e 16), as curvas se sobrepõem de maneira bastante fiel para $n \geq 10$. Como essa convergência visual persiste na maior parte da simulação, torna-se inviável comparar o desempenho global entre os métodos de Euler e Trapézio apenas por inspeção gráfica. Por isso, as aproximações foram avaliadas quantitativamente por meio do cálculo do erro relativo percentual médio.

Como ilustrado na Figura 17, a adoção de $n \geq 10$ reduz significativamente o erro e proporciona maior estabilidade nos resultados, mostrando-se uma escolha adequada para o projeto. Na estimação da corrente do indutor, os três métodos apresentam erros bastante próximos, indicando pouca influência do método de discretiza-

Figura 17 – Erro relativo percentual médio da corrente (i_L) e da tensão (v_C) em função do fator n, tendo o QSpice como referência



Fonte: Elaborado pelo autor.

ção sobre essa variável. Para a tensão no capacitor, entretanto, o método de Euler apresenta um erro ligeiramente superior aos métodos ZOH e do Trapézio para a dinâmica específica deste conversor.

Diante desses resultados, a escolha do método pode ser definida considerando também os aspectos relacionados à implementação computacional. No caso do ZOH, embora a obtenção das matrizes discretas envolva operações mais complexas, como o cálculo da exponencial de matriz, essas operações podem ser realizadas previamente. Durante a execução da simulação, são utilizadas apenas as matrizes A_d, B_{d_1} e B_{d_2} já calculadas, sem a necessidade de repetir essas operações a cada iteração. Isso torna o método adequado para a implementação em microcontrolador, permitindo a execução do modelo em tempo real.

Assim, considerando a precisão obtida para $n \geq 10$, a ausência de erros de truncamento associados à aproximação da derivada e a possibilidade de pré-calcular as matrizes discretas, adota-se, neste trabalho, o método ZOH com $n = 10$ para a discretização do modelo do conversor.

4.2 Projeto do filtro antialiasing

Um filtro antialiasing, é um filtro analógico passa-baixas utilizado antes do conversor A/D em sistemas de controle digital. Sua principal função é atenuar as componentes de alta frequência presentes no sinal e no ruído, especialmente aquelas próximas ou acima da frequência de Nyquist, evitando que sejam interpretadas como componentes de menor frequência durante a amostragem devido ao fenômeno de aliasing (Franklin, 2018). Como o filtro atua sobre o sinal contínuo antes da conversão A/D, sua implementação deve ser analógica. Os polos e o atraso de fase introduzidos pelo filtro também devem ser considerados no projeto do sistema de controle, pois podem alterar a dinâmica e as margens de estabilidade da malha fechada (Franklin, 2018).

Neste trabalho, foi utilizado um filtro passa-baixas Bessel de segunda ordem, escolhido por apresentar características favoráveis de fase e resposta temporal. Esse tipo de filtro possui uma resposta de fase aproximadamente linear e um atraso de grupo mais uniforme na faixa de interesse. Além disso, sua resposta ao degrau apresenta baixo sobre-sinal, reduzindo a possibilidade de oscilações no sinal medido (Franklin, 2018). A escolha de uma estrutura de segunda ordem também permite obter uma atenuação assintótica de aproximadamente -40 dB/década, mantendo uma implementação relativamente simples.

O projeto foi realizado a partir do protótipo normalizado do filtro Bessel de segunda ordem. A frequência de corte foi definida como um décimo da frequência de chaveamento do conversor, conforme

$$F_c = \frac{F_{sw}}{10}, \tag{51}$$

em que F_{sw} representa a frequência de chaveamento do conversor. A frequência angular de corte é dada por $\omega_c = 2\pi F_c$. Os coeficientes do protótipo foram obtidos no MATLAB por meio da função `besself` e posteriormente ajustados para a frequência de corte especificada. Dessa forma, obteve-se a função de transferência

$$H(s) = \frac{4\,{F_c}^2\,\pi^2}{4\,\pi^2\,{F_c}^2 + 2\,\sqrt{3}\,\pi\,F_c\,s + s^2}. \tag{52}$$

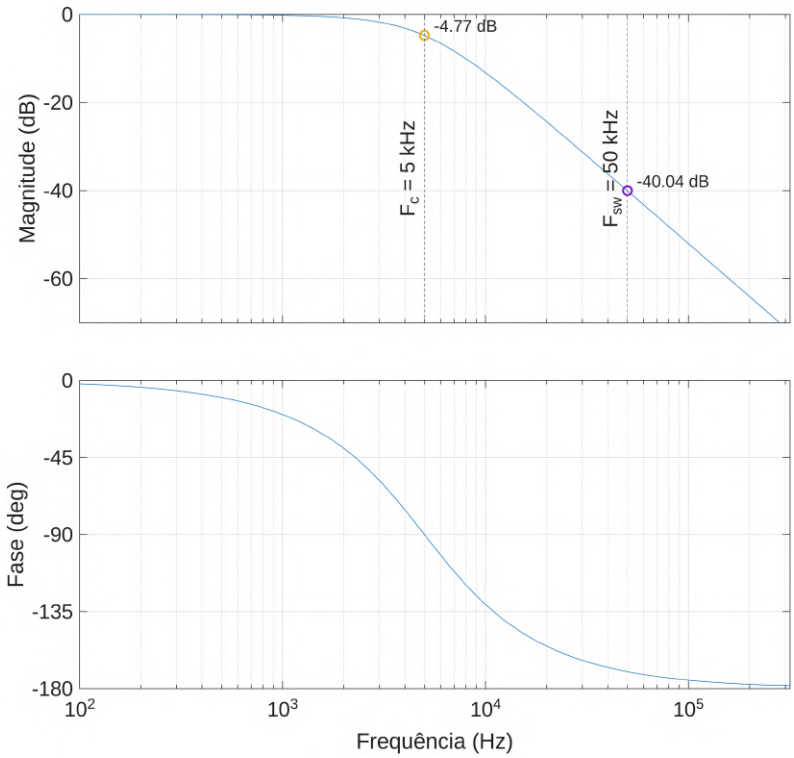
O filtro é inserido entre a saída da planta simulada e o conversor A/D, condicionando o sinal antes de sua amostragem pelo sistema de controle digital.

Para o conversor Buck utilizado como caso de teste neste trabalho e na validação dos modelos discretizados desenvolvidos em 4.1.2.4, considerou-se uma frequência de chaveamento de $F_{sw} = 50$ kHz. A partir da relação definida anteriormente, obtém-se uma frequência de corte de $F_c = 5$ kHz. Substituindo esse valor na função de transferência, obtém-se

$$H(s) = \frac{9{,}87 \times 10^8}{s^2 + 5{,}441 \times 10^4 s + 9{,}87 \times 10^8}. \tag{53}$$

A resposta em frequência correspondente a essa função de transferência pode ser observada no diagrama de Bode apresentado na Figura 18. Assim, o filtro proporciona a atenuação das componentes de alta frequência presentes na tensão de saída antes da conversão A/D, reduzindo a possibilidade de aliasing e fornecendo ao sistema de controle um sinal adequado para a amostragem.

Figura 18 – Diagrama de Bode do filtro passa-baixas antialiasing



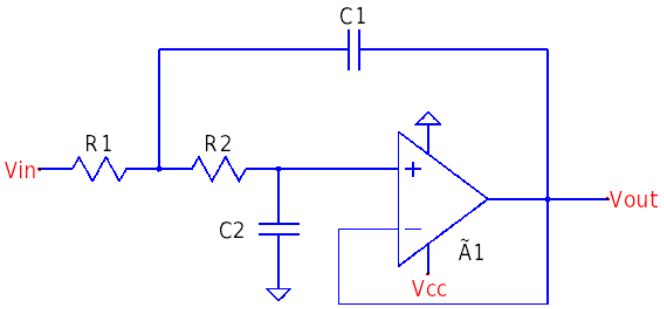
Fonte: Elaborado pelo autor.

Como o filtro antialiasing deve ser implementado no domínio analógico, optou-se pela utilização da topologia Sallen-Key, apresentada na Figura 19.

Para um filtro passa-baixa Sallen-Key de segunda ordem, a função de transferência genérica pode ser expressa como (Texas Instruments, 2021)

$$H(s) = \frac{a_0}{s^2 + a_1 s + a_0}, \tag{54}$$

Figura 19 – Topologia de um filtro passa-baixa Sallen-Key de segunda ordem



Fonte: Elaborado pelo autor com base em Texas Instruments (2021).

em que os coeficientes a_0 e a_1 são determinados a partir dos valores dos componentes do circuito, conforme (Texas Instruments, 2021)

$$a_0 = \frac{1}{R_1 R_2 C_1 C_2} \quad \text{e} \quad a_1 = \frac{1}{R_1 C_1} + \frac{1}{R_2 C_1}. \tag{55}$$

Comparando a função de transferência do filtro, apresentada em (53), com a função de transferência genérica da topologia Sallen-Key, dada em (54), obtêm-se os seguintes valores para os coeficientes:

$$a_0 = 9{,}87 \times 10^8 \quad \text{e} \quad a_1 = 5{,}441 \times 10^4. \tag{56}$$

Como há quatro componentes a serem determinados (R_1, R_2, C_1 e C_2), mas apenas duas equações de restrição, correspondentes aos coeficientes a_0 e a_1, é necessário adotar condições adicionais para definir os valores dos componentes. Para simplificar o projeto, considera-se $R_1 = R_2 = R$, de modo que as expressões dos coeficientes podem ser escritas como

$$a_0 = \frac{1}{R^2 C_1 C_2} \quad \text{e} \quad a_1 = \frac{2}{R C_1}. \tag{57}$$

A partir da equação de a_1, o resistor pode ser expresso em função de C_1 como

$$R = \frac{2}{a_1 C_1}. \tag{58}$$

Substituindo essa expressão na equação de a_0, obtém-se uma relação entre os capacitores que depende apenas dos coeficientes da função de transferência. Após a simplificação, essa relação é dada por

$$\frac{C_2}{C_1} = \frac{a_1^2}{4 a_0}. \tag{59}$$

Substituindo os valores de a_0 e a_1 obtidos anteriormente, resulta em

$$\frac{C_2}{C_1} = \frac{(5{,}441 \times 10^4)^2}{4 \cdot (9{,}87 \times 10^8)} = 0{,}7498\cdots \approx 0{,}75. \tag{60}$$

Assim, para atender à razão de aproximadamente $0{,}75$ utilizando valores comerciais, são escolhidos

$$C_1 = 20 \text{ nF} \quad \text{e} \quad C_2 = 15 \text{ nF}. \tag{61}$$

Com os valores dos capacitores definidos, o resistor pode ser determinado diretamente pela expressão de a_1:

$$R = \frac{2}{a_1 C_1} = \frac{2}{5{,}441 \times 10^4 \cdot 20 \times 10^{-9}} = 1837{,}8974\ldots\ \Omega \approx 1{,}84\ \text{k}\Omega. \tag{62}$$

Com isso, foram definidos os valores dos componentes do filtro, sendo $R_1 = R_2 = 1{,}84\ \text{k}\Omega$, $C_1 = 20$ nF e $C_2 = 15$ nF. Esses valores permitem implementar, por meio da topologia Sallen-Key, o filtro passa-baixas projetado, com uma frequência de corte próxima de 5 kHz. Assim, o circuito pode ser utilizado para atenuar as componentes de alta frequência da tensão de saída do conversor antes da conversão A/D.

4.3 Projeto e discretização do controlador

O projeto do controlador apresentado nesta seção é realizado para o conversor Buck cujos parâmetros são apresentados na Tabela 6. Embora a simulação em tempo real da planta na plataforma HIL utilize o modelo discreto definido em 4.1.2, o projeto analítico do controlador é realizado a partir das funções de transferência contínuas obtidas do modelo médio linearizado de pequenos sinais. Dessa forma, a representação contínua da dinâmica do conversor é utilizada para determinar os parâmetros do controlador, que posteriormente será discretizado para implementação no microcontrolador e utilizado como dispositivo sob teste na plataforma HIL.

A partir do diagrama de blocos da malha de controle apresentado na Figura 4, a função de transferência de malha fechada pode ser obtida como

$$T(s) = \frac{\hat{v}_O(s)}{\hat{V}_{ref}(s)} = \frac{C(s)G(s)}{1 + C(s)G(s)H(s)}, \tag{63}$$

em que $C(s)$ representa a função de transferência do controlador, $G(s)$ a função de transferência da planta e $H(s)$ a função de transferência do filtro antialiasing (53). Como optou-se por realizar o controle da tensão de saída de maneira direta, a dinâmica da planta é definida por $G(s) = G_{vd}(s)$ (29). Essa relação será utilizada como base para a análise da resposta dinâmica do sistema e para o ajuste dos parâmetros do controlador.

Com base nessa representação da malha de controle, o projeto do controlador é dividido em duas etapas. Na primeira, apresentada em 4.3.1, são determinados os parâmetros do controlador a partir do modelo contínuo da planta, buscando atender aos requisitos de desempenho estabelecidos para o sistema em malha fechada. Em seguida, em 4.3.2, a função de transferência contínua do controlador projetado é discretizada, de modo a obter uma representação adequada para sua implementação no microcontrolador.

4.3.1 Projeto do controlador

Para o projeto e ajuste do controlador, optou-se pela utilização do método do Lugar das Raízes (*Root Locus*) (Ogata, 2009). Desenvolvido por W. R. Evans, esse método permite analisar graficamente a trajetória dos polos de malha fechada no plano complexo s em função da variação de um parâmetro do sistema, geralmente o ganho. O Lugar das Raízes é determinado a partir da equação característica

$$1 + C(s)G(s)H(s) = 0, \tag{64}$$

cujos pontos pertencentes às trajetórias devem satisfazer as condições de ângulo e magnitude (Ogata, 2009).

Antes da definição do controlador, devem ser estabelecidas as especificações de desempenho do sistema, que quantificam os requisitos de operação em regime permanente e transitório (Ogata, 2009). Entre as principais estão o erro de regime permanente (e_{ss}), o sobressinal máximo (M_p), o tempo de subida (t_r) e o tempo de assentamento (t_s). O erro de regime permanente corresponde à diferença entre a referência e a saída após o período transitório, enquanto o sobressinal máximo representa o maior desvio da resposta em relação ao valor de regime permanente. O tempo de subida corresponde ao intervalo necessário para a resposta variar de 10% a 90% do valor final, e o tempo de assentamento indica o intervalo necessário para que a resposta entre e permaneça dentro de uma faixa de tolerância, usualmente de 2% ou 5%. Para sistemas de segunda ordem, o tempo de assentamento pode ser aproximado, respectivamente, por $t_s \approx \frac{4}{\zeta\omega_n}$ e $t_s \approx \frac{3}{\zeta\omega_n}$ para os critérios de 2% e 5% (Ogata, 2009).

Como especificações para o controlador a ser projetado, foram definidos os seguintes requisitos para o sistema em malha fechada: dinâmica dominante de primeira ordem, de modo a obter uma resposta sem sobresinal; tempo de assentamento $t_s \leq 1\,\text{ms}$, considerando o critério de 2%; e erro de regime permanente nulo para uma entrada do tipo degrau. Esses requisitos estabelecem as condições desejadas para o comportamento transitório e para a precisão no rastreamento da referência, sendo utilizados como critérios para a avaliação do desempenho do sistema e para a definição da estrutura e dos parâmetros do controlador.

A função de transferência da planta utilizada no projeto é a apresentada em (25), considerando os parâmetros do conversor apresentados na Tabela 6, exceto pela resistência de carga, cujo valor é determinado de acordo com a condição de operação adotada para o projeto. Para a representação do filtro anti-aliasing, foi utilizada a função de transferência apresentada em (53).

Considerando que a variação da carga modifica a dinâmica da planta, o controlador foi projetado para uma condição de operação correspondente a 10% da potência nominal. Essa condição foi adotada como caso de projeto devido ao aumento da resistência de carga associado à redução da potência, resultando em uma condição de operação distinta daquela correspondente à carga nominal. Dessa forma, a resistência de carga utilizada no projeto foi calculada por

$$R = \frac{R_{\text{nom}}}{P/P_{\text{nom}}}, \tag{65}$$

em que R_{nom} corresponde à resistência de carga na condição de potência nominal, enquanto P/P_{nom} representa a fração da potência nominal considerada no projeto. Para $P/P_{\text{nom}} = 0{,}1$ e $R_{\text{nom}} = 41{,}7\,\Omega$, obtém-se

$$R = \frac{41{,}7}{0{,}1} = 417\,\Omega. \tag{66}$$

Assim, os parâmetros utilizados na função de transferência da planta correspondem aos valores apresentados na Tabela 6, com a resistência de carga ajustada para $417\,\Omega$, correspondente à condição de 10% da potência nominal.

Para atender ao requisito de erro de regime permanente nulo no rastreamento de uma referência constante, foi inicialmente considerada a utilização de uma ação integral. A presença de um polo na origem fornece ganho infinito em corrente contínua, permitindo eliminar o erro de regime permanente para uma entrada do tipo degrau, desde que o sistema permaneça estável. A estrutura inicial do controlador foi, portanto, definida como

$$C(s) = \frac{1}{s}. \tag{67}$$

Embora o integrador permita atender ao requisito de erro de regime permanente, sua utilização isolada não oferece graus de liberdade suficientes para ajustar adequadamente a resposta transitória do sistema. Por esse motivo, foi adotada uma estrutura proporcional-integral (PI), acrescentando-se um zero ao controlador. A estrutura considerada foi

$$C(s) = K\frac{s+z}{s}, \tag{68}$$

em que z representa a posição do zero e K corresponde ao ganho do controlador.

A posição do zero foi definida com o auxílio do ambiente *Control System Designer*, disponível no MATLAB. Nessa etapa, buscou-se uma configuração que permitisse ajustar a posição dos polos de malha fechada de acordo com a dinâmica desejada, mantendo o sistema estável. A partir dessa análise, foi selecionado o zero em

$$z = -6{,}5 \times 10^4\,\text{rad/s}, \tag{69}$$

resultando no controlador

$$C(s) = K\frac{s + 6{,}5 \times 10^4}{s}. \tag{70}$$

Com a estrutura do controlador definida, o ganho K foi determinado a partir da condição de magnitude do Lugar das Raízes. Para atender ao requisito de tempo de assentamento, adotou-se uma dinâmica dominante de primeira ordem. Nesse caso, considerando o critério de 2%, o tempo de assentamento pode ser aproximado por

$$t_s \approx \frac{4}{a}, \tag{71}$$

em que $-a$ representa a posição do polo dominante no eixo real (Ogata, 2009). Para $t_s = 1\,\mathrm{ms}$, resulta em

$$a = \frac{4}{1 \times 10^{-3}} = 4000\,\mathrm{rad/s}, \tag{72}$$

portanto, o polo dominante desejado é $s_0 = -4000\,\mathrm{rad/s}$

A partir da equação característica apresentada em (64), um ponto s_0 pertence ao Lugar das Raízes quando a condição de magnitude é satisfeita. Considerando a estrutura do controlador adotada, essa condição pode ser escrita como

$$\left| K \frac{s_0 + 6{,}5 \times 10^4}{s_0} G_{vd}(s_0) H(s_0) \right| = 1. \tag{73}$$

Assim, o ganho do controlador pode ser determinado por

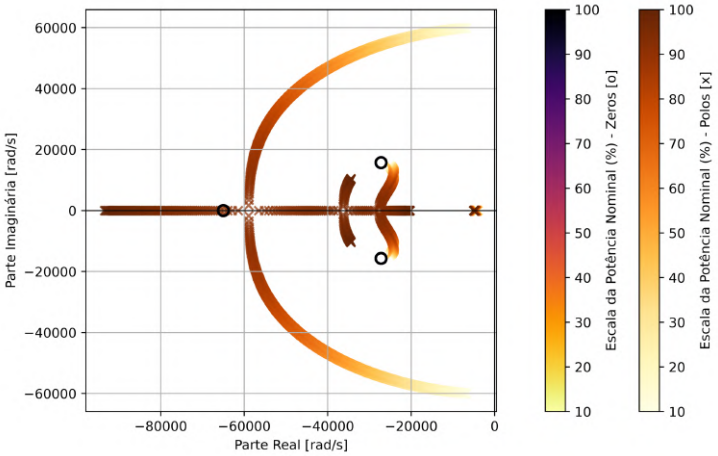
$$K = \frac{1}{\left| \dfrac{s_0 + 6{,}5 \times 10^4}{s_0} G_{vd}(s_0) H(s_0) \right|}. \tag{74}$$

Aplicando essa condição à planta e ao filtro de medição considerados no projeto, obteve-se o ganho $K = 0{,}001$. Dessa forma, o controlador PI projetado é dado por

$$C(s) = 0{,}001 \frac{s + 6{,}5 \times 10^4}{s}. \tag{75}$$

Possuindo as funções de transferência da planta (25), do filtro (53) e do controlador (75), foram analisados os polos de malha fechada e a resposta temporal ao degrau para diferentes condições de carga, correspondentes a potências nominais entre 10% $(1{,}5\,\mathrm{W})$ e 100% $(15\,\mathrm{W})$. Essa análise permite verificar o comportamento do sistema diante da variação da carga e avaliar se o controlador projetado atende aos requisitos estabelecidos ao longo de toda a faixa de operação considerada, principalmente em relação à estabilidade, à ausência de sobressinal, ao tempo de assentamento e ao erro de regime permanente.

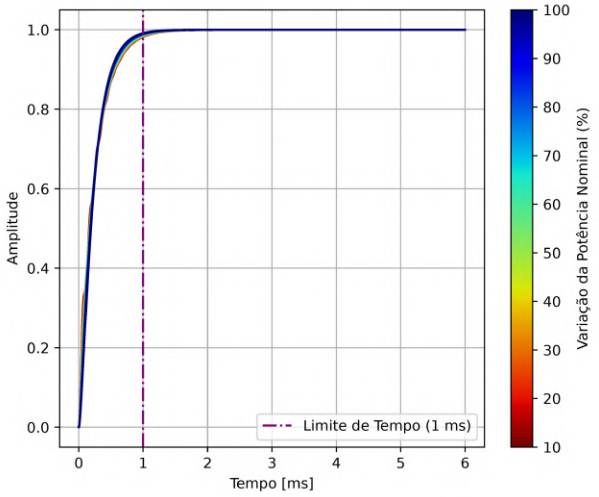
Figura 20 – Posição dos polos e zeros do sistema em função da variação da potência nominal



Fonte: Elaborado pelo autor.

Como pode ser observado nas Figuras 20 e 21, o sistema mantém um comportamento dinâmico dominante de primeira ordem ao longo de toda a faixa de carga analisada. Os polos permanecem no semiplano

Figura 21 – Resposta transitória do sistema em função da variação da potência nominal



Fonte: Elaborado pelo autor.

esquerdo do plano complexo, indicando que o sistema se mantém estável para todas as condições de operação avaliadas. Além disso, a resposta temporal apresenta comportamento sem sobressinal, enquanto o tempo de assentamento permanece aproximadamente em 1 ms ao longo da faixa de potência considerada.

4.3.2 Discretização do controlador

Para a discretização do controlador PI, foi utilizado o método de Tustin, ou Transformação Bilinear, baseado na integração trapezoidal. Esse método realiza o mapeamento do plano-s para o plano-z, preservando a estabilidade do sistema (Angélico; Neves, 2023). A transformação utilizada é dada por

$$s = \frac{2}{T_s}\frac{z-1}{z+1}, \tag{76}$$

em que T_s é o período de amostragem. Aplicando essa transformação ao controlador PI definido em (75), obtém-se a seguinte função de transferência discreta:

$$C(z) = 0{,}001\,\frac{\frac{2}{T_s}\frac{z-1}{z+1} + 6{,}5\times 10^4}{\frac{2}{T_s}\frac{z-1}{z+1}} = \frac{(0{,}001 + 32{,}5\,T_s)\,z + (32{,}5\,T_s - 0{,}001)}{z-1}. \tag{77}$$

Para a implementação digital, a expressão é reescrita em termos de z^{-1}, considerando $C(z) = U(z)/E(z)$:

$$\frac{U(z)}{E(z)} = \frac{(0{,}001 + 32{,}5\,T_s) + (32{,}5\,T_s - 0{,}001)\,z^{-1}}{1 - z^{-1}}. \tag{78}$$

Assim, a lei de controle em tempo discreto é expressa pela equação em diferenças

$$u[k] = u[k-1] + (0{,}001 + 32{,}5\,T_s)\,e[k] + (32{,}5\,T_s - 0{,}001)\,e[k-1]. \tag{79}$$

A lei de controle discreta foi implementada em código C++ no QSPICE para sua validação durante as simulações, conforme apresentado no Código 1. A implementação utiliza o erro de tensão da amostra atual e o erro da amostra anterior para calcular o novo valor do sinal de controle. Essa estrutura também serve como referência para a implementação do controlador no microcontrolador, mantendo a mesma forma discreta da lei de controle.

Código 1 – Implementação do controlador no QSPICE.

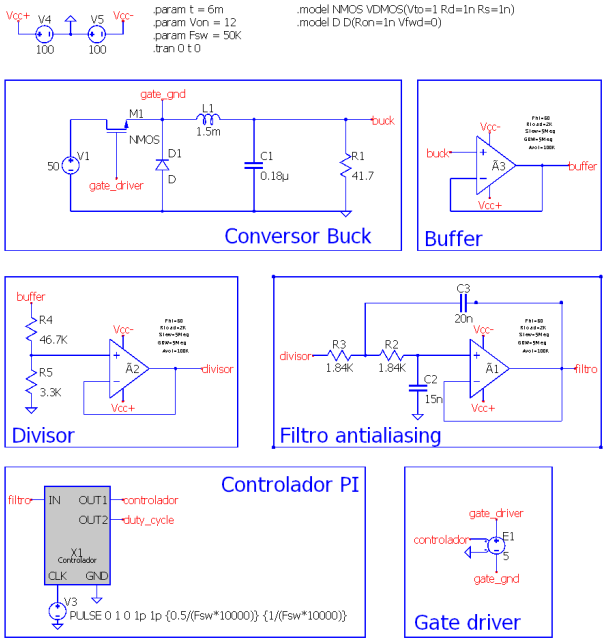
```
1  extern "C" __declspec(dllexport)
2  void controlador(void **opaque, double t, union uData *data)
3  {
4      double IN = data[0].d * (50.0 / 3.3); // ADC input
5      bool CLK = data[1].b; // Input de clock
6      bool &OUT1 = data[2].b; // Saída para PWM
7      double &OUT2 = data[3].d; // Saída para duty cycle
8
9      // VARIÁVEIS DO TEMPORIZADOR
10     static unsigned int CLK_counter = 0;
11     static bool CLK_pastVal = false;
12     static const unsigned int CLK_counter_max = 10000;
13
14     // VARIÁVEIS DO CONTROLADOR
15     static double Vref = Vref_inicial;
16     static double U = 0.0, pastU = 0.0, E = 0.0, pastE = 0.0; // Duty cycle e Erro
17     static bool PWM_out = false;
18
19     if (CLK && !CLK_pastVal) // DETECÇÃO DA BORDA DE SUBIDA DO CLOCK
20     {
21         CLK_counter++;
22
23         if (CLK_counter >= CLK_counter_max)
24         {
25             CLK_counter = 0;
26
27             // GERAÇÃO DO DEGRAU DE REFERÊNCIA USANDO O TEMPO ABSOLUTO 't'
28             if (t >= tempo_degrau) {
29                 Vref = Vref_inicial + Vref_incremento;
30             }
31             else {
32                 Vref = Vref_inicial;
33             }
34
35             // CÁLCULO DO ERRO E LEI DE CONTROLE PI
36             E = Vref - IN;
37             U = pastU + 0.000357719 * E + 0.000346881 * pastE;
38
39             // SATURAÇÃO DO SINAL DE CONTROLE
40             if (U > 1.0) U = 1.0;
41             if (U < 0.0) U = 0.0;
42
43             // ATUALIZAÇÃO DOS ESTADOS
44             pastE = E;
45             pastU = U;
46         }
47
48         if ((U * CLK_counter_max) > CLK_counter) { // GERAÇÃO DO SINAL PWM
49             PWM_out = true;
50         }
51         else {
52             PWM_out = false;
53         }
54     }
55
56     OUT1 = PWM_out;
57     OUT2 = U; // Sempre atualiza a saída
58
59     CLK_pastVal = CLK;
60 }
```

Fonte: Elaborado pelo autor.

4.4 Validação do Modelo em Malha Fechada

Para validar a capacidade do modelo digital de representar adequadamente o comportamento do conversor Buck em condições de operação em malha fechada, foi realizada uma simulação utilizando os parâmetros apresentados na Tabela 6. O sistema foi composto pelo controlador PI, obtido em 4.3, e pelo filtro *anti-aliasing*, projetado em 4.2. A validação foi realizada por meio de duas abordagens: a simulação do circuito eletrônico no QSPICE e uma simulação numérica, implementada em Python, utilizando o modelo matemático discretizado do sistema. O circuito implementado no QSPICE para essa etapa da validação é apresentado na Figura 22. Em ambas as abordagens, foi considerado um tempo total de simulação de 6 ms, com a aplicação de um degrau de 10 V na tensão de referência V_{ref}, que passou de 25 V para 35 V no instante $t = 3$ ms.

Figura 22 – Circuito do conversor Buck em malha fechada implementado no QSPICE



Fonte: Elaborado pelo autor.

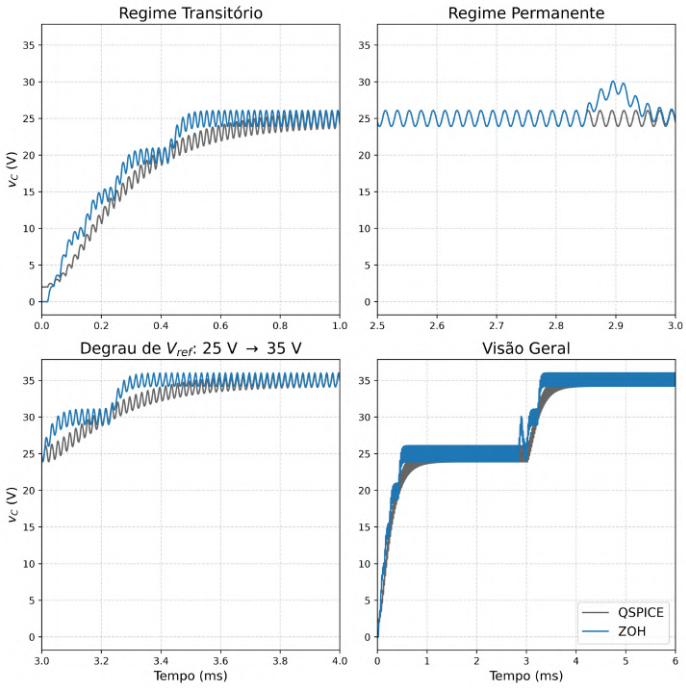
Inicialmente, foi utilizado o fator de sobreamostragem $n = 10$. Nessa condição, observou-se um erro superior ao obtido na análise apresentada na Figura 17, na qual a planta havia sido avaliada em malha aberta. Para investigar esse comportamento e avaliar a influência do fator n na precisão do modelo em malha fechada, foi realizada uma comparação sistemática entre os resultados obtidos pela simulação numérica em Python e aqueles fornecidos pelo QSPICE, adotados como referência.

A implementação em Python utiliza as equações de espaço de estados discretizadas e uma abordagem de múltiplas taxas. Nessa abordagem, a dinâmica da planta é calculada com um passo de integração menor que o período de atualização do controlador, permitindo representar de forma mais detalhada o comportamento do chaveamento PWM. O número de subdivisões do período de chaveamento é determinado pelo fator de sobreamostragem n. Para avaliar sua influência na precisão da simulação, foram considerados os seguintes valores:

$$n \in \{1, 2, \ldots, 10\} \cup \{20, 30, \ldots, 100\} \cup \{200, 300, \ldots, 1000\} \cup \{2000, 3000, \ldots, 10000\} \tag{80}$$

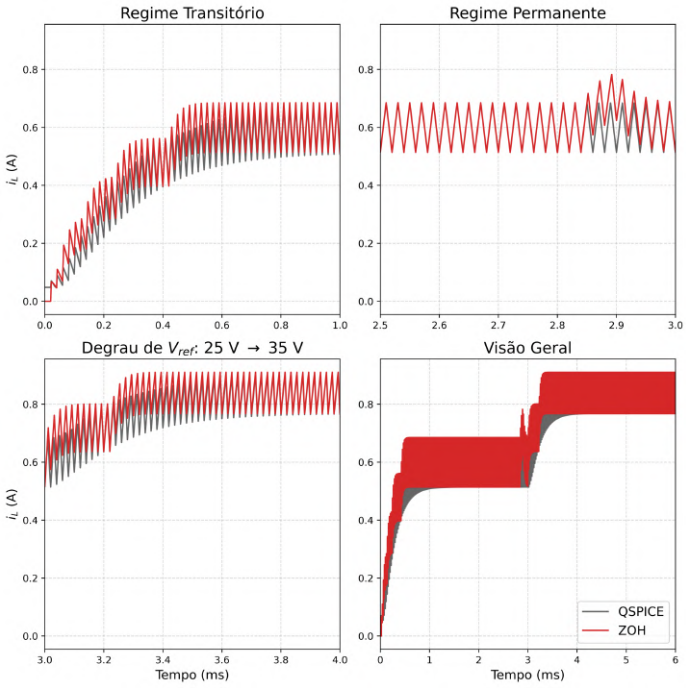
Para cada n, foram calculados os erros percentuais médios de i_L, v_C e D em relação aos resultados obtidos no QSPICE. Os valores calculados foram utilizados para gerar os gráficos apresentados abaixo, permitindo analisar a relação entre a resolução temporal da simulação e a precisão do modelo.

Figura 23 – Comparação do Sinal v_C: Simulação Numérica ($n = 10$) vs. QSPICE



Fonte: Elaborado pelo autor.

Figura 24 – Comparação do Sinal i_L: Simulação Numérica ($n = 10$) vs. QSPICE

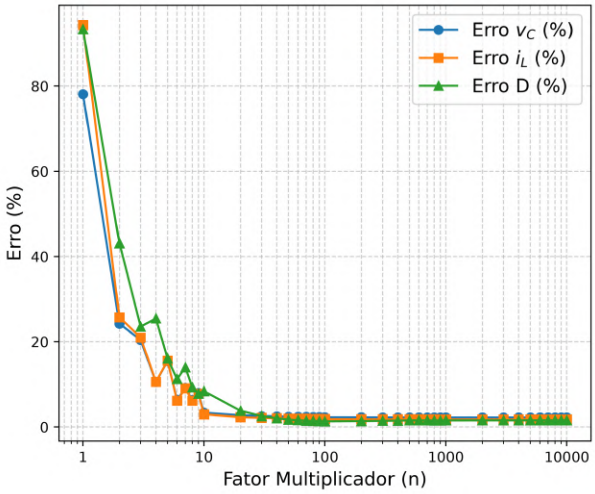


Fonte: Elaborado pelo autor.

As Figuras 23 e 24 mostram que, para $n = 10$, as formas de onda obtidas pela simulação numérica apresentam distorções e defasagens periódicas em relação ao QSPICE. Esse comportamento está relacionado à resolução temporal utilizada para representar o chaveamento PWM. Como o período de chaveamento é dividido em apenas dez intervalos, o *duty cycle* apresenta uma resolução de aproximadamente 10%. Assim, quando o controlador determina um valor intermediário, como 11%, esse valor não pode ser representado exatamente pela simulação e precisa ser aproximado para um dos níveis disponíveis. Da mesma forma, um *duty cycle* de 15% também é submetido a essa limitação de resolução.

Essa quantização do sinal PWM altera a tensão média aplicada à planta e, consequentemente, introduz diferenças nas formas de onda de i_L e v_C em relação à simulação de referência. É importante destacar que esse efeito não foi observado na validação apresentada em 4.1.2.4. Naquela etapa, o *duty cycle* foi mantido constante em 50% durante todos os testes. Dessa forma, não havia a atuação de um controlador determinando continuamente o sinal de controle, e a resolução utilizada na representação do PWM não introduzia variações adicionais no *duty cycle* ao longo da simulação. Na presente validação, por outro lado, o *duty cycle* é determinado dinamicamente pelo controlador PI e pode assumir diferentes valores ao longo do tempo. Assim, a limitação da resolução temporal utilizada para representar o PWM passa a produzir uma quantização do sinal de controle, constituindo uma fonte adicional de diferença entre o modelo numérico e a simulação de referência.

Figura 25 – Erro relativo percentual médio em malha fechada da corrente (i_L), da tensão (v_C) e *duty cycle* (D) em função do fator n, tendo o QSPICE como referência



Fonte: Elaborado pelo autor.

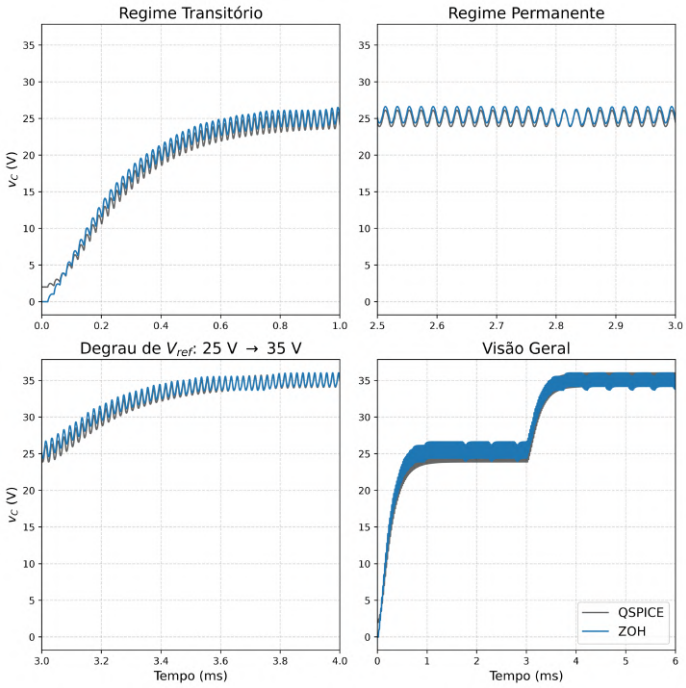
A influência do fator de sobreamostragem pode ser observada na Figura 25. A partir de $n = 10$, verifica-se uma tendência de redução do erro à medida que a resolução temporal da simulação aumenta, comportamento semelhante ao observado na análise em malha aberta apresentada na Figura 17. Entretanto, os erros obtidos em malha fechada são, em geral, superiores aos observados anteriormente. Isso ocorre porque, além das diferenças decorrentes da discretização da planta, a simulação em malha fechada está sujeita à quantização do *duty cycle* e à interação dinâmica entre o controlador e a planta.

Para $n = 10$, os erros médios de v_C, i_L e D foram de aproximadamente $3{,}34\%$, $2{,}98\%$ e $8{,}39\%$, respectivamente, indicando que essa resolução ainda é insuficiente para representar adequadamente o comportamento do sistema em malha fechada. Com o aumento de n, observa-se uma redução progressiva dos erros, principalmente para o *duty cycle*. A partir de $n = 100$, os erros já apresentam uma redução significativa, atingindo aproximadamente $2{,}29\%$ para v_C, $1{,}84\%$ para i_L e $1{,}33\%$ para D. Entretanto, como será observado nas comparações a seguir, ainda são perceptíveis diferenças entre as formas de onda do modelo numérico e do QSPICE, associadas principalmente à resolução do PWM.

As comparações apresentadas nas Figuras 26 e 27 mostram que o aumento de n proporciona uma redução significativa das diferenças entre as simulações. Para $n = 100$, o período de chaveamento é dividido em cem intervalos, permitindo representar o *duty cycle* com resolução de aproximadamente 1%. Embora essa resolução seja suficiente para reduzir consideravelmente o efeito de quantização observado para valores menores de n, ainda é possível perceber diferenças entre as formas de onda obtidas pelo modelo numérico e pelo QSPICE.

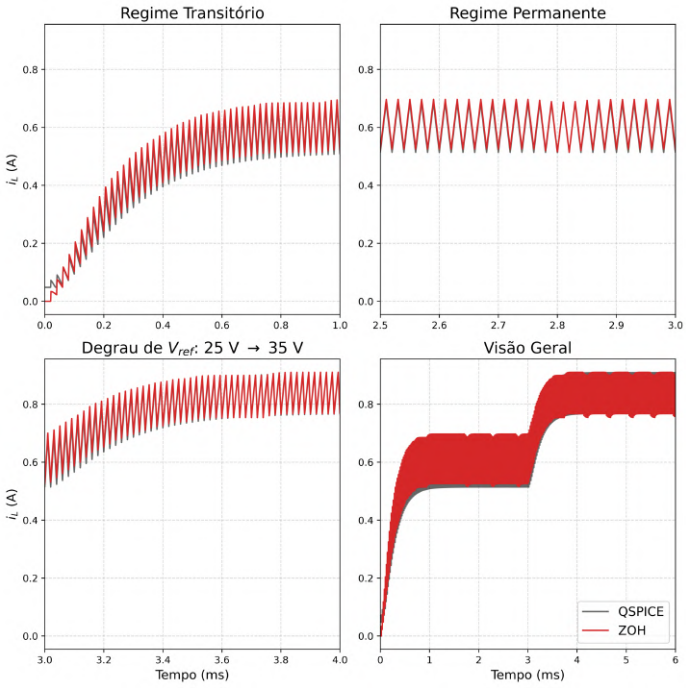
Para avaliar o efeito de uma resolução ainda maior, foram gerados os gráficos dos sinais obtidos nas simulações com $n = 1000$. Nesse caso, o período de chaveamento é dividido em mil intervalos, permitindo representar o *duty cycle* com uma resolução de aproximadamente $0{,}1\%$. Com isso, o erro de quantização do sinal de controle é ainda mais reduzido.

Figura 26 – Comparação do Sinal v_C: Simulação Numérica ($n = 100$) vs. QSPICE



Fonte: Elaborado pelo autor.

Figura 27 – Comparação do Sinal i_L: Simulação Numérica ($n = 100$) vs. QSPICE

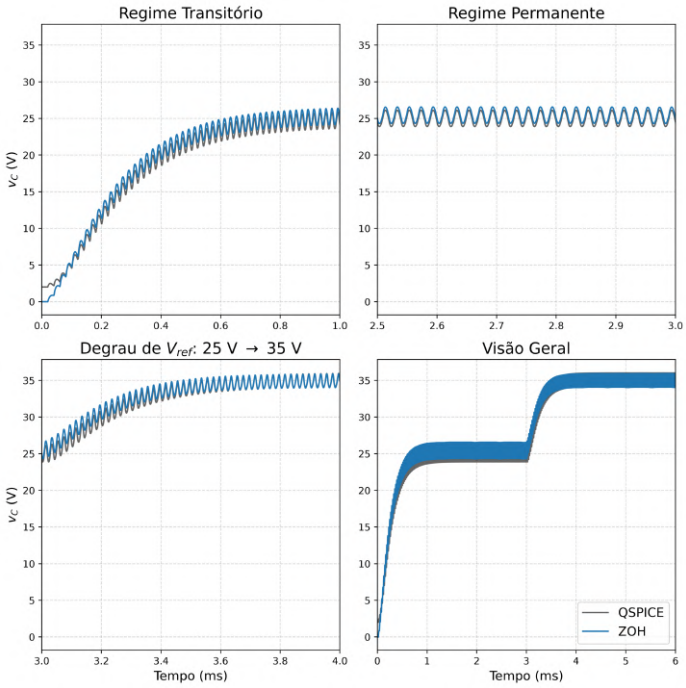


Fonte: Elaborado pelo autor.

Nas Figuras 28 e 29, observa-se que, com $n = 1000$, as diferenças entre as formas de onda do modelo numérico e do QSPICE tornam-se menos perceptíveis em comparação ao caso $n = 100$. Esse resultado evidencia que o aumento da resolução do PWM reduz o efeito da quantização do *duty cycle*, proporcionando uma representação mais próxima do comportamento observado na simulação de referência.

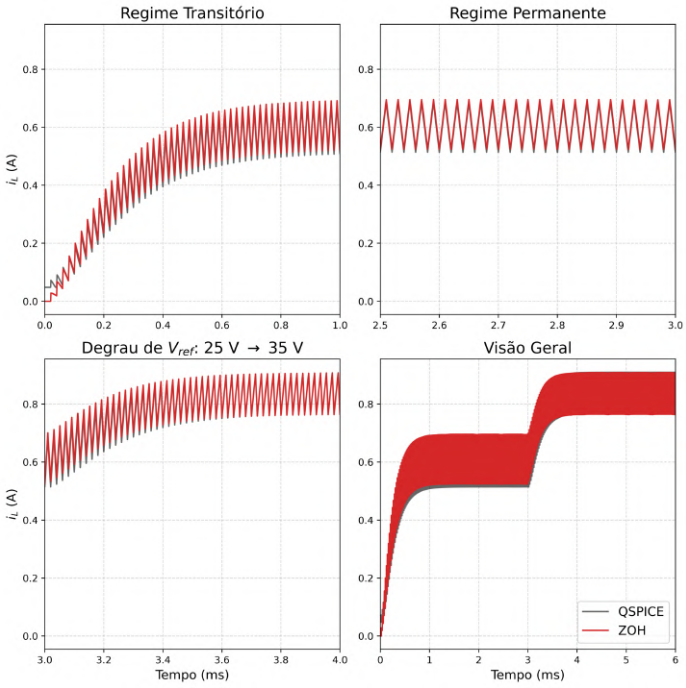
Embora fatores de sobreamostragem superiores pudessem proporcionar pequenas reduções adicionais nos erros e o valor $n = 1000$ tenha alcançado os melhores resultados de precisão, a sua adoção exigiria uma frequência de *clock* inviável para microcontroladores que possuem placas de desenvolvimento baratas e de fácil

Figura 28 – Comparação do Sinal v_C: Simulação Numérica ($n = 1000$) vs. QSPICE



Fonte: Elaborado pelo autor.

Figura 29 – Comparação do Sinal i_L: Simulação Numérica ($n = 1000$) vs. QSPICE



Fonte: Elaborado pelo autor.

acesso. Dessa forma, para garantir a viabilidade de execução em plataformas acessíveis sem comprometer a representação da dinâmica do conversor, adota-se o fator de sobreamostragem $n = 100$ nas implementações subsequentes.

4.5 Plataforma de Monitoramento e Configuração do Simulador

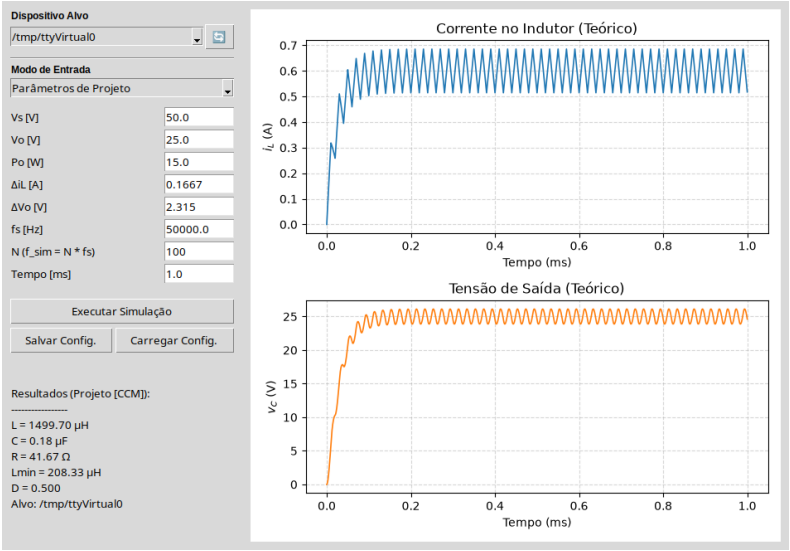
A Plataforma de Monitoramento e Configuração do Simulador foi desenvolvida integralmente em Python e organizada em dois módulos principais: o núcleo de modelagem matemática e discretização, implementado no arquivo `buck_converter.py`, e a interface gráfica com gerenciamento da comunicação serial, implementada no arquivo `buck_gui.py`. No contexto da plataforma HIL, o software executado no computador atua como ambiente hospedeiro (*host*), sendo responsável pela configuração da planta simulada, pelo cálculo dos parâmetros necessários à sua discretização, pelo envio das configurações ao sistema embarcado e pelo monitoramento dos sinais provenientes da execução em tempo real. Para permitir o desenvolvimento e a validação da interface antes da disponibilidade do microcontrolador, a comunicação serial foi inicialmente testada utilizando um dispositivo virtual, cuja implementação é apresentada no Apêndice C.

O núcleo matemático é encapsulado na classe `BuckConverterCCM`, responsável por representar o modelo físico do conversor Buck operando em CCM. A implementação utiliza as bibliotecas `NumPy` e `SciPy`, sendo empregada especificamente a função `expm` para realizar a discretização das matrizes de espaço de estados contínuas do conversor pelo método ZOH. Dessa forma, considerando as matrizes contínuas A e B, são obtidas as matrizes discretas A_d e B_d utilizadas posteriormente na execução do modelo em tempo real. O período de discretização da planta é determinado a partir da frequência de chaveamento f_s e do fator de sobreamostragem empregado na simulação, permitindo que a plataforma adapte automaticamente o modelo matemático à configuração selecionada pelo usuário.

Além da discretização, o núcleo de modelagem possui mecanismos de validação dos parâmetros elétricos fornecidos. Entre essas verificações, destaca-se a comparação entre a indutância especificada e a indutância crítica do conversor. Caso a condição $L \leq L_{\min}$ seja identificada, a plataforma impede a configuração do modelo como conversor operando em CCM e informa ao usuário que os parâmetros fornecidos podem levar o circuito ao Modo de Condução Descontínua (DCM). Essa verificação evita que o modelo matemático empregado seja utilizado fora das condições para as quais foi desenvolvido.

A interação com o usuário é realizada por meio de uma Interface Gráfica de Usuário (GUI), desenvolvida com as bibliotecas `Tkinter` e `Matplotlib`. A interface permite configurar a planta simulada por meio de duas abordagens. Na primeira, denominada entrada por parâmetros de projeto, o usuário informa os requisitos de operação do conversor, como tensão de entrada, tensão de saída, potência nominal e ondulações máximas de corrente no indutor (Δi_L) e tensão de saída (ΔV_o). A partir dessas especificações, a plataforma determina os valores dos componentes passivos necessários. A Figura 30 apresenta esse modo de configuração, no qual também é exibida a resposta simulada localmente para o conversor projetado em malha aberta, permitindo analisar as formas de onda esperadas de i_L e v_C antes da execução em tempo real.

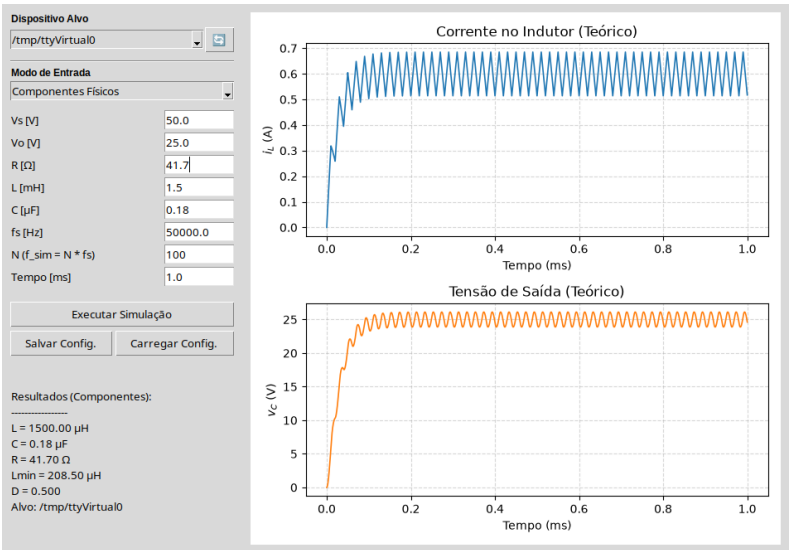
Figura 30 – Interface gráfica do simulador configurada no modo de entrada por parâmetros de projeto.



Fonte: Elaborado pelo autor.

A segunda abordagem, apresentada na Figura 31, corresponde à entrada por componentes físicos. Nesse caso, são fornecidos diretamente os parâmetros de uma planta previamente definida, como a resistência de carga R, a indutância L e a capacitância C. A interface utiliza esses valores para configurar o modelo e apresentar sua resposta simulada em malha aberta. Dessa forma, os dois modos permitem tanto a configuração da planta quanto a verificação preliminar de seu comportamento, diferenciando-se apenas pela forma de definição dos parâmetros do conversor.

Figura 31 – Interface gráfica do simulador configurada no modo de entrada por componentes físicos.



Fonte: Elaborado pelo autor.

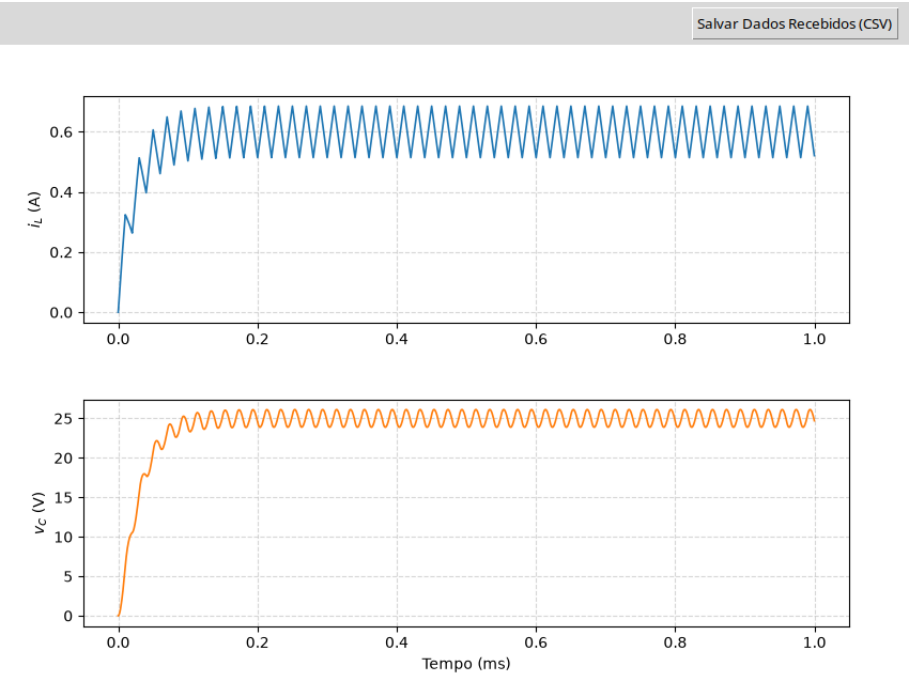
Independentemente do modo de configuração selecionado, a plataforma calcula o comportamento teórico esperado para as principais variáveis de estado do conversor, permitindo visualizar a corrente no indutor (i_L) e a tensão no capacitor (v_C). Esses resultados fornecem uma referência analítica para a execução posterior do modelo em tempo real, possibilitando verificar se o comportamento obtido pelo sistema HIL é compatível com o modelo matemático. A interface também disponibiliza mecanismos para salvar e carregar as configurações em arquivos no formato JSON, favorecendo a reprodutibilidade dos experimentos e a repetição de diferentes condições de teste.

Após a configuração da planta, a etapa de execução em tempo real é iniciada com o envio dos parâmetros ao microcontrolador. Nesse processo, a plataforma organiza as matrizes discretas A_d e B_d, juntamente com os

demais parâmetros necessários à execução do modelo, em um pacote de configuração (*payload*). Esse conjunto de dados é transmitido ao microcontrolador por meio da interface serial, utilizando a biblioteca `pyserial`. Após o recebimento correto da configuração e a confirmação por meio de uma mensagem de reconhecimento (*ACK*), a plataforma inicia a etapa de aquisição e monitoramento dos dados provenientes do sistema embarcado.

A Figura 32 apresenta a janela de monitoramento utilizada durante essa etapa, simulando o conversor projetado em Apêndice A. Nessa implementação, uma *thread* dedicada é empregada para realizar a recepção assíncrona dos dados pela interface serial, evitando que a aquisição dos dados bloqueie a execução da interface gráfica. Os valores recebidos são processados pela aplicação permitindo a representação gráfica dos sinais de tensão e corrente ao longo do tempo.

Figura 32 – Janela de monitoramento e recepção dos sinais de telemetria serial em tempo real.



Fonte: Elaborado pelo autor.

Como exemplo de funcionamento da plataforma, a execução apresentada na Figura 32 foi realizada em malha aberta. Nesse ensaio, o conversor Buck foi acionado por um sinal PWM com razão cíclica (*duty cycle*) constante de 50%, sem a atuação de um controlador de tensão em malha fechada. Essa configuração foi utilizada para verificar o funcionamento da cadeia de comunicação, a execução do modelo da planta em tempo real e a aquisição dos sinais de telemetria pelo computador, constituindo uma etapa preliminar de validação da plataforma HIL.

Portanto, o resultado apresentado nessa figura não representa a validação do sistema de controle em malha fechada, mas sim a verificação do funcionamento da infraestrutura de simulação em tempo real. A utilização de um *duty cycle* fixo de 50% permite estabelecer uma condição de operação conhecida para o conversor, facilitando a comparação entre o comportamento esperado pelo modelo matemático e os sinais obtidos durante a execução no microcontrolador.

Durante a aquisição, os dados recebidos são armazenados pela aplicação e apresentados graficamente em tempo real, permitindo acompanhar a evolução das variáveis da planta. Ao final do ensaio, a plataforma permite exportar a série temporal adquirida para um arquivo no formato CSV. Esse recurso possibilita o processamento posterior dos resultados, incluindo a comparação quantitativa entre a resposta obtida no *hardware* e a resposta calculada pelo modelo de referência, além da utilização dos dados em ferramentas externas de análise e visualização.

4.6 Implementação do Firmware do Controlador

O *firmware* do controlador PI digital, projetado na Seção 4.3, foi implementado em linguagem C no microcontrolador STM32F103C8T6, escolhido por já estar disponível em uma placa de desenvolvimento do autor, facilitando a implementação e a realização dos testes experimentais. Para configurar e gerenciar os periféricos, foram utilizadas as bibliotecas HAL. A implementação utiliza um temporizador para estabelecer a base de tempo do sistema e sincronizar a geração do sinal PWM com a aquisição das amostras pelo ADC.

O Código 2 apresenta a configuração do temporizador TIM1. Seus parâmetros definem o período de contagem e, consequentemente, a frequência do PWM e a taxa de amostragem do controlador, de acordo com a configuração do relógio do microcontrolador. O valor do registrador de recarga (*Auto-Reload Register* - ARR), configurado no parâmetro `Period`, foi definido como $1440-1$ para estabelecer a frequência de operação exata de 50 kHz. Como o *prescaler* está configurado em 0 (sem divisão prévia), o temporizador conta os ciclos diretamente na velocidade do barramento de *clock*. Considerando que o temporizador é alimentado por um *clock* base de 72 MHz, a divisão dessa frequência pelo período total de contagem resulta na taxa desejada ($72.000.000$ Hz$/1440 = 50.000$ Hz).

Código 2 – Configuração do temporizador do controlador PI.

```
1  htim1.Instance = TIM1;
2  htim1.Init.Prescaler = 0;
3  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
4  htim1.Init.Period = 1440-1;
5  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
6  htim1.Init.RepetitionCounter = 0;
7  htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
8
```

Fonte: Elaborado pelo autor.

A aquisição da tensão é realizada pelo ADC, configurado no modo injetado. Nesse modo, as conversões são realizadas a partir de um disparo específico, que pode ser gerado por um evento de hardware do temporizador. No *firmware* desenvolvido, esse mecanismo permite sincronizar a aquisição da tensão com o ciclo de chaveamento do conversor, definindo os instantes de amostragem de forma previsível.

Ao término de cada conversão injetada, uma interrupção aciona a rotina de controle apresentada no Código 3. Para fins de avaliação de desempenho computacional, o tempo de execução dessa rotina é mensurado por meio do contador de ciclos do periférico DWT (*Data Watchpoint and Trigger*). Nessa rotina, a leitura do ADC é convertida para a escala utilizada pelo controlador, o erro entre a referência e a tensão medida é calculado e a lei de controle PI digital é executada. O sinal de controle é limitado ao intervalo $[0,1]$, correspondente aos valores mínimo e máximo da razão cíclica. Em seguida, esse valor é convertido para a escala de contagem do temporizador e escrito no registrador de comparação do PWM, `CCR1`, definindo a razão cíclica aplicada à saída. Por fim, captura-se novamente o valor do contador DWT para registrar o pior caso de ciclos de *clock* consumidos.

Código 3 – Rotina de interrupção do ADC para execução do controlador PI.

```
1  void HAL_ADCEx_InjectedConvCpltCallback(ADC_HandleTypeDef *hadc) {
2    if (hadc->Instance == ADC1) {
3      ciclos_inicio = DWT->CYCCNT; // <--- CAPTURA O CLOCK INICIAL
4
5      uint32_t adc_raw = HAL_ADCEx_InjectedGetValue(&hadc1, ADC_INJECTED_RANK_1);
6      float IN = (float)adc_raw * 0.0122100122f;
7
8      t_ticks++;
9      if (t_ticks >= TICKS_PARA_DEGRAU) Vref = VREF_INICIAL + VREF_INCREMENTO;
10     else Vref = VREF_INICIAL;
11
12     float E = Vref - IN;
13     float U = pastU + 0.000357719f * E + 0.000346881f * pastE;
14
15     if (U > 1.0f) U = 1.0f;
16     else if (U < 0.0f) U = 0.0f;
17
18     pastE = E;
19     pastU = U;
20
21     TIM1->CCR1 = (uint32_t)(U * 1440.0f); // Saida do PWM
22
23     ciclos_fim = DWT->CYCCNT; // <--- CAPTURA O CLOCK FINAL
24     ciclos_atual = ciclos_fim - ciclos_inicio;
25     if (ciclos_atual > ciclos_maximo)  ciclos_maximo = ciclos_atual;
26   }
27 }
28
```

Fonte: Elaborado pelo autor.

A inicialização dos periféricos e a partida do sistema são apresentadas no Código 4. Primeiro, são configurados os GPIOs, o ADC e o temporizador. Em seguida, são iniciados o PWM, a contagem do temporizador, o ADC com interrupção e, por fim, os registradores do *CoreDebug* e do DWT são habilitados para permitir a contagem precisa de ciclos de máquina. A partir da chamada `HAL_ADCEx_InjectedStart_IT()`, as conversões passam a ser acionadas pelos eventos de *hardware* configurados, mantendo o sincronismo entre a aquisição e a execução do controlador.

Código 4 – Inicialização dos periféricos do controlador PI.

```
1  /* Initialize all configured peripherals */
2  MX_GPIO_Init();
3  MX_ADC1_Init();
4  MX_TIM1_Init();
5
6  /* USER CODE BEGIN 2 */
7  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1); // Inicia o gerador de PWM
8  HAL_TIM_Base_Start(&htim1); // Inicia o Timer base (para gerar os eventos TRGO)
9  HAL_ADCEx_InjectedStart_IT(&hadc1); // Inicia o ADC em modo injetado com interrupcao
10 CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; // Habilita o Trace
11 DWT->CYCCNT = 0;                                // Zera o contador
12 DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;            // Habilita o contador
13 /* USER CODE END 2 */
14
```

Fonte: Elaborado pelo autor.

Considerando que o microcontrolador opera com um *clock* base de 72 MHz e a frequência de amostragem do controlador é de 50 kHz, o intervalo de tempo disponível para a execução da rotina de controle antes da próxima amostragem corresponde a 1440 ciclos de *clock*. Durante os ensaios experimentais, após manter o controlador em operação contínua por 5 minutos, o número máximo de ciclos consumidos na rotina de interrupção, medido pelo DWT, foi de 987 ciclos.

Cabe destacar que o STM32F103C8T6 não possui uma FPU em *hardware*. Consequentemente, as operações aritméticas com variáveis do tipo `float` presentes no algoritmo são emuladas via *software*, elevando a quantidade de ciclos de *clock* exigidos. Embora a adoção de um microcontrolador provido de FPU resultasse em um tempo de execução substancialmente menor, o consumo observado de 987 ciclos representa aproximadamente $68{,}5\%$ da capacidade de processamento disponível em cada período de amostragem, restando uma margem de 453 ciclos. Esse resultado demonstra a viabilidade da implementação do controlador PI a 50 kHz no microcontrolador usado. A margem de processamento disponível permite atender ao período de amostragem sem que a rotina de controle ultrapasse o intervalo previsto, evitando o encavalamento das execuções. Dessa forma, os resultados justificam a manutenção do *hardware* selecionado, dispensando a migração para um componente de maior custo ou complexidade.

Apesar de a viabilidade computacional do controlador original ter sido comprovada no *hardware* selecionado, a execução conjunta com a simulação HIL pode impor gargalos de processamento. Caso o simulador da planta não consiga alcançar a taxa de execução exigida de 5 MHz para manter a fidelidade e a estabilidade da simulação em tempo real frente a um controle de 50 kHz, torna-se necessário adotar uma abordagem com menor exigência temporal. Nesses cenários, o *firmware* pode ser adaptado para utilizar o controlador alternativo, apresentado no Apêndice B, cuja frequência de amostragem é de 5 kHz. Para isso, mantendo o mesmo *clock* e o *prescaler*, o período de contagem do temporizador deve ser aumentado em dez vezes, passando de `1440-1` para `14400-1`. Da mesma forma, o fator de escala utilizado na atualização do registrador `CCR1` deve passar de 1440 para 14400, e os coeficientes da equação de diferenças devem ser substituídos pelos valores correspondentes ao controlador alternativo. Ressalta-se que a validação da taxa de 50 kHz no *hardware* já garante folga de processamento para a versão de 5 kHz, dispensando a reavaliação de sua viabilidade computacional.

4.7 Implementação do Simulador Digital em Tempo Real (RTDS)

Nesta seção são apresentadas as etapas de implementação física e algorítmica do RTDS, contemplando a transição dos modelos matemáticos previamente validados para uma plataforma embarcada dedicada. Para isso, inicialmente são definidos os requisitos mínimos de *hardware* necessários à execução do simulador em tempo real, considerando as restrições de processamento, tempo de execução, aquisição do sinal de controle e geração dos sinais de saída.

Conforme demonstrado na Seção 4.4, a adoção de um fator de multiplicidade $n = 100$ em relação à frequência de chaveamento apresenta precisão adequada para a representação da dinâmica do circuito. Considerando a frequência de chaveamento do conversor Buck projetado, $F_s = 50$ kHz, a frequência mínima de simulação necessária é de:

$$F_{sim} = F_s \cdot n = 50 \text{ kHz} \cdot 100 = 5 \text{ MHz} \tag{81}$$

Essa frequência estabelece uma janela de apenas 200 ns por passo de simulação, durante a qual o microcontrolador deve executar o cálculo dos estados do modelo e atualizar as suas saídas. Além da capacidade de processamento, o *hardware* deve possuir periféricos de entrada capazes de realizar a aquisição do sinal de controle externo (*duty cycle*) de forma sincronizada à frequência de chaveamento do conversor, $F_s = 50$ kHz. Também são necessários periféricos de saída adequados à exportação dos sinais simulados.

Outro requisito está relacionado à representação numérica utilizada nos cálculos. É necessário verificar se a precisão simples é suficiente para a execução do modelo ou se seria necessário utilizar operações em precisão dupla. Da mesma forma, deve-se avaliar a resolução dos conversores D/A, uma vez que a quantização dos sinais de saída pode afetar a representação da planta simulada quando os sinais são posteriormente aplicados à interface analógica.

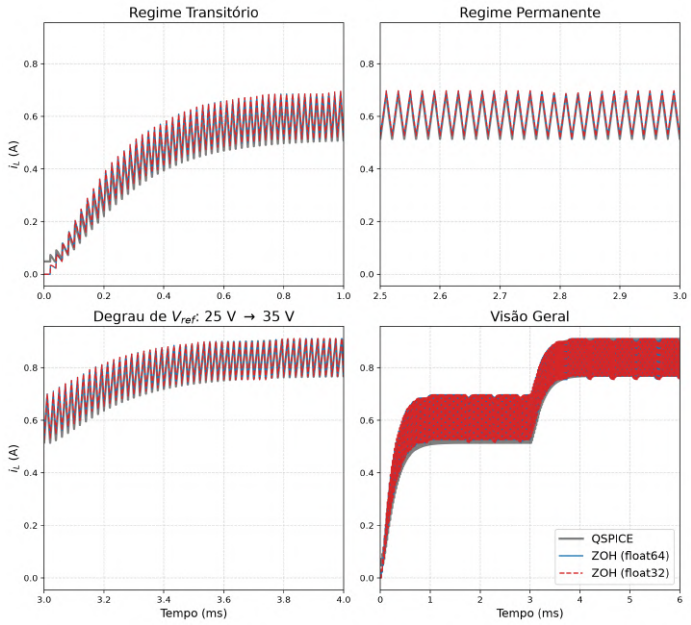
A partir desses requisitos, são realizadas as análises necessárias para avaliar as características do *hardware* e sua adequação à implementação do RTDS. Entre essas análises, a Subseção 4.7.1 avalia a influência da precisão numérica da Unidade de Ponto Flutuante (FPU) nos cálculos do modelo, enquanto a Subseção 4.7.2 analisa o efeito de diferentes resoluções do conversor Digital-Analógico (DAC) sobre a qualidade dos sinais de saída da simulação. Na sequência, a Subseção 4.7.3 apresenta o dimensionamento computacional necessário para a execução do modelo e fundamenta a seleção do microcontrolador utilizado na implementação do RTDS.

4.7.1 Análise do Impacto da Resolução da FPU no Desempenho do Simulador

Para atestar o requisito de *hardware* referente à Unidade de Ponto Flutuante (FPU), é necessário definir se o microcontrolador requer suporte nativo à precisão dupla (64 bits) ou se a precisão simples (32 bits) é suficiente para garantir a estabilidade numérica e evitar erros de quantização no algoritmo de controle sem comprometer o desempenho.

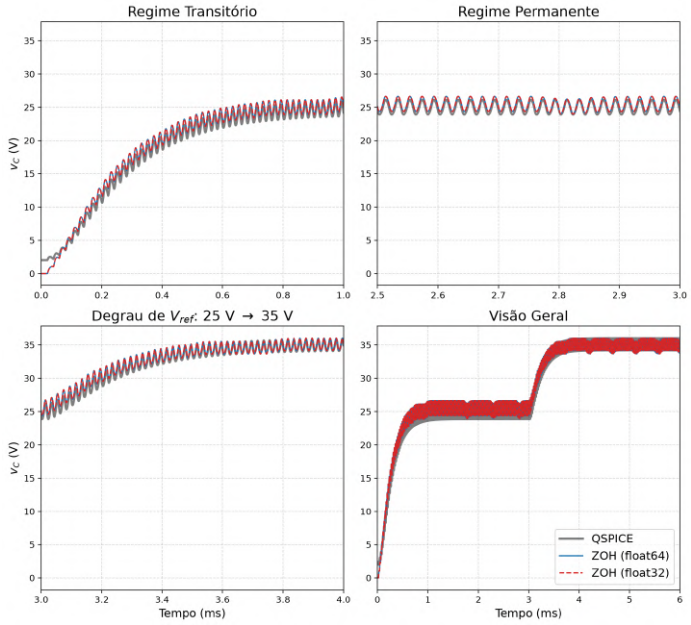
Para validar esta restrição, a planta matemática em malha fechada foi emulada digitalmente forçando as variáveis e coeficientes do sistema para as precisões `float32` e `float64`. Os resultados comparativos dos transitórios de tensão e corrente estão dispostos nas Figuras 33 e 34 a seguir.

Figura 33 – Comparação do sinal de corrente no indutor i_L: Precisão Simples (`float32`) vs. Dupla (`float64`) e Referência QSPICE



Fonte: Elaborado pelo autor.

Figura 34 – Comparação do sinal de tensão no capacitor v_C: Precisão Simples (`float32`) vs. Dupla (`float64`) e Referência QSPICE



Fonte: Elaborado pelo autor.

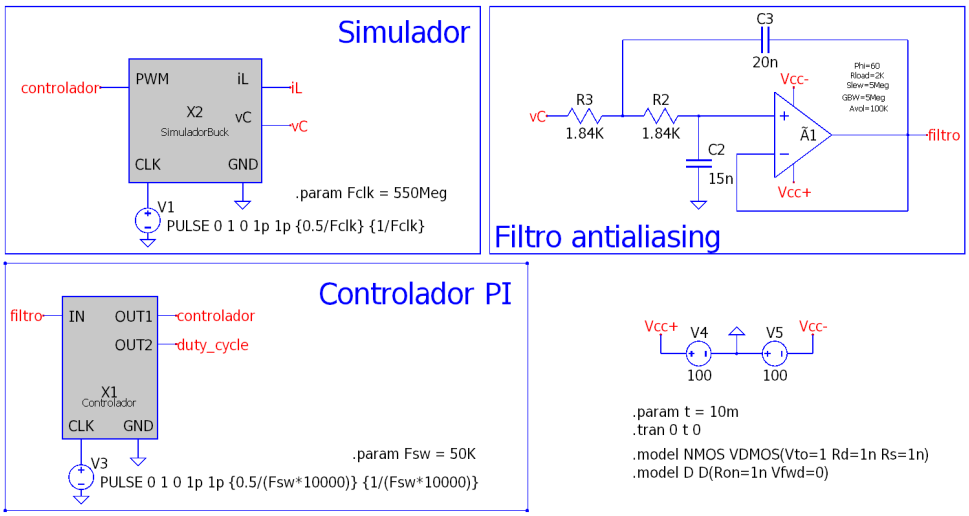
Como pode ser observado nas imagens geradas, as curvas de tensão e corrente processadas em 32 bits sobrepõem-se quase perfeitamente às referências obtidas em 64 bits, tanto durante o regime transitório quanto em regime permanente. Os resultados indicam, portanto, que o erro de truncamento associado à utilização da precisão simples é desprezível para a aplicação e não compromete a estabilidade do sistema, consolidando o emprego de um microcontrolador equipado com uma FPU de 32 bits.

4.7.2 Análise do Impacto da Resolução do DAC no Desempenho do Simulador

Para analisar o impacto da resolução do conversor D/A na qualidade da modelagem e no desempenho dinâmico do sistema, auxiliando diretamente na seleção do microcontrolador ideal para a plataforma embarcada, o modelo de simulação no QSPICE foi estruturado de forma parametrizada. Essa abordagem permite avaliar os efeitos de diferentes tamanhos de palavra digital (por exemplo, 8, 10, 12, 16 e 18 bits) sobre o erro de quantização e o comportamento da malha de controle.

A Figura 35 ilustra a visão geral do circuito de simulação no QSPICE. Nele, os blocos C que simulam microcontroladores representam a planta e a malha de controle: a planta do conversor é implementada pelo bloco Simulador (`X2 SimuladorBuck`), responsável por resolver numericamente as equações do Buck, enquanto a estratégia de controle é embarcada no bloco Controlador PI (`X1 Controlador`), que processa a tensão de saída filtrada e atualiza a razão cíclica enviada ao simulador.

Figura 35 – Circuito com simulação do simulador do Buck implementado em microcontrolador no QSPICE.



Fonte: Elaborado pelo autor.

Para simular diferentes limitações de resolução do *hardware*, a rotina foi parametrizada por meio de uma diretiva de pré-processamento que permite definir a resolução do DAC. O Código 5 apresenta a configuração desses parâmetros, incluindo a macro `DAC_BITS`, que pode ser ajustada para avaliar o comportamento do simulador com resoluções de 8, 10, 12, 16 ou 18 bits, juntamente com a função responsável pela quantização do sinal de saída.

Código 5 – Configuração do DAC e função de quantização do modelo do Buck no bloco SimuladorBuck no QSPICE.

```
// CONFIGURAÇÃO DO DAC
static const float VDAC_MAX = 3.3f;    // Tensão máxima do DAC [V]
static const float VC_MAX   = 50.0f;   // Faixa máxima de tensão da planta [V]
static const float IL_MAX   = 5.0f;    // Faixa máxima de corrente da planta [A]
static const unsigned int DAC_BITS = 12; // Resolução do DAC
static const unsigned int DAC_LEVELS = (1u << DAC_BITS) - 1;

static float quantizeDAC(float voltage)
{
    if (voltage < 0.0f) voltage = 0.0f;
    if (voltage > VDAC_MAX) voltage = VDAC_MAX;

    // Converte tensão para código digital
    float code = voltage * (float)DAC_LEVELS / VDAC_MAX;

    // Arredonda para o código inteiro mais próximo
    unsigned int code_quantized = (unsigned int)(code + 0.5f);

    // Converte novamente o código para tensão
    return ((float)code_quantized * VDAC_MAX) / (float)DAC_LEVELS;
}
```

Fonte: Elaborado pelo autor.

Em seguida, o núcleo da simulação do conversor Buck é executado por meio da rotina descrita no Código 6. O algoritmo utiliza um sinal de clock de referência de 550 MHz, escolhido por corresponder ao limite superior de frequência de operação entre os microcontroladores da família STM32H7 avaliados para este projeto (STMicroelectronics, 2026c). A cada 110 ciclos desse clock, o modelo é atualizado, resultando em uma frequência de simulação de 5 MHz. Esse mecanismo permite estabelecer o intervalo temporal entre as atualizações do modelo no QSPICE, com aplicação da quantização dinâmica correspondente à resolução do DAC em teste. Como o QSPICE não executa o modelo em tempo real, o intervalo de 110 ciclos serve apenas como referência temporal para a atualização do modelo, sem impor um limite ao tempo de processamento necessário para cada atualização.

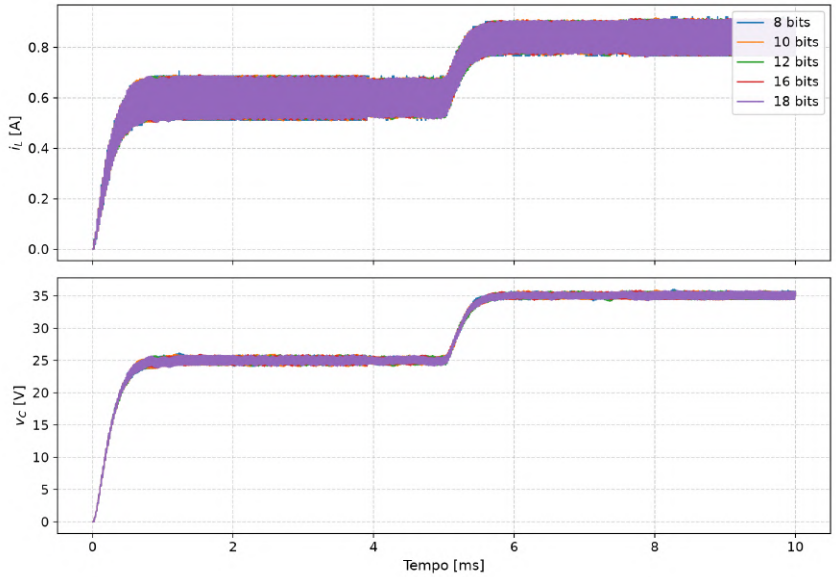
Código 6 – Implementação do algoritmo do controle PI no bloco Controlador no QSPICE.

```
1  extern "C" __declspec(dllexport)
2  void simuladorbuck(void **opaque, double t, union uData *data)
3  {
4      bool PWM = data[0].b; // Entrada PWM
5      bool CLK = data[1].b; // Clock de 550 MHz
6      double &iL = data[2].d; // Saída para corrente -> tensão DAC
7      double &vC = data[3].d; // Saída para tensão -> tensão DAC
8
9      // VARIÁVEIS DO TEMPORIZADOR
10     static unsigned int CLK_counter = 0;
11     static bool CLK_pastVal = false;
12
13     // VARIÁVEIS DO DAC
14     static float vC_DAC = 0.0f, iL_DAC = 0.0f;
15
16     // VARIÁVEIS DO MODELO DO BUCK
17     static const float Ad_00 = 0.9999265803813119f;
18     static const float Ad_01 = -0.00013156940067175223f;
19     static const float Ad_10 = 1.0964116722646022f;
20     static const float Ad_11 = 0.9736337345236475f;
21     static const float Bd1_00_Vs = 0.00666650314952061f;
22     static const float Bd1_10_Vs = 0.0036709809344059874f;
23     static const float Bd2_00_Vs = 0.0f, Bd2_10_Vs = 0.0f;
24     static float iL_prev = 0.0f, vC_prev = 0.0f;
25     static float iL_aux = 0.0f, vC_aux = 0.0f;
26
27     if (CLK && !CLK_pastVal) // DETECÇÃO DA BORDA DE SUBIDA DO CLOCK
28     {
29         CLK_counter++;
30
31         // EXECUTA O MODELO DO BUCK A 5 MHz
32         if (CLK_counter >= 110) // 550 MHz / 5 MHz = 110
33         {
34             CLK_counter = 0;
35
36             if (PWM) { // PWM = 1 -> Chave fechada
37                 iL_aux = Ad_00 * iL_prev + Ad_01 * vC_prev + Bd1_00_Vs;
38                 vC_aux = Ad_10 * iL_prev + Ad_11 * vC_prev + Bd1_10_Vs;
39             }
40             else { // PWM = 0 -> Chave aberta
41                 iL_aux = Ad_00 * iL_prev + Ad_01 * vC_prev + Bd2_00_Vs;
42                 vC_aux = Ad_10 * iL_prev + Ad_11 * vC_prev + Bd2_10_Vs;
43             }
44
45             // ATUALIZAÇÃO DOS ESTADOS
46             iL_prev = iL_aux;
47             vC_prev = vC_aux;
48
49             // CONDICIONAMENTO PARA O DAC
50             vC_DAC = vC_aux * VDAC_MAX / VC_MAX;
51             iL_DAC = iL_aux * VDAC_MAX / IL_MAX;
52             vC_DAC = quantizeDAC(vC_DAC);
53             iL_DAC = quantizeDAC(iL_DAC);
54         }
55     }
56
57     iL = (double)iL_DAC;
58     vC = (double)vC_DAC;
59     CLK_pastVal = CLK;
60 }
```

Fonte: Elaborado pelo autor.

A Figura 36 apresenta o comportamento geral da corrente no indutor e da tensão no capacitor ao longo de toda a simulação.

Figura 36 – Comportamento completo da corrente no indutor e tensão no capacitor para diferentes resoluções do DAC.

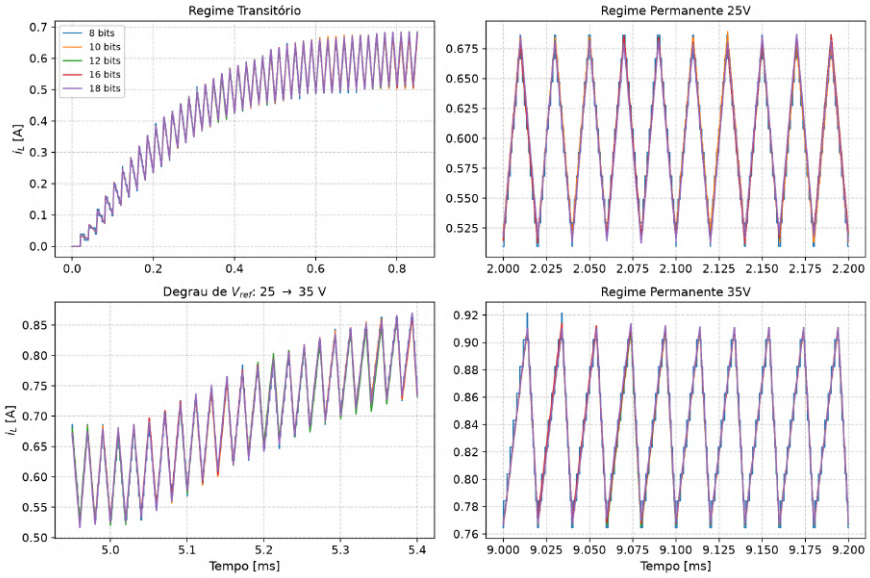


Fonte: Elaborado pelo autor.

No entanto, devido à ampla escala temporal e de amplitude do gráfico da Figura 36, não é possível observar de maneira adequada as diferenças impostas pela quantização de cada resolução nesta visão global. Para contornar essa limitação e permitir uma avaliação precisa do impacto da limitação de bits, os sinais foram detalhados em quatro intervalos de tempo específicos, apresentados nas Figuras 37 e 38. Os recortes selecionados ilustram os seguintes momentos críticos de operação do conversor:

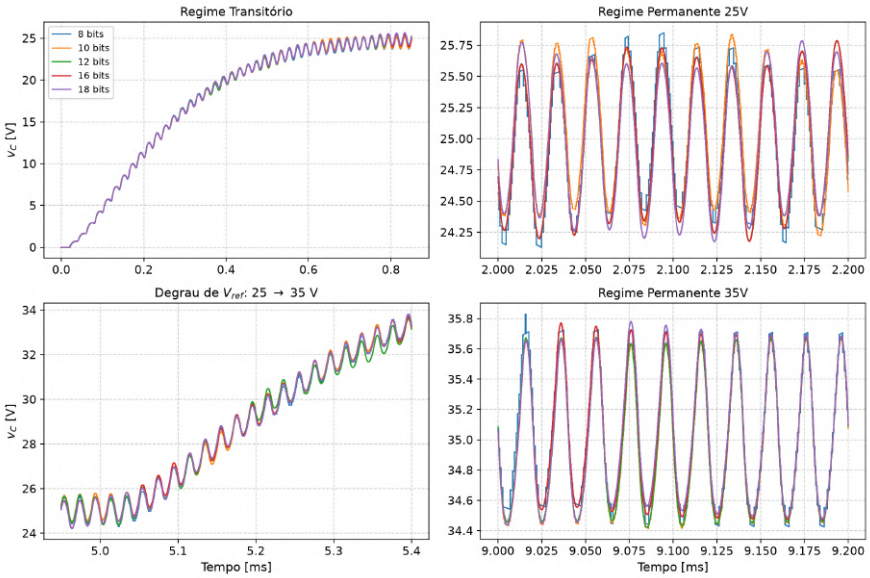
- **0,0 a 0,85 s**: Regime transitório de inicialização;
- **2,0 a 2,2 s**: Regime permanente operando com 25 V;
- **4,95 a 5,4 s**: Resposta ao degrau de V_{ref} (transição de 25 V $\rightarrow$ 35 V);
- **9,0 a 9,2 s**: Regime permanente operando com 35 V.

Figura 37 – Detalhamento da corrente no indutor (i_L) em diferentes etapas de operação em função da resolução do DAC.



Fonte: Elaborado pelo autor.

Figura 38 – Detalhamento da tensão no capacitor (v_C) em diferentes etapas de operação em função da resolução do DAC.

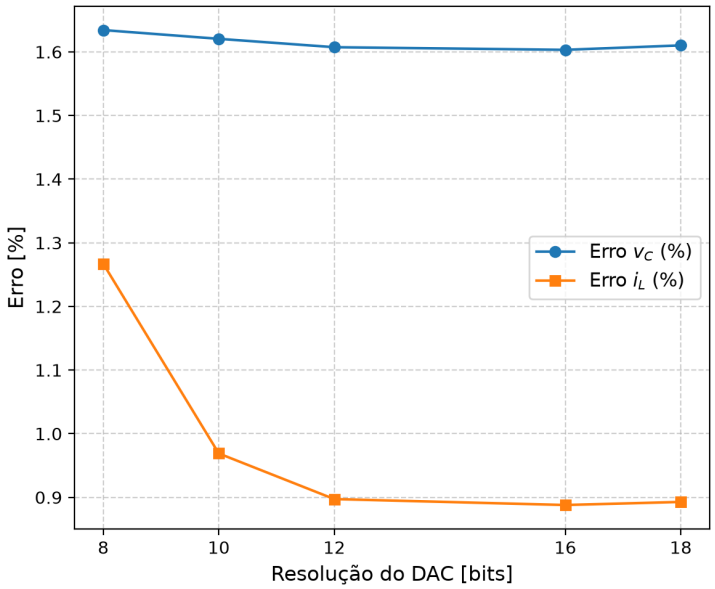


Fonte: Elaborado pelo autor.

A análise das formas de onda detalhadas revela o peso do erro de quantização no sinal sintetizado. Para as resoluções de 8 e 10 bits, pode-se observar claramente a formação de degraus, afastando-se da dinâmica contínua real de um conversor. Em contrapartida, a partir de 12 bits de resolução, as curvas se tornam substancialmente mais suaves e realistas, mitigando os efeitos visíveis e sistêmicos da quantização.

Para avaliar de forma quantitativa o comportamento do erro em função da resolução do DAC, realizou-se uma comparação entre a simulação digital, na qual a planta é simulada por um microcontrolador (Figura 35), e a simulação de referência, cuja planta é o próprio circuito analógico real (Figura 22). O resultado dessa análise comparativa é ilustrado na Figura 39.

Figura 39 – Erro relativo percentual médio em malha fechada da corrente (i_L) e da tensão (v_C) em função do número de bits do DAC, tendo uma simulação analógica QSPICE como referência.



Fonte: Elaborado pelo autor.

Com base nestes resultados, a tomada de decisão para a seleção do microcontrolador a ser utilizado na plataforma embarcada recai sobre duas diretrizes principais de projeto. A primeira e mais direta é a escolha de um dispositivo que possua um DAC integrado com resolução de, no mínimo, 12 bits. Caso restrições de custo ou disponibilidade exijam a utilização de um microcontrolador com um DAC de menor resolução, será adequada a inclusão de um filtro passa-baixa analógico externo no circuito. Esse filtro terá a função de suavizar a saída do conversor D/A, atenuando os degraus de quantização e tornando o sinal sintetizado mais fiel ao comportamento dinâmico da planta analógica real.

4.7.3 Dimensionamento Computacional e Seleção do Microcontrolador

Esta Subseção tem como objetivo selecionar o microcontrolador a ser utilizado no projeto, considerando a capacidade computacional necessária para a execução do modelo do conversor Buck em tempo real. Para isso, a partir das equações discretizadas do conversor, foi desenvolvido um código em C com estrutura semelhante à de uma implementação embarcada do modelo, conforme apresentado no Código 7.

Código 7 – Código em C estruturado para simulação e estimativa de ciclos do conversor Buck.

```c
#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

typedef struct {
    float iL;
    float vC;
} BuckState;

static const float Ad_00 = 0.9999265803813119f;
static const float Ad_01 = -0.00013156940067175223f;
static const float Ad_10 = 1.0964116722646022f;
static const float Ad_11 = 0.9736337345236475f;

static const float Bd1_00_Vs = 0.00666650314952061f;
static const float Bd1_10_Vs = 0.0036709809344059874f;

static const float Bd2_00_Vs = 0.0f;
static const float Bd2_10_Vs = 0.0f;

__attribute__((noinline))
void buck_step(BuckState *x, bool switch_closed) {
    float iL = x->iL;
    float vC = x->vC;

    if (switch_closed) {
        x->iL = Ad_00 * iL + Ad_01 * vC + Bd1_00_Vs;
        x->vC = Ad_10 * iL + Ad_11 * vC + Bd1_10_Vs;
    } else {
        x->iL = Ad_00 * iL + Ad_01 * vC + Bd2_00_Vs;
        x->vC = Ad_10 * iL + Ad_11 * vC + Bd2_10_Vs;
    }
}

int main(void) {
    BuckState state = { .iL = 0.0f, .vC = 0.0f };
    const uint32_t N_steps = 29999;

    for (uint32_t k = 0; k < N_steps - 1; k++) {
        bool switch_closed = (k % 100) < 50;
        buck_step(&state, switch_closed);
    }

    return 0;
}
```

Fonte: Elaborado pelo autor.

O código é compilado para a arquitetura alvo ARM Cortex-M7 por meio da ferramenta `arm-none-eabi-gcc`, utilizando o nível máximo de otimização (`-O3`) e suporte à FPU. A partir do Assembly gerado, são identificadas as instruções relevantes e atribuído um custo de processamento a cada uma delas: 1 ciclo para as operações de ponto flutuante (`vmul.f32`, `vfma.f32` e `vadd.f32`), 1 ciclo para as leituras de memória (`vldr`) e 2 ciclos para as escritas (`vstr`). Para as instruções de controle de fluxo `cbz`, `b` e `bx`, considera-se um custo de 2 ciclos. Esses valores são utilizados como base para a estimativa do custo computacional de cada passo.

No trecho analisado, foram identificadas 9 instruções `vldr`, 2 `vmul.f32`, 2 `vfma.f32`, 1 `cbz`, 4 `vadd.f32`, 4 `vstr` e 2 `bx`. A aplicação dos custos de execução definidos para cada tipo de instrução resulta em um total estimado de 31 ciclos de clock por passo de execução do modelo.

O número de ciclos de clock disponíveis em cada passo de simulação foi calculado a partir da relação

$$N_{\mathrm{disp}} = \frac{f_{\mathrm{CPU}}}{f_{\mathrm{sim}}}, \tag{82}$$

em que N_{disp} representa o número de ciclos de clock disponíveis por passo, f_{CPU} corresponde à frequência de operação do microcontrolador e f_{sim} à frequência de execução da simulação.

A porcentagem de ciclos ocupados pelo modelo foi então estimada pela razão entre o número de ciclos necessários para sua execução e o número de ciclos disponíveis em cada passo, conforme

$$O = \frac{N_{\mathrm{modelo}}}{N_{\mathrm{disp}}} \times 100, \tag{83}$$

em que N_{modelo} corresponde ao número de ciclos estimados para a execução do modelo.

Com base nesse procedimento, a Tabela 1 apresenta a quantidade de ciclos disponíveis por passo e a respectiva ocupação estimada para cada microcontrolador, contemplando tanto o projeto principal quanto o projeto do buck alternativo, com frequência de chaveamento de 5 kHz e frequência de simulação de 500 kHz, a ser utilizado caso haja limitação de processamento, conforme detalhado no Anexo B.

Tabela 1 – Comparação da capacidade computacional e dos periféricos analógicos de microcontroladores STM32 para o projeto principal e o projeto alternativo.

Microcontrolador	Núcleo	$f_{\mathrm{CPU_{max}}}$	Projeto principal		Buck alternativo		DAC	FPU
			Ciclos/Passo	Ocupação	Ciclos/Passo	Ocupação		
STM32F767ZI	Cortex-M7	216 MHz	43,2	71,76%	432,0	7,18%	12 bits	Single/Double
STM32F769NI	Cortex-M7	216 MHz	43,2	71,76%	432,0	7,18%	12 bits	Single/Double
STM32H7A3ZI	Cortex-M7	280 MHz	56,0	55,36%	560,0	5,54%	12 bits	Single/Double
STM32H7B3ZI	Cortex-M7	280 MHz	56,0	55,36%	560,0	5,54%	12 bits	Single/Double
STM32H742ZI	Cortex-M7	480 MHz	96,0	32,29%	960,0	3,23%	12 bits	Single/Double
STM32H743ZI	Cortex-M7	480 MHz	96,0	32,29%	960,0	3,23%	12 bits	Single/Double
STM32H750VB	Cortex-M7	480 MHz	96,0	32,29%	960,0	3,23%	12 bits	Single/Double
STM32H753ZI	Cortex-M7	480 MHz	96,0	32,29%	960,0	3,23%	12 bits	Single/Double
STM32H723ZI	Cortex-M7	550 MHz	110,0	28,18%	1100,0	2,82%	12 bits	Single/Double
STM32H723ZG	Cortex-M7	550 MHz	110,0	28,18%	1100,0	2,82%	12 bits	Single/Double
STM32H733ZI	Cortex-M7	550 MHz	110,0	28,18%	1100,0	2,82%	12 bits	Single/Double
STM32H725ZI	Cortex-M7	550 MHz	110,0	28,18%	1100,0	2,82%	12 bits	Single/Double
STM32H735ZI	Cortex-M7	550 MHz	110,0	28,18%	1100,0	2,82%	12 bits	Single/Double
STM32H730VB	Cortex-M7	550 MHz	110,0	28,18%	1100,0	2,82%	12 bits	Single/Double

Fonte: Elaborado pelo autor com base em STMicroelectronics (2026b) e STMicroelectronics (2026c).

Esse resultado estabelece uma referência para o dimensionamento computacional do sistema, permitindo comparar diferentes microcontroladores em função de suas frequências de operação e da quantidade de ciclos disponíveis em cada intervalo de simulação. A relação entre os ciclos consumidos pelo modelo e aqueles disponíveis permite estimar a fração do tempo de processamento destinada à simulação e, assim, avaliar preliminarmente sua viabilidade em tempo real.

Dentre as opções avaliadas, o microcontrolador selecionado para o projeto foi o STM32H723ZGT6. Essa escolha justifica-se não apenas por ele proporcionar uma boa margem de ocupação computacional, garantindo folga de processamento para as demais tarefas, mas também pela disponibilidade de uma placa de desenvolvimento comercial que já possui os pinos de conexão USB soldados à porta, o que facilita significativamente o processo de montagem, prototipagem e comunicação para configuração e monitoramento.

É importante destacar, contudo, que essa análise considera inicialmente apenas o processamento do modelo do conversor Buck. Na implementação do RTDS, também devem ser considerados os recursos computacionais necessários para as demais tarefas, como a atualização das saídas pelo conversor digital-analógico (D/A) e a comunicação USB para configuração e monitoramento. Dessa forma, os 31 ciclos estimados não representam o custo computacional total do sistema, sendo necessária uma margem adicional para acomodar essas operações.

O número real de ciclos de clock consumidos pela implementação do simulador será determinado experimentalmente durante os testes na plataforma STM32H723ZGT6, utilizando o contador de ciclos de execução DWT (*Data Watchpoint and Trace*), disponível no núcleo ARM Cortex-M7. Esse recurso permite medir o tempo de processamento em ciclos de clock, possibilitando avaliar o custo computacional efetivo da implementação e verificar se as tarefas são executadas dentro do período de simulação estabelecido. Considerando a capacidade de processamento do microcontrolador selecionado, espera-se que haja margem suficiente para a execução das demais tarefas sem comprometer a operação em tempo real.

4.7.4 Arquitetura e Projeto do Firmware do Simulador

Esta Subseção apresenta a arquitetura e o projeto do firmware responsável pela execução do simulador do conversor Buck. O firmware utiliza uma máquina de estados finitos para controlar a configuração e a execução do modelo matemático discretizado, além de um protocolo binário USB para receber parâmetros e transmitir os resultados da simulação. Essa estrutura busca reduzir a sobrecarga de processamento e atender aos requisitos temporais da simulação em alta frequência.

Para organizar a comunicação entre o microcontrolador e o computador, foram definidos cabeçalhos numéricos que identificam cada tipo de mensagem. Todos os pacotes começam com um cabeçalho de 16 bits, seguido dos dados específicos da mensagem, quando aplicável. A Tabela 2 apresenta os identificadores e as direções de transmissão.

Tabela 2 – Dicionário de cabeçalhos do protocolo binário USB.

Cabeçalho (hex.)	Identificador	Direção
`0x0001`	`CONFIG`	Computador → Microcontrolador
`0x0002`	`CONFIG_SUCCESS`	Microcontrolador → Computador
`0x0003`	`CONFIG_FAIL`	Microcontrolador → Computador
`0x0004`	`DATA`	Microcontrolador → Computador
`0x0005`	`SIMULATION_END`	Microcontrolador → Computador

Fonte: Elaborado pelo autor.

O protocolo utiliza a ordem de bytes *little-endian*, na qual o byte menos significativo de cada campo numérico com múltiplos bytes é transmitido primeiro. Os campos numéricos são representados pelos tipos `float32`, `uint32` e `uint16`, correspondentes a números de ponto flutuante de 32 bits e inteiros sem sinal de 32 e 16 bits, respectivamente.

A mensagem `CONFIG` possui 46 bytes e contém os limites de conversão dos sinais, os coeficientes do modelo discretizado e os parâmetros da simulação. A distribuição desses campos é apresentada na Tabela 3.

Tabela 3 – Estrutura de campos da mensagem de configuração (`CONFIG`).

Campo	Tipo	Tamanho (bytes)	Offset (bytes)
`header`	`uint16`	2	0
`vO_max`	`float32`	4	2
`iL_max`	`float32`	4	6
`Ad_00`	`float32`	4	10
`Ad_01`	`float32`	4	14
`Bd1_0`	`float32`	4	18
`Ad_10`	`float32`	4	22
`Ad_11`	`float32`	4	26
`Bd1_1`	`float32`	4	30
`fs`	`float32`	4	34
`n`	`uint32`	4	38
`total_steps`	`uint32`	4	42

Fonte: Elaborado pelo autor.

Os coeficientes das matrizes de estado discretizadas (`Ad`) e da matriz de entrada (`Bd1`) são calculados pela aplicação no computador e enviados ao microcontrolador. Para reduzir o custo computacional de cada iteração, o coeficiente `Bd1_1` já é transmitido multiplicado pela tensão de entrada do conversor Buck. Assim, o firmware não precisa realizar essa multiplicação durante a execução do modelo.

As mensagens `CONFIG_SUCCESS`, `CONFIG_FAIL` e `SIMULATION_END` contêm apenas o cabeçalho de 2 bytes. Já a mensagem de telemetria `DATA` possui 14 bytes e transporta o índice do passo de simulação e os valores das variáveis de estado. Sua estrutura é apresentada na Tabela 4.

A frequência de atualização do modelo é determinada pela frequência de chaveamento do conversor (f_s) e pelo número de passos de discretização por período (n), conforme a Equação 84.

$$f_{\text{sim}} = f_s \cdot n \tag{84}$$

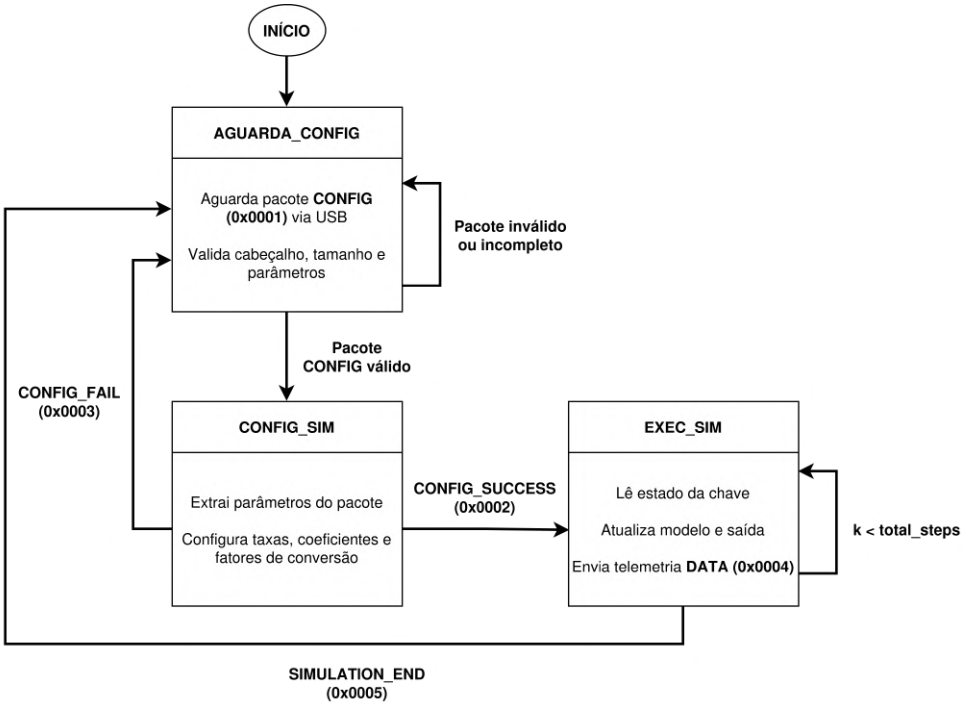
Tabela 4 – Estrutura de campos da mensagem de telemetria (`DATA`).

Campo	Tipo	Tamanho (bytes)	Offset (bytes)
`header`	`uint16`	2	0
`k`	`uint32`	4	2
`iL`	`float32`	4	6
`vC_val`	`float32`	4	10

Fonte: Elaborado pelo autor.

A atualização das saídas analógicas e a transmissão da telemetria ocorrem à frequência f_s. Ambas as frequências são configuradas por meio dos temporizadores do microcontrolador.

Figura 40 – Diagrama de transição de estados do firmware do simulador do conversor Buck.



Fonte: Elaborado pelo autor.

Conforme ilustrado na Figura 40, o firmware possui três estados: `AGUARDA_CONFIG`, `CONFIG_SIM` e `EXEC_SIM`. Cada estado corresponde a uma etapa do processo, desde o recebimento dos parâmetros até o término da simulação.

O estado `AGUARDA_CONFIG` é o estado inicial do firmware e o único em que são aceitas mensagens de configuração pela USB. O microcontrolador aguarda o recebimento completo de um pacote `CONFIG` de 46 bytes e verifica sua estrutura e seus parâmetros. Se o pacote for válido, a máquina de estados avança para `CONFIG_SIM`; caso contrário, permanece no estado atual, aguardando uma nova mensagem válida.

No estado `CONFIG_SIM`, o firmware extrai os parâmetros recebidos, configura os temporizadores e define as escalas de conversão entre as variáveis do modelo e as tensões aplicadas aos pinos físicos. Se todas as etapas forem concluídas com sucesso, envia a mensagem `CONFIG_SUCCESS` e inicia a simulação, transitando para `EXEC_SIM`. Caso ocorra uma falha de configuração ou algum parâmetro viole os limites estabelecidos, envia `CONFIG_FAIL` e retorna a `AGUARDA_CONFIG`.

No estado `EXEC_SIM`, o firmware atualiza as variáveis de estado do circuito a cada passo, de acordo com as equações do modelo discretizado. Nesse processo, lê o estado da chave, calcula os novos valores das variáveis e atualiza as saídas analógicas. A transmissão dos pacotes `DATA` ocorre paralelamente à execução do modelo, sem bloquear o processamento no caminho crítico. Quando o índice de simulação atinge o valor definido por `total_steps`, o firmware envia `SIMULATION_END` e retorna a `AGUARDA_CONFIG`.

A Tabela 5 resume as condições, as ações e os estados de destino associados às transições da máquina de estados.

Tabela 5 – Resumo das transições de estado do firmware do simulador.

Estado de Origem	Condição	Ação	Estado de Destino
AGUARDA_CONFIG	Pacote CONFIG válido	Iniciar configuração	CONFIG_SIM
AGUARDA_CONFIG	Pacote inválido ou incompleto	Aguardar recepção	AGUARDA_CONFIG
CONFIG_SIM	Configuração concluída	Enviar CONFIG_SUCCESS	EXEC_SIM
CONFIG_SIM	Falha na configuração	Enviar CONFIG_FAIL	AGUARDA_CONFIG
EXEC_SIM	Índice inferior ao total de passos	Executar próximo passo	EXEC_SIM
EXEC_SIM	Total de passos atingido	Enviar SIMULATION_END	AGUARDA_CONFIG

Fonte: Elaborado pelo autor.

4.7.5 Implementação do firmware do simulador

O firmware do simulador foi desenvolvido em linguagem C, utilizando a camada de abstração de hardware (HAL) disponibilizada pela STMicroelectronics. Sua execução é organizada em duas partes: as rotinas de interrupção de *hardware*, responsáveis pelas operações que exigem temporização precisa, e o laço infinito da função principal (`main`), no qual são executadas as tarefas de gerenciamento do sistema e comunicação USB. Essa organização permite separar as operações com requisitos temporais rigorosos das atividades que não exigem execução em instantes específicos, conforme apresentado no Código 8.

Código 8 – Laço principal de execução do firmware com abordagem sequencial (*bare-metal*).

```
1 while (1)
2 {
3     SM_Process();
4     USB_Comm_ProcessTx();
5 }
```

Fonte: Elaborado pelo autor.

A função `SM_Process()` é responsável pelo gerenciamento da máquina de estados do simulador, enquanto `USB_Comm_ProcessTx()` verifica a existência de dados pendentes e conduz o processo de transmissão pela interface USB. Ambas são executadas continuamente, permitindo que as tarefas de gerenciamento e comunicação sejam realizadas sem interferir diretamente na temporização das operações executadas pelas interrupções.

A recepção de configurações ocorre por meio da interface USB CDC, que aciona a função de retorno `USB_Comm_ReceiveCallback()` quando uma mensagem é recebida do computador. Essa função verifica se o tamanho dos dados corresponde ao esperado para a estrutura `Protocol_ConfigPacket_t` e se o cabeçalho identifica uma mensagem do tipo `HEAD_CONFIG`. Atendidas essas condições, o conteúdo recebido é copiado para uma estrutura estática interna por meio de `memcpy`, e a variável `new_config_received` é ativada para sinalizar a disponibilidade de uma nova configuração, conforme apresentado no Código 9.

Código 9 – Rotina de recepção de dados USB por meio de um *callback* de interrupção.

```
1  void USB_Comm_ReceiveCallback(uint8_t *Buf, uint32_t *Len) {
2      if (*Len == sizeof(Protocol_ConfigPacket_t)) {
3          Protocol_ConfigPacket_t *pkt = (Protocol_ConfigPacket_t *)Buf;
4          if (pkt->header == HEAD_CONFIG) {
5              memcpy(&rx_config, pkt, sizeof(Protocol_ConfigPacket_t));
6              new_config_received = true;
7          }
8      }
9  }
```

Fonte: Elaborado pelo autor.

Essa abordagem separa a recepção dos dados de seu processamento. Assim, a rotina de *callback* limita-se à identificação e ao armazenamento da mensagem, enquanto o tratamento dos parâmetros é realizado posteriormente pela máquina de estados, durante a execução do laço principal.

O gerenciamento do simulador é realizado pela função `SM_Process()`, que controla as transições entre os estados de espera por configuração (`AGUARDA_CONFIG`), configuração do *hardware* (`CONFIG_SIM`) e execução da simulação (`EXEC_SIM`). A implementação dessa função é apresentada no Código 10.

Código 10 – Função de processamento da máquina de estados do simulador.

```
1  void SM_Process(void) {
2      switch(current_state) {
3          case AGUARDA_CONFIG:
4              if (USB_Comm_GetNewConfig(&cfg)) {
5                  __stop_sim_hardware();
6                  current_state = CONFIG_SIM;
7              }
8          break;
9
10         case CONFIG_SIM:
11             if (cfg.fs > 0.0f && cfg.n > 0 && cfg.total_steps > 0) {
12                 __config_hardware_freqs();
13
14                 k = 0;
15                 iL_val = 0.0f;
16                 vC_val = 0.0f;
17
18                 __send_header_only(HEAD_CONFIG_SUCCESS);
19                 current_state = EXEC_SIM;
20                 __start_sim_hardware();
21             } else {
22                 __send_header_only(HEAD_CONFIG_FAIL);
23                 current_state = AGUARDA_CONFIG;
24             }
25             break;
26
27         case EXEC_SIM:
28             if (k >= cfg.total_steps) {
29                 __stop_sim_hardware();
30                 __send_header_only(HEAD_SIM_END);
31                 current_state = AGUARDA_CONFIG;
32             }
33             break;
34
35         default:
36             current_state = AGUARDA_CONFIG;
37             break;
38     }
39 }
```

Fonte: Elaborado pelo autor.

Inicialmente, o sistema permanece no estado AGUARDA_CONFIG, no qual verifica a disponibilidade de uma nova configuração por meio de USB_Comm_GetNewConfig(). Quando uma configuração é identificada, seus parâmetros são armazenados e a função __stop_sim_hardware() é acionada para garantir que os temporizadores e a saída do DAC permaneçam desabilitados durante a configuração. Em seguida, a máquina de estados avança para CONFIG_SIM.

Nesse estado, os parâmetros recebidos são validados por meio da verificação das frequências de chaveamento, do número de passos computacionais por período e da quantidade total de passos da simulação. Caso os valores sejam válidos, a função __config_hardware_freqs() configura os temporizadores, enquanto as variáveis de estado do modelo e o contador de passos são inicializados. Após essa etapa, o *firmware* envia a confirmação HEAD_CONFIG_SUCCESS, altera o estado para EXEC_SIM e inicia os periféricos responsáveis pela temporização da simulação.

Caso algum dos parâmetros não atenda às condições de validação, é enviada a mensagem HEAD_CONFIG_FAIL, e a máquina retorna ao estado AGUARDA_CONFIG, permanecendo disponível para receber uma nova configuração.

Durante a execução, o estado EXEC_SIM verifica se o contador k atingiu o limite definido em cfg.total_steps. Quando isso ocorre, a função __stop_sim_hardware() interrompe os temporizadores e desabilita a saída do DAC. Em seguida, o *firmware* envia a mensagem HEAD_SIM_END para sinalizar ao computador o encerramento da simulação e retorna ao estado AGUARDA_CONFIG.

A configuração dos temporizadores é realizada pela função __config_hardware_freqs(), apresentada no Código 11.

Código 11 – Função de configuração dinâmica das frequências dos temporizadores no *hardware*.

```
1  static void __config_hardware_freqs(void) {
2      uint32_t pclk1 = HAL_RCC_GetPCLK1Freq(); // Barramento do TIM2 (APB1)
3      uint32_t pclk2 = HAL_RCC_GetPCLK2Freq(); // Barramento do TIM1 (APB2)
4      uint32_t tim2_clock = pclk1 * 2; // TIM2 está no APB1, que tem um multiplicador de 2
5      uint32_t tim1_clock = pclk2 * 2; // TIM1 está no APB2, que tem um multiplicador de 2
6
7      float f_tel = cfg.fs * 2;
8      float f_sim = cfg.fs * (float)cfg.n;
9      uint32_t arr_sim = (uint32_t)(tim1_clock / f_sim) - 1;
10     uint32_t arr_fs  = (uint32_t)(tim2_clock / f_tel) - 1;
11
12     __HAL_TIM_SET_AUTORELOAD(&htim1, arr_sim);
13     __HAL_TIM_SET_AUTORELOAD(&htim2, arr_fs);
14     __HAL_TIM_SET_COUNTER(&htim1, 0);
15     __HAL_TIM_SET_COUNTER(&htim2, 0);
16 }
```

Fonte: Elaborado pelo autor.

A função utiliza a frequência do relógio do sistema, obtida por meio de SystemCoreClock, juntamente com os parâmetros fs e n recebidos na configuração. A partir desses valores, determina a frequência de execução do modelo e envio da telemetria.

Com as frequências definidas, são calculados os valores de recarga automática dos temporizadores (ARR, *Auto-Reload Register*). Para o TIM1, o cálculo considera a frequência de simulação, enquanto o TIM2 utiliza a expressão implementada no código, baseada em fs. Esses valores são aplicados aos respectivos periféricos por meio da macro __HAL_TIM_SET_AUTORELOAD(), enquanto os contadores são zerados para estabelecer uma referência inicial de temporização.

O TIM1 determina os instantes de execução dos passos computacionais do modelo, operando em f_{sim}. Já o TIM2 controla a temporização das operações relacionadas ao envio periódico da telemetria. A utilização desses periféricos permite estabelecer os instantes de execução diretamente pelo *hardware*, reduzindo a dependência de mecanismos de temporização por software e favorecendo a previsibilidade do sistema.

A execução do modelo matemático do conversor Buck é realizada pela rotina de interrupção `SM_Timer_MathStep_ISR()`, acionada pelo `TIM1` a cada período de simulação. Essa rotina calcula os novos valores das variáveis de estado do modelo discreto e atualiza a saída analógica do microcontrolador, conforme apresentado no Código 12.

Código 12 – Rotina de execução do passo de simulação computacional do conversor Buck.

```
1  void SM_Timer_MathStep_ISR(void) {
2      if (current_state != EXEC_SIM || k >= cfg.total_steps) return;
3
4      float new_iL = (cfg.Ad_00 * iL_val) + (cfg.Ad_01 * vC_val);
5      float new_vC = (cfg.Ad_10 * iL_val) + (cfg.Ad_11 * vC_val);
6
7      if ((GPIOA->IDR & GPIO_PIN_0) != 0) {
8          new_iL += cfg.Bd1_0;
9          new_vC += cfg.Bd1_1;
10     }
11
12     iL_val = new_iL;
13     vC_val = new_vC;
14
15     float dac_float = (vC_val / cfg.vO_max) * 4095.0f;
16
17     if (dac_float < 0.0f) dac_float = 0.0f;
18     else if (dac_float > 4095.0f) dac_float = 4095.0f;
19
20     HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_1, DAC_ALIGN_12B_R, (uint32_t)dac_float);
21     k++;
22 }
```

Fonte: Elaborado pelo autor.

Inicialmente, são verificadas duas condições: se o simulador permanece no estado `EXEC_SIM` e se o contador `k` ainda não atingiu o limite estabelecido em `cfg.total_steps`. Caso alguma dessas condições não seja atendida, a rotina retorna sem executar um novo passo computacional.

Atendidas as condições, os novos valores da corrente no indutor e da tensão no capacitor são calculados a partir das equações de estado discretizadas. Para isso, são utilizados os coeficientes `Ad_00`, `Ad_01`, `Ad_10` e `Ad_11`, que representam os elementos da matriz de estados A_d.

A contribuição da entrada do modelo é determinada pela leitura do estado lógico do pino PA0, realizada diretamente pelo registrador de entrada da porta GPIO (`GPIOA->IDR`). Quando o pino está em nível lógico alto, os coeficientes `Bd1_0` e `Bd1_1` são adicionados às respectivas equações, representando a contribuição da entrada previamente multiplicada pela tensão de alimentação. Dessa forma, a evolução das variáveis de estado considera diretamente o estado físico da chave do conversor.

Após os cálculos, os novos valores são armazenados em `iL_val` e `vC_val`. A tensão no capacitor é então convertida para um código digital compatível com a resolução de 12 bits do DAC, considerando a tensão máxima definida em `cfg.vO_max`. Essa conversão é realizada pela relação entre a tensão simulada e a faixa de tensão representada na saída analógica, multiplicada pelo valor máximo de 4095.

O código resultante é limitado ao intervalo de 0 a 4095, garantindo que permaneça dentro da faixa de operação do periférico. Assim, valores negativos são convertidos em zero, enquanto valores superiores ao limite máximo são saturados em 4095. Por fim, o valor é convertido para inteiro e enviado ao DAC por meio de `HAL_DAC_SetValue()`, utilizando alinhamento de 12 bits à direita (`DAC_ALIGN_12B_R`). Ao término da rotina, o contador `k` é incrementado para registrar a execução do passo computacional.

A preparação dos pacotes de telemetria é realizada pela rotina de interrupção `SM_Timer_Telemetry_ISR()`, acionada periodicamente pelo `TIM2`. Inicialmente, a rotina verifica se o simulador está no estado `EXEC_SIM`. Caso esteja, os valores atuais do contador de passos `k`, da corrente no indutor `iL_val` e da

tensão no capacitor `vC_val` são utilizados para preencher uma estrutura do tipo `Protocol_DataPacket_t`, identificada pelo cabeçalho `HEAD_DATA`.

Após o preenchimento, a estrutura é encaminhada à função `USB_Comm_QueueTxData()`, que insere o pacote no *buffer* circular para posterior transmissão pelo laço principal. Assim, a rotina de interrupção limita-se à preparação e ao enfileiramento dos dados, sem aguardar a conclusão da comunicação USB, conforme apresentado no Código 13.

Código 13 – Rotina de interrupção responsável pelo enfileiramento dos dados de telemetria.

```
1  void SM_Timer_Telemetry_ISR(void) {
2      if (current_state != EXEC_SIM) return;
3
4      Protocol_DataPacket_t data;
5      data.header = HEAD_DATA;
6      data.k = k;
7      data.iL = iL_val;
8      data.vC_val = vC_val;
9      USB_Comm_QueueTxData((uint8_t*)&data, sizeof(Protocol_DataPacket_t));
10 }
```

Fonte: Elaborado pelo autor.

Para desacoplar a geração de dados da transmissão USB, foi implementado um *buffer* circular de transmissão com capacidade de 8192 bytes. Essa estrutura funciona como uma fila de tamanho fixo, na qual os dados são inseridos e removidos por meio de índices independentes de escrita e leitura. Ao alcançar o final da região de memória, cada índice retorna ao início, permitindo o reaproveitamento do espaço sem a necessidade de deslocar os dados armazenados. A inserção dos pacotes nessa estrutura é realizada pela função `USB_Comm_QueueTxData()`, apresentada no Código 14.

Código 14 – Função de enfileiramento de dados de transmissão USB com proteção de seção crítica.

```
1  bool USB_Comm_QueueTxData(const uint8_t *data, uint16_t len) {
2      if (len >= TX_RING_BUFFER_SIZE || len == 0) return false;
3
4      uint32_t primask = __get_PRIMASK();
5      __disable_irq();
6
7      uint16_t free_space = (tx_tail <= tx_head) ?
8                            (TX_RING_BUFFER_SIZE - tx_head + tx_tail - 1) :
9                            (tx_tail - tx_head - 1);
10
11     if (len > free_space) {
12         uint16_t bytes_needed = len - free_space;
13         uint16_t packets_to_drop = (bytes_needed + len - 1) / len;
14         uint16_t bytes_to_discard = packets_to_drop * len;
15         tx_tail = (tx_tail + bytes_to_discard) % TX_RING_BUFFER_SIZE;
16     }
17
18     for (uint16_t i = 0; i < len; i++) {
19         tx_buffer[tx_head] = data[i];
20         tx_head = (tx_head + 1) % TX_RING_BUFFER_SIZE;
21     }
22
23     __set_PRIMASK(primask);
24     return true;
25 }
```

Fonte: Elaborado pelo autor.

Como a função `USB_Comm_QueueTxData()` é chamada pelas interrupções dos temporizadores, enquanto o consumo da fila ocorre no laço principal, seu acesso envolve diferentes contextos de execução. Essa concorrência exige proteção para evitar inconsistências durante a atualização dos índices e a manipulação dos

dados armazenados. Para isso, são utilizadas seções críticas que desabilitam temporariamente as interrupções globais da CPU.

A proteção é realizada por meio das funções `__get_PRIMASK()`, `__disable_irq()` e `__set_PRIMASK()`. Inicialmente, o estado atual das interrupções é armazenado, seguido de sua desabilitação durante as operações críticas. Ao final, o estado anterior é restaurado, evitando alterações indevidas no estado global das interrupções. Como essas seções críticas envolvem apenas operações sobre os índices e os dados da fila, seu tempo de execução é limitado, reduzindo os impactos sobre o atendimento das interrupções.

A função calcula inicialmente o espaço disponível na fila a partir dos índices de escrita (`tx_head`) e leitura (`tx_tail`). Quando a capacidade restante não é suficiente para armazenar um novo pacote, são descartados dados previamente enfileirados em quantidade suficiente para liberar o espaço necessário. Esse descarte ocorre em múltiplos do tamanho do novo pacote, preservando o alinhamento das mensagens armazenadas. Após essa verificação, os dados são inseridos sequencialmente, com atualização do índice de escrita a cada byte.

O envio dos dados armazenados é realizado pela função `USB_Comm_ProcessTx()`, executada continuamente no laço principal e apresentada no Código 15. Antes de iniciar uma nova transmissão, a função verifica a variável `is_transmitting` para identificar se já existe uma operação em andamento. Caso contrário, verifica se há dados disponíveis na fila e, quando necessário, prepara um bloco de até 64 bytes para transmissão.

Código 15 – Processamento assíncrono e transmissão dos dados enfileirados pela interface USB CDC.

```
1  void USB_Comm_ProcessTx(void) {
2      if (is_transmitting) return;
3
4      uint16_t bytes_to_send = 0;
5      uint32_t primask = __get_PRIMASK();
6      __disable_irq();
7
8      if (tx_head == tx_tail) {
9          __set_PRIMASK(primask);
10         return;
11     }
12
13     uint16_t temp_tail = tx_tail;
14     while (temp_tail != tx_head && bytes_to_send < 64) {
15         temp_buf[bytes_to_send++] = tx_buffer[temp_tail];
16         temp_tail = (temp_tail + 1) % TX_RING_BUFFER_SIZE;
17     }
18
19     is_transmitting = true;
20     __set_PRIMASK(primask);
21
22     if (CDC_Transmit_HS(temp_buf, bytes_to_send) == USBD_OK) {
23         uint32_t primask_tx = __get_PRIMASK();
24         __disable_irq();
25         tx_tail = temp_tail;
26         __set_PRIMASK(primask_tx);
27     } else {
28         is_transmitting = false;
29     }
30 }
```

Fonte: Elaborado pelo autor.

A preparação dos dados ocorre por meio da cópia dos bytes da fila para um *buffer* temporário estático. Para isso, utiliza-se um índice auxiliar, de modo que o índice de leitura original permaneça inalterado até que a solicitação de transmissão seja aceita. Após a cópia, a variável `is_transmitting` é ativada e o estado anterior das interrupções é restaurado. A função `CDC_Transmit_HS()` é então utilizada para solicitar o envio dos dados pela interface USB CDC.

Quando a função retorna `USBD_OK`, o índice de leitura da fila é atualizado para indicar que os dados foram aceitos para transmissão. Essa atualização é realizada em uma nova seção crítica, na qual o estado atual

das interrupções é novamente consultado e preservado. Caso a solicitação não seja aceita, a variável `is_transmitting` é desativada, permitindo uma nova tentativa nas próximas iterações do laço principal.

Dessa maneira, o enfileiramento e o envio dos dados são realizados de forma independente, evitando que as rotinas de interrupção precisem aguardar a conclusão das operações de comunicação USB.

4.7.6 Testes do firmware do simulador

Inicialmente, foram realizados testes isolados do firmware utilizando o conversor Buck originalmente projetado, com frequência de chaveamento de 50 kHz. Os testes foram conduzidos em malha aberta, sem a integração com o controlador digital. Para representar o estado de comutação da chave, foi utilizado um sinal PWM gerado externamente por outro microcontrolador, com frequência de 50 kHz e razão cíclica de 50%.

Durante esses testes, verificou-se que o firmware não conseguia executar adequadamente a simulação na frequência estabelecida. Para investigar a causa dessa limitação, foi realizada uma análise do tempo de processamento de cada passo da simulação, utilizando o contador de ciclos de clock do núcleo Cortex-M7, o *Data Watchpoint and Trace* (DWT). Para isso, foram adicionadas instruções de leitura do registrador `CYCCNT` antes e depois das operações de atualização das variáveis de estado e escrita do resultado no DAC. A diferença entre essas leituras permite determinar a quantidade de ciclos de clock consumidos pelo processamento de cada passo, conforme apresentado no Código 16.

Código 16 – Rotina de execução do passo de simulação modificada para medição de ciclos de clock.

```
1  void SM_Timer_MathStep_ISR(void) {
2      if (current_state != EXEC_SIM || k >= cfg.total_steps) return;
3
4      uint32_t start_cycles = DWT->CYCCNT; // TESTE
5
6      ... Código interno se manteve igual
7
8      // TESTE
9      uint32_t end_cycles = DWT->CYCCNT;
10     isr_cycles_current = end_cycles - start_cycles;
11     isr_cycles_total += isr_cycles_current;
12     isr_exec_count++;
13
14     if (isr_cycles_current > isr_cycles_max) {
15         isr_cycles_max = isr_cycles_current;
16     }
17 }
```

Fonte: Elaborado pelo autor.

As medições resultaram em uma média de 476,39 ciclos de clock por passo de simulação, com um pior caso de 592 ciclos. Esses valores foram comparados ao orçamento de processamento disponível na configuração original. Considerando a frequência de clock de 550 MHz do microcontrolador, a frequência de chaveamento de 50 kHz e o fator de discretização $N = 100$, a frequência de execução do modelo discreto é de 5 MHz. Consequentemente, o intervalo entre passos corresponde a apenas 110 ciclos de clock, conforme calculado na Equação 85.

$$N_{\text{ciclos}} = \frac{f_{\text{clock}}}{f_{\text{sim}}} = \frac{550\,\text{MHz}}{50\,\text{kHz} \cdot 100} = 110 \tag{85}$$

A comparação entre os ciclos disponíveis e os efetivamente consumidos demonstrou que o processamento de cada passo excedia o intervalo de tempo estabelecido, inclusive em sua condição média. Portanto, a configuração original não era compatível com a capacidade de processamento disponível no microcontrolador, impossibilitando a execução da simulação em tempo real nessas condições.

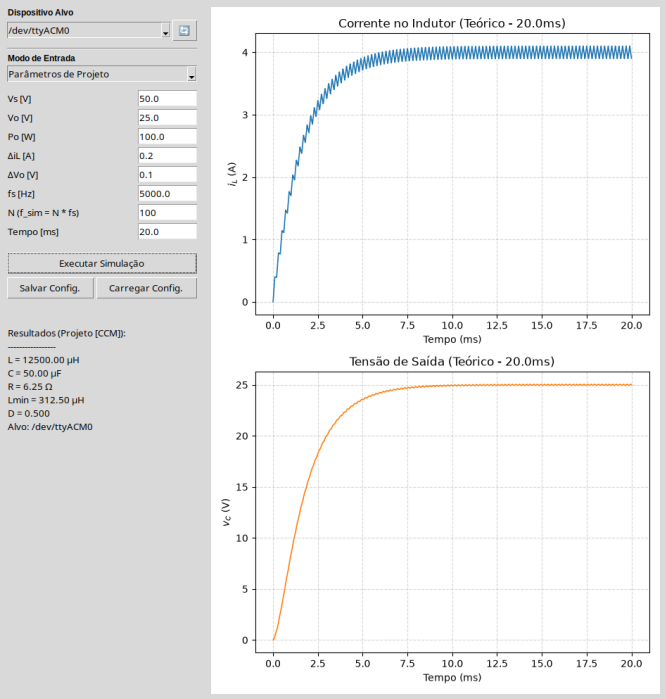
Diante dessa limitação, optou-se por utilizar o conversor Buck alternativo apresentado no Apêndice B, projetado para operar com frequência de chaveamento de 5 kHz. Mantendo-se o fator de discretização $N = 100$,

a frequência de execução do modelo discreto passa a ser de 500 kHz, proporcionando um orçamento de 1100 ciclos de clock por passo. Esse intervalo é superior ao pior caso observado nas medições, permitindo a execução do modelo dentro do período estabelecido.

Para os testes funcionais com o conversor alternativo, foi utilizado novamente um segundo microcontrolador para gerar o sinal PWM de entrada, agora com frequência de 5 kHz e razão cíclica de 50%. Esse sinal foi conectado à entrada digital do simulador responsável pela identificação do estado de comutação da chave do conversor.

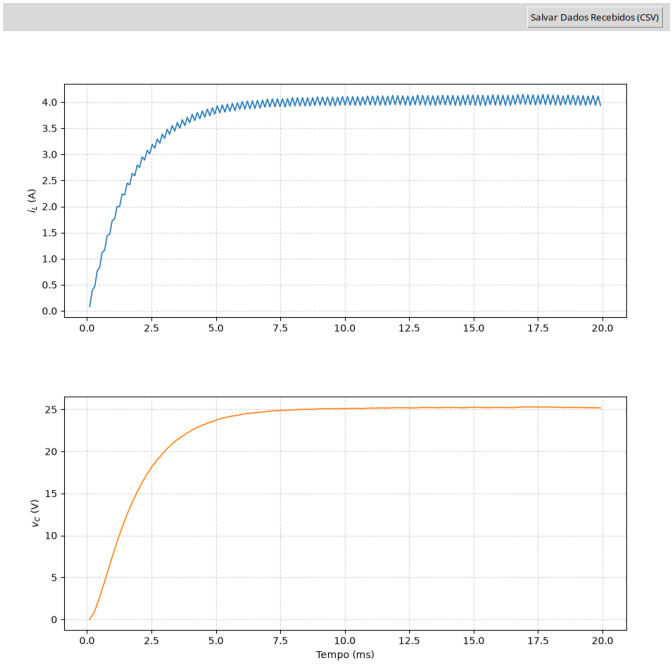
A configuração dos parâmetros da simulação foi realizada por meio da interface gráfica, conforme ilustrado na Figura 41. Durante a execução, o firmware realizou a atualização periódica das variáveis de estado do modelo discreto e a conversão da tensão calculada para a saída analógica por meio do DAC. Os sinais obtidos para a corrente no indutor e a tensão no capacitor são apresentados na Figura 42.

Figura 41 – Interface de configuração com os parâmetros da simulação do conversor Buck.



Fonte: Elaborado pelo autor.

Figura 42 – Sinais de saída da corrente no indutor e tensão do capacitor obtidos na simulação.



Fonte: Elaborado pelo autor.

Os resultados permitiram verificar o funcionamento das etapas de configuração, execução do modelo discreto, leitura do sinal externo de comutação e atualização das saídas analógicas. Dessa forma, foi possível validar o funcionamento isolado do firmware do simulador em malha aberta, utilizando a configuração alternativa do conversor Buck, antes de sua integração com o controlador digital.

5 RESULTADOS

Aqui preciso mostrar os testes e resultados em malha fechada

6 CONCLUSÃO

o que eu faria diferente se eu pudesse voltar no tempo com o conhecimento que tenho agora:

- usaria uma frequencia de chaveamento menor logo do inicio do trabalho, assim evitando o retrabalho de ter que fazer o projeto alternativo

- teria usado o QSPICE logo no inicio no lugar do LTSpice, achei mais facíl de usar, mais rapido de simular e com mais funções

- não teria dito que faria uma interface grafica -> tempo apertado, assim eu teria feito só se eu tivesse o tempo livre

o que gostei:

- tive que revisar sobre a parte de controle

- tive que revisar processamento digital

- aprendi mais sobre discretização e simulação de plantas físicas

- aprendi mais sobre a importancia de HIL

- aprendi o por que é FPGA são usadas no lugar de micro em alguns produtos

- aprendi mais sobre como selecionar o microcontrolador para um projeto

o que poderia ser expandido no trabalho: - calcuclar que (frequencia * n) seria o maximo poder no harware atual

- usar DMA para ler o pino do pwm e para atualizar a saida, isso permitiria deixar a sim e a atualização da saida em freq diferentes, alem de o dma poder ajudar a diminuir o numero de ciclos usados por passo da simulação. Tentei utilizar mas não funcionou DMA, aparentemente era alguma configuração da MPU que fiz errado mas por o tempo estar apertado não quis arriscar de ir atrás e acabar perdendo muito tempo nisso

- opção para simular outros conversores chaveados: boost, buck-boost

- opção de simular conversores projetados para operar em DCM

- implementar simulador em FPGA para permitir simular frequencias maiores de chaveamento

- procurar como simulações SPICE funcionam por debaixo dos panos e se poderia ser adaptado como forma de configurar uma simulação em tempo real

Parte final do texto, na qual se apresentam as conclusões do trabalho acadêmico, usualmente denominada Considerações Finais. Pode ser usada outra denominação similar que indique a conclusão do trabalho.

REFERÊNCIAS

ANGÉLICO, B. A.; NEVES, G. P. d. **Controle Digital Aplicado**. São Paulo, Brasil: Blucher, 2023.

BARBI, I. **Modelagem de Conversores CC-CC Empregando Modelo Médio em Espaço de Estado**. [*S.l.*: *s.n.*], 2015. ISBN 9788590104698.

Brasil. Conselho Nacional de Desenvolvimento Científico e Tecnológico. **Portaria CNPq nº 2.664, de 6 de março de 2026**. Brasília, DF, 2026. Institui a Política de Integridade na Atividade Científica do CNPq. Disponível em: http://memoria2.cnpq.br/web/guest/view/-/journal_content/56_INSTANCE_0oED/10157/23142775.

BROOKS, L. Siemens Digital Industries Software **Consistent test reuse across MIL → SIL → HIL in a model-driven development workflow**. [*S.l.*], 2020. Disponível em: https://www.siemens.com/autosar.

CHA, S. T. *et al.* Real-time hardware-in-the-loop (hil) testing for power electronics controllers. *In*: 2012 ASIA-PACIFIC POWER AND ENERGY ENGINEERING CONFERENCE. 2012. **Anais [...]** [*S.l.*: *s.n.*], 2012. p. 1–6.

FRANKLIN, G. F. **Feedback Control of Dynamic Systems**. [*S.l.*]: Pearson, 2018. ISBN 9780134685717.

GAVIRIA-CANO, A. *et al.* Cost-effective and versatile hardware-in-the-loop system for dc/dc converter emulation in education and research. **HardwareX**, Elsevier v. 24„, Dec 2025. ISSN 2468-0672. Disponível em: https://doi.org/10.1016/j.ohx.2025.e00701.

GUO, X. *et al.* Hardware in the loop real-time simulation for the associated discrete circuit modeling optimization method of power converters. **Energies**, v. 11, n. 11„ 2018. ISSN 1996-1073. Disponível em: https://www.mdpi.com/1996-1073/11/11/3237.

HART, D. W. **Power Electronics**. New York, NY: McGraw-Hill Professional, 2010.

ISERMANN, R.; SCHAFFNIT, J.; SINSEL, S. Hardware-in-the-loop simulation for the design and testing of engine-control systems. **Control Engineering Practice**, v. 7, n. 5, p. 643–653, 1999. ISSN 0967-0661. Disponível em: https://www.sciencedirect.com/science/article/pii/S0967066198002056.

KIREI, B. S. *et al.* Hardware emulation of step-down converter power stages for digital control design. **Electronics**, v. 12, n. 6„ 2023. ISSN 2079-9292. Disponível em: https://www.mdpi.com/2079-9292/12/6/1328.

KOCHEM, M.; KEMPER, H.; ISTOC, S. B. Data and workflow management for virtual development of software-defined vehicles. **ATZ Worldw.**, Springer Science and Business Media LLC v. 125, n. 7-8, p. 74–79, jul. 2023.

LAMBERT, H. *et al.* X-in-the-loop methodology for proton exchange membrane fuel cell systems design: Review of advances and challenges. **Energies**, v. 18, n. 14„ 2025. ISSN 1996-1073. Disponível em: https://www.mdpi.com/1996-1073/18/14/3774.

MEHL, E. L. M. **Aplicação do Modelo Linear de Vorpérian ao Conversor tipo Buck**, . 2026. Material didático, Universidade Federal do Paraná (UFPR). Acessado em: 5 abr. 2026. Disponível em: https://www.eletrica.ufpr.br/mehl/downloads/modelamento-buck.pdf.

Microsoft. **Visual Studio Code**, . 2026. Acesso em: 15 maio 2026. Disponível em: https://code.visualstudio.com/.

MIDDLEBROOK, R. D.; CUK, S. A general unified approach to modelling switching-converter power stages. *In*: 1976 IEEE POWER ELECTRONICS SPECIALISTS CONFERENCE. 1976. **Anais [...]** [*S.l.*: *s.n.*], 1976. p. 18–34.

MIHALIč, F.; TRUNTIč, M.; HREN, A. Hardware-in-the-loop simulations: A historical overview of engineering challenges. **Electronics**, v. 11, n. 15„ 2022. ISSN 2079-9292. Disponível em: https://www.mdpi.com/2079-9292/11/15/2462.

OGATA, K. **Modern Control Engineering**. 5. ed. Upper Saddle River, NJ: Pearson, 2009. ISBN 978-0136156734.

PESTANA, L.; MIRANDA, B. A rapid review on advanced software-in-the-loop methods for automotive software verification. *In*: ANAIS DO IX SIMPóSIO BRASILEIRO DE TESTES DE SOFTWARE SISTEMáTICO E AUTOMATIZADO. 2024, Porto Alegre, RS, Brasil. **Anais [...]** Porto Alegre, RS, Brasil: SBC, 2024. p. 56–65. ISSN 0000-0000. Disponível em: https://sol.sbc.org.br/index.php/sast/article/view/30216.

PICARD, J.-L. **Star Trek: The Next Generation - Episode: Peak Performance**, . 1989. Television series episode. Directed by Bob Scheerer. Written by David Kemper. Music by Dennis McCarthy. Season 2, Episode 21. Production code: 147. Air date: July 10, 1989.

RASHID, M. H. **Eletrônica de Potência: Dispositivos, Circuitos e Aplicações**. 4. ed. São Paulo: Pearson, 2014. ISBN 978-8543005942.

SAMIEE, A. *et al.* Model-driven-engineering in education. *In*: 2018 18TH INTERNATIONAL CONFERENCE ON MECHATRONICS - MECHATRONIKA (ME). 2018. **Anais [...]** [*S.l.*: *s.n.*], 2018. p. 1–6.

STMicroelectronics. **STM32CubeIDE**, . 2026. Acesso em: 15 maio 2026. Disponível em: https://www.st.com/en/development-tools/stm32cubeide.html.

STMicroelectronics. **STM32F7 Series**, . 2026. https://www.st.com/en/microcontrollers-microprocessors/stm32f7-series.html. Acesso em: 30 set. 2026.

STMicroelectronics. **STM32H7 Series**, . 2026. https://www.st.com/en/microcontrollers-microprocessors/stm32h7-series.html. Acesso em: 30 set. 2026.

SüLI, E.; MAYERS, D. F. **An Introduction to Numerical Analysis**. [*S.l.*]: Cambridge University Press, 2003.

Texas Instruments. Texas Instruments **Single-supply, 2nd-order, Sallen-Key low-pass filter circuit**. [*S.l.*], 2021. Disponível em: https://www.ti.com/lit/an/sboa226/sboa226.pdf.

YANG, A. *et al.* A model-based design and verification framework for virtual ecus in automotive seat control systems. **Electronics**, v. 15, n. 3„ 2026. ISSN 2079-9292. Disponível em: https://www.mdpi.com/2079-9292/15/3/569.

APÊNDICES

APÊNDICE A – Projeto do Conversor Buck

Neste apêndice, são apresentadas as equações e a metodologia unificadas utilizadas no dimensionamento dos componentes dos conversores Buck adotados como estudos de caso e de comparação neste trabalho. O projeto considera a operação dos conversores em Modo de Condução Contínua (CCM) e baseia-se no equacionamento clássico apresentado na literatura de Eletrônica de Potência (Hart, 2010).

Metodologia e Equações Gerais de Projeto

O conversor Buck é um circuito CC-CC abaixador de tensão. Para a operação ideal em regime permanente, a razão cíclica nominal (D) é determinada pela relação entre a tensão de saída desejada (V_O) e a tensão de entrada (V_S):

$$D = \frac{V_O}{V_S} \tag{86}$$

A resistência de carga (R), a corrente média no indutor (I_L), a indutância necessária para uma ondulação máxima de corrente (ΔI_L), a indutância crítica (L_{min}) para garantia do CCM e a capacitância de saída (C) para a ondulação de tensão desejada (ΔV_O) são obtidas pelas expressões gerais:

$$R = \frac{V_O^2}{P_{nominal}}, \quad I_L = \frac{V_O}{R} \tag{87}$$

$$L = \frac{V_O \cdot (1 - D)}{\Delta I_L \cdot F_s}, \quad L_{min} = \frac{(1 - D) \cdot R}{2 \cdot F_s} \tag{88}$$

$$C = \frac{V_O \cdot (1 - D)}{8 \cdot L \cdot \Delta V_O \cdot F_s^2} \tag{89}$$

Conversor Buck Projetado

O conversor foi projetado considerando uma frequência de chaveamento de $F_s = 50$ kHz e uma potência nominal de 15 W.

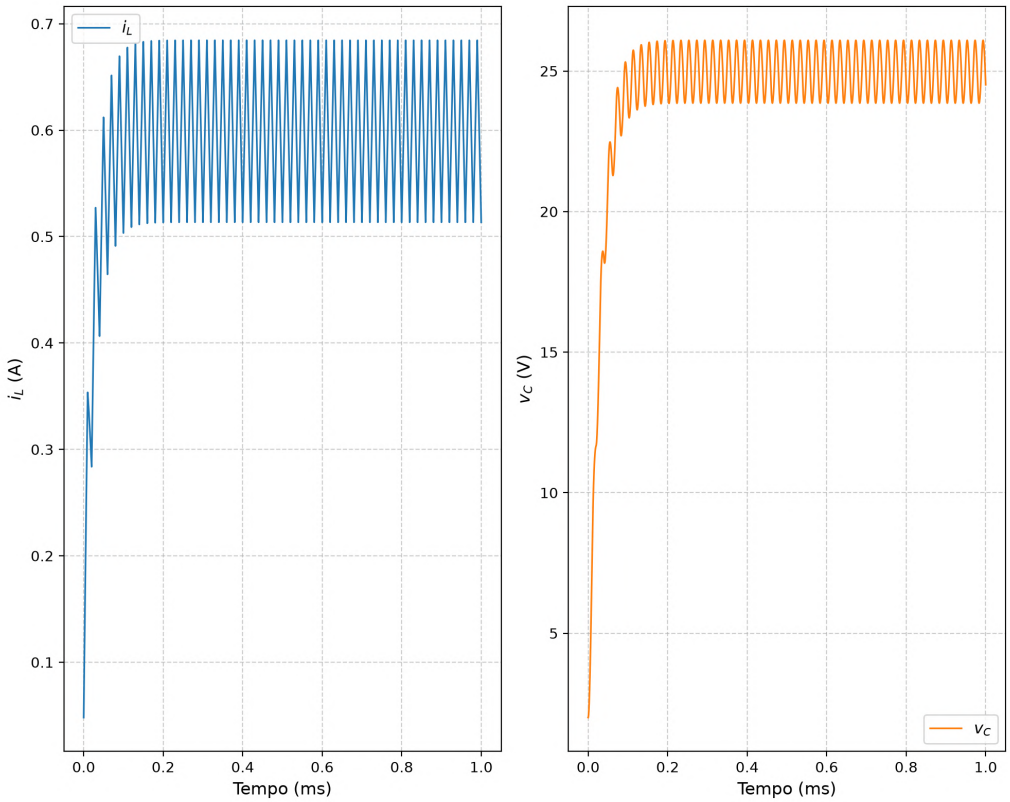
Com os valores comerciais adotados de $L = 1{,}5$ mH e $C = 0{,}18$ μF, as ondulações reais de corrente e tensão recalculadas atendem aos limites especificados de 30% e 10%, respectivamente. Os parâmetros consolidados deste primeiro projeto são apresentados na Tabela 6. A resposta do conversor obtida por meio da simulação no QSPICE, é apresentada na Figura 43.

Tabela 6 – Parâmetros de projeto e especificações do conversor Buck.

Parâmetro	Símbolo	Valor
Tensão de entrada	V_S	50 V
Tensão de saída nominal	V_O	25 V
Frequência de chaveamento	F_s	50 kHz
Potência real	P	15 W
Ondulação máxima de tensão	ΔV_O	10% de V_O
Ondulação máxima de corrente	ΔI_L	30% de I_L
Razão cíclica nominal	D	50%
Indutância	L	$1{,}5$ mH
Resistência de carga	R	$41{,}7\ \Omega$
Capacitância	C	$0{,}18$ μF
Corrente no indutor	I_L	$0{,}5995$ A

Fonte: Elaborado pelo autor.

Figura 43 – Simulação QSPICE da corrente no indutor e tensão no condensador do conversor Buck projetado.



Fonte: Elaborado pelo autor.

APÊNDICE B – Alternativa de Projeto para o Conversor Buck e Controlador

Este apêndice apresenta um projeto alternativo para o conversor Buck, desenvolvido como estratégia de contingência diante de possíveis limitações de processamento do microcontrolador na validação experimental.

Nesta etapa, o conversor, o filtro antialiasing e a malha de controle foram reprojetados seguindo as metodologias descritas no Apêndice A, nas Seções 4.2 e 4.3, respectivamente. A principal alteração consistiu na redução da frequência de chaveamento, visando viabilizar a execução da simulação em tempo real sem comprometer as especificações de estabilidade e o desempenho dinâmico.

Mantiveram-se as tensões de entrada ($V_S = 50$ V) e saída ($V_O = 25$ V) do projeto original. Contudo, a potência nominal foi elevada para 100 W e a frequência de chaveamento reduzida para 5 kHz. Estabeleceram-se limites de ondulação de 5% para a corrente no indutor e de $0{,}4\%$ para a tensão na saída. Com base nesses requisitos, obtiveram-se os novos parâmetros do conversor (Tabela 7). Os resultados iniciais de simulação — comparando o QSPICE com o modelo numérico para $n = 100$ — são exibidos nas Figuras 44 e 45.

Tabela 7 – Parâmetros de projeto e especificações do conversor Buck alternativo.

Parâmetro	Símbolo	Valor
Tensão de entrada	V_S	50 V
Tensão de saída nominal	V_O	25 V
Potência nominal	P_O	100 W
Frequência de chaveamento	F_s	5 kHz
Ondulação máxima de corrente	ΔI_L	5% de I_L
Ondulação máxima de tensão	ΔV_O	$0{,}4\%$ de V_O
Razão cíclica nominal	D	50%
Indutância	L	$12{,}5$ mH
Resistência de carga	R	$6{,}25\ \Omega$
Capacitância	C	$50\ \mu$F
Corrente no indutor	I_L	4 A

Fonte: Elaborado pelo autor.

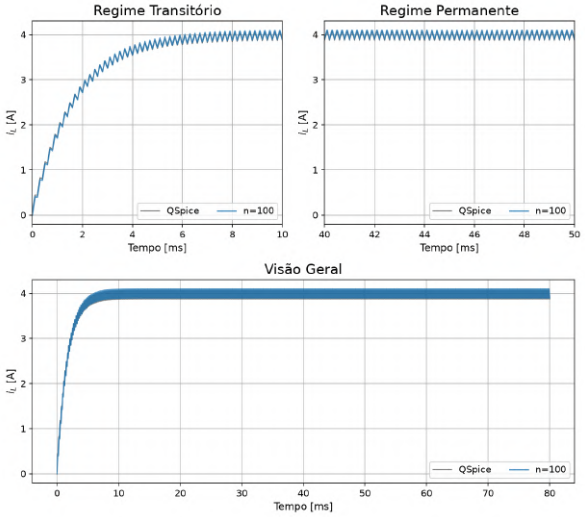


Figura 44 – Corrente no indutor do conversor Buck alternativo.

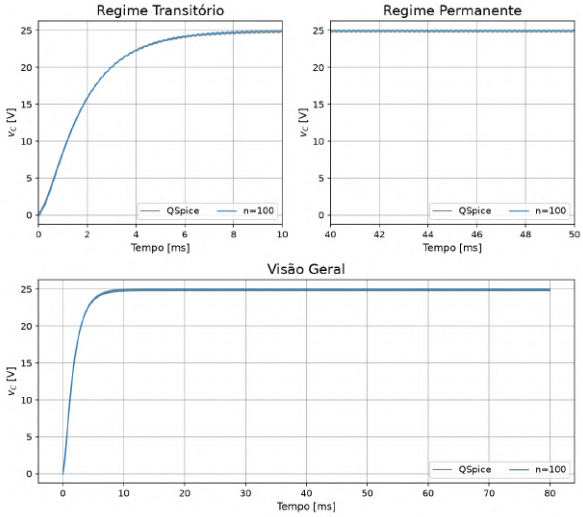


Figura 45 – Tensão no capacitor do conversor Buck alternativo.

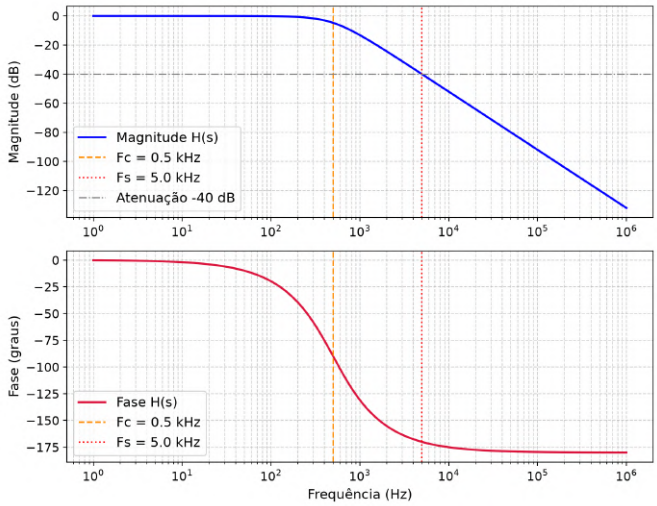
Fonte: Elaborado pelo autor.

Devido à nova frequência de chaveamento ($5\,\text{kHz}$), o filtro antialiasing também precisou ser ajustado. Mantendo o critério de fixar a frequência de corte uma década abaixo da frequência de amostragem, adotou-se o valor de $500\,\text{Hz}$.

Para a implementação do circuito, selecionaram-se os componentes comerciais $R_1 = R_2 = 68\,\text{k}\Omega$, $C_1 = 5{,}6\,\text{nF}$ e $C_2 = 3{,}9\,\text{nF}$. A função de transferência resultante está descrita na Equação 90, e sua respectiva resposta em frequência pode ser observada no diagrama de Bode da Figura 46.

$$H(s) = \frac{9{,}87 \times 10^6}{s^2 + 5441s + 9{,}87 \times 10^6}. \tag{90}$$

Figura 46 – Diagrama de Bode da resposta em frequência do filtro antialiasing do conversor Buck alternativo.



Fonte: Elaborado pelo autor.

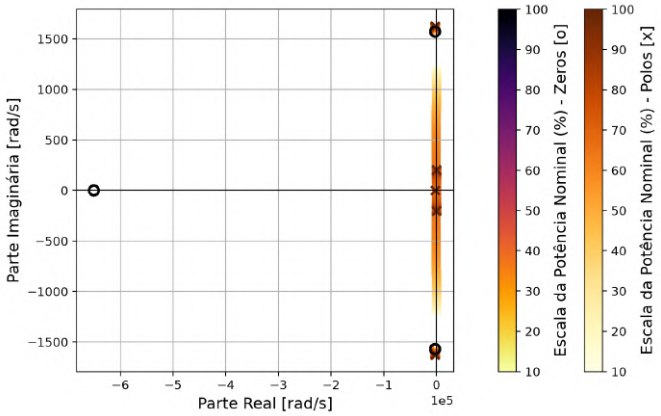
O controlador foi recalculado para a nova planta e para o filtro atualizado. Assim como no projeto original, adotou-se o pior caso de operação: 10% da potência nominal, condição que eleva a resistência de carga. Os requisitos de desempenho exigiram um tempo de acomodação de $20\,\text{ms}$ e erro nulo em regime permanente. A nova função de transferência do controlador contínuo é dada pela Equação 91, enquanto sua versão discreta, obtida pelo método de Tustin, encontra-se na Equação 92.

$$C(s) = 5{,}4195 \times 10^{-6}\,\frac{s + 6{,}5 \times 10^5}{s}. \tag{91}$$

$$u[k] = u[k-1] + 0{,}000358\,e[k] + 0{,}000347\,e[k-1]. \tag{92}$$

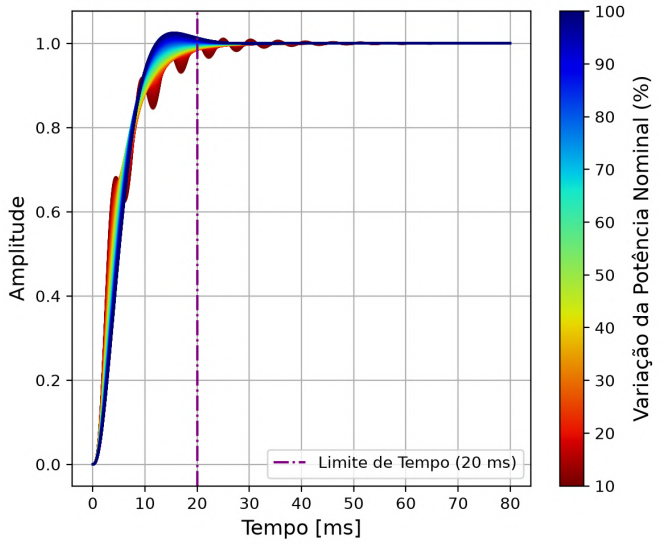
Com as funções de transferência da planta (25), do filtro (90) e do controlador (91) definidas, analisaram-se os polos de malha fechada e a resposta ao degrau para potências variando de $10\,\text{W}$ a $100\,\text{W}$. As Figuras 47 e 48 demonstram que o sistema exibe um comportamento predominantemente de primeira ordem. Como todos os polos estão localizados no semiplano esquerdo, garante-se a estabilidade em toda a faixa de operação. Além disso, a resposta temporal confirma o cumprimento dos requisitos de sobressinal, tempo de acomodação e erro em regime permanente.

Figura 47 – Posição dos polos e zeros do sistema de controle do projeto alternativo para diferentes níveis de potência de operação.



Fonte: Elaborado pelo autor.

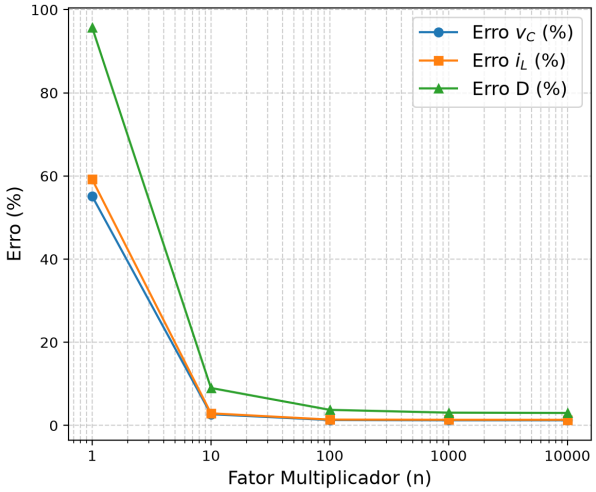
Figura 48 – Resposta transitória do sistema de controle do projeto alternativo para diferentes níveis de potência de operação.



Fonte: Elaborado pelo autor.

Para validar o modelo discretizado desenvolvido neste trabalho, comparou-se a simulação em malha fechada no QSPICE com os resultados numéricos. O teste consistiu em manter a tensão de referência em $25\,\mathrm{V}$ até os $40\,\mathrm{ms}$, instante em que um degrau de $10\,\mathrm{V}$ elevou a referência para $35\,\mathrm{V}$. Primeiramente, avaliou-se a influência do fator de discretização n no erro médio do modelo em relação ao QSPICE. A Figura 49 ilustra que o erro diminui à medida que n aumenta. Esse comportamento ocorre porque intervalos menores entre as atualizações proporcionam uma representação mais fiel da dinâmica contínua do conversor.

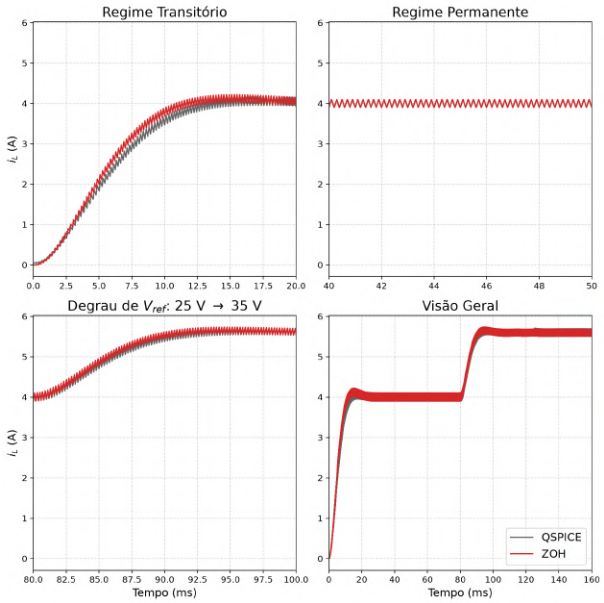
Figura 49 – Erro relativo percentual médio em malha fechada alternativo da corrente (i_L), da tensão (v_C) e *duty cycle* (D) em função do fator n, tendo o QSPICE como referência.



Fonte: Elaborado pelo autor.

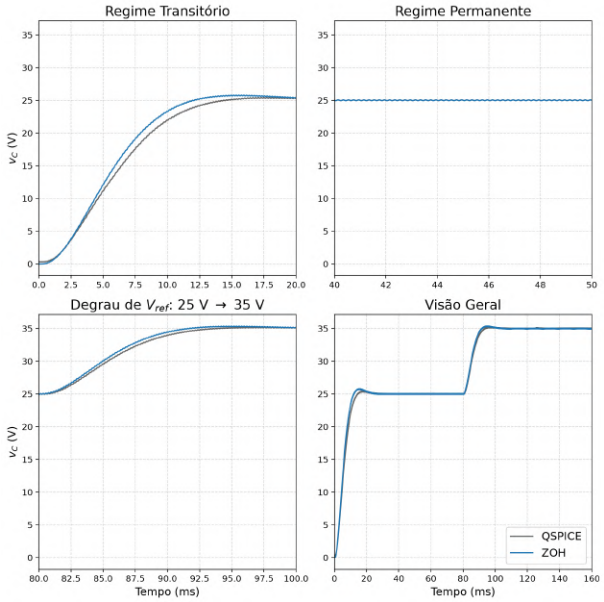
Para uma avaliação visual detalhada, as Figuras 50 e 51 ilustram o comportamento do modelo com fator de discretização elevado ($n = 1000$), detalhando a corrente no indutor e a tensão no capacitor, respectivamente. Nota-se uma excelente concordância com a simulação do QSPICE, tanto em regime permanente quanto durante o transitório provocado pela variação da tensão de referência.

Figura 50 – Comparação do Sinal i_L do conversor Buck alternativo: Simulação Numérica (n = 1000) vs. QSPICE.



Fonte: Elaborado pelo autor.

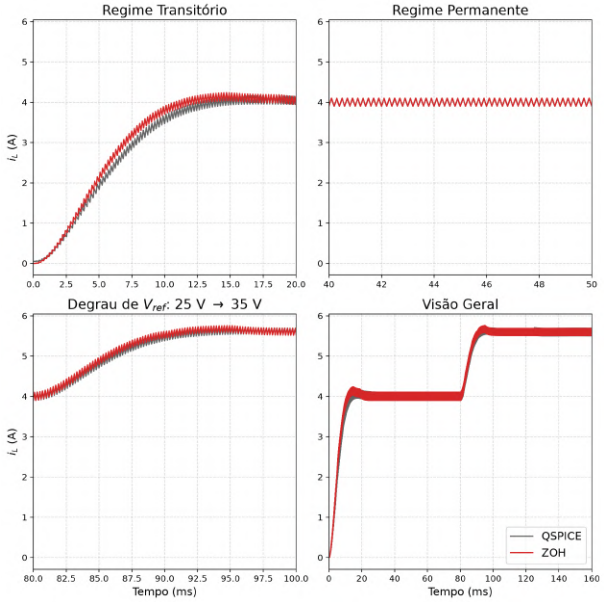
Figura 51 – Comparação do Sinal v_C do conversor Buck alternativo: Simulação Numérica (n = 1000) vs. QSPICE.



Fonte: Elaborado pelo autor.

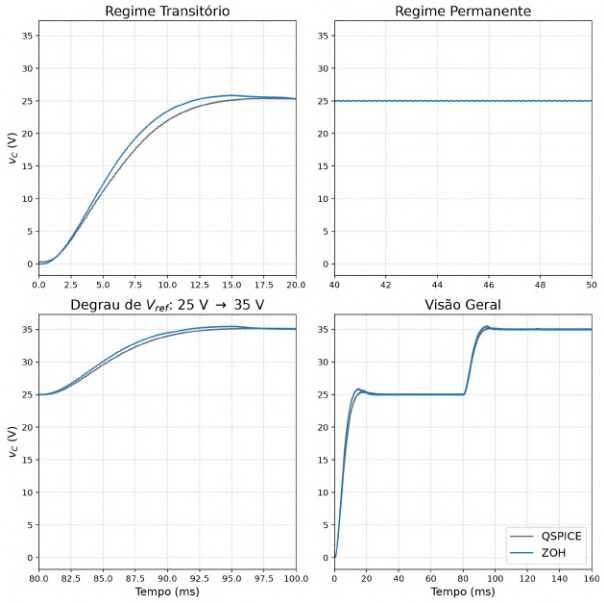
De modo semelhante, as Figuras 52 e 53 exibem as mesmas grandezas para $n = 100$. Apesar de um leve desvio visual em comparação ao caso anterior, o modelo ainda reproduz o comportamento do conversor com precisão bastante satisfatória, demonstrando a robustez da abordagem. Cabe destacar que, com a adoção desse fator de discretização ($n = 100$) associado à nova frequência de chaveamento de 5 kHz, a frequência de simulação exigida para a execução do modelo numérico deste conversor Buck resulta em 500 kHz.

Figura 52 – Comparação do Sinal i_L do conversor Buck alternativo: Simulação Numérica (n = 100) vs. QSPICE.



Fonte: Elaborado pelo autor.

Figura 53 – Comparação do Sinal v_C do conversor Buck alternativo: Simulação Numérica (n = 100) vs. QSPICE.



Fonte: Elaborado pelo autor.

APÊNDICE C – Dispositivo serial virtual

Durante o desenvolvimento da interface gráfica da plataforma HIL, foi desenvolvido um dispositivo serial virtual para permitir a implementação e os testes da comunicação antes da integração com o microcontrolador. Dessa forma, foi possível verificar o fluxo de configuração da simulação e a recepção dos dados pela GUI de forma independente do *hardware*.

O dispositivo foi implementado em Python utilizando pseudo-terminais (PTY) disponibilizados pelo sistema operacional Linux. Um *link* simbólico permite disponibilizar o terminal como uma porta serial acessível pela GUI. A implementação utiliza mensagens binárias compatíveis com o protocolo a ser usado para o firmware do simulador. O Código 17 apresenta os parâmetros iniciais do dispositivo.

Código 17 – Configuração do dispositivo serial virtual.

```
1  import os
2  import struct
3
4  DAC_MAX_VOLTAGE = 3.3
5  DAC_BITS = 12
6  DAC_LEVELS = (1 << DAC_BITS) - 1
7
8  HEADER_CONFIG = 0x0001
9  HEADER_CONFIG_SUCCESS = 0x0002
10 HEADER_CONFIG_FAIL = 0x0003
11 HEADER_DATA = 0x0004
12 HEADER_SIMULATION_END = 0x0005
13
14 class VirtualUSBDevice:
15     def __init__(self, symlink_path="/tmp/ttyVirtual0"):
16         self.symlink_path = symlink_path
17         self.master_fd = None
18         self.slave_fd = None
19         self.slave_name = None
20
21         self.dac_max_v = DAC_MAX_VOLTAGE
22         self.dac_levels = DAC_LEVELS
23
```

Fonte: Elaborado pelo autor.

A recepção da configuração consiste na identificação do cabeçalho e na validação do tamanho do pacote recebido. Em seguida, os parâmetros são extraídos e utilizados para inicializar o modelo discreto do conversor. Também são calculados os fatores de conversão das variáveis de estado para a faixa de tensão utilizada na representação das saídas analógicas.

O Código 18 apresenta a interpretação dos parâmetros de uma mensagem de configuração completa.

A partir dos parâmetros recebidos, o dispositivo virtual executa o modelo discreto do conversor Buck. A frequência de atualização é determinada pela frequência de chaveamento f_s e pelo número de passos de discretização por período n, conforme a Equação 84. O período de atualização é dado por $T_{\text{sim}} = 1/(f_s n)$, enquanto o período de chaveamento é $T_s = 1/f_s$.

Durante a execução, o *duty cycle* permanece fixo e é definido internamente pela constante `DUTY_CYCLE`. Essa condição permite validar a evolução da planta e a comunicação com a GUI sem executar o controlador digital. A cada passo, o estado da chave determina se os termos associados à entrada do conversor devem ser considerados na atualização das variáveis de estado, conforme apresentado no Código 19.

Para representar as saídas analógicas, os estados da planta são convertidos para a faixa de 0 a 3,3 V e quantizados de acordo com a resolução configurada para o DAC. Essa operação permite avaliar a representação dos sinais nas saídas analógicas sem modificar os valores físicos empregados na telemetria. O Código 20 apresenta a função de quantização.

A telemetria é transmitida em formato binário e contém o índice do passo de simulação e os valores das variáveis de estado em suas grandezas físicas. Diferentemente da implementação textual utilizada durante o

Código 18 – Processamento dos parâmetros de configuração no dispositivo virtual.

```
def process_config(self, packet):
    if len(packet) != 46:
        return False

    fields = struct.unpack("<H9fII", packet)

    if fields[0] != HEADER_CONFIG:
        return False

    _, vO_max, iL_max, ad00, ad01, bd10, ad10, ad11, bd11, fs, n, total_steps = fields

    if vO_max <= 0.0 or iL_max <= 0.0:
        return False

    if fs <= 0.0 or n == 0 or total_steps == 0:
        return False

    self.vC_max = vO_max
    self.iL_max = iL_max

    self.ad00 = ad00
    self.ad01 = ad01
    self.bd10 = bd10
    self.ad10 = ad10
    self.ad11 = ad11
    self.bd11 = bd11

    self.fs = fs
    self.n = n
    self.total_steps = total_steps

    self.conversion_factor_vC = self.dac_max_v / self.vC_max
    self.conversion_factor_iL = self.dac_max_v / self.iL_max

    return True

```

Fonte: Elaborado pelo autor.

Código 19 – Atualização do modelo discreto do conversor Buck.

```
if (elapsed % T_switching) < (DUTY_CYCLE * T_switching):
    next_iL = self.ad00 * iL_val + self.ad01 * vC_val + self.bd10
    next_vC = self.ad10 * iL_val + self.ad11 * vC_val + self.bd11
else:
    next_iL = self.ad00 * iL_val + self.ad01 * vC_val
    next_vC = self.ad10 * iL_val + self.ad11 * vC_val

iL_val = next_iL
vC_val = next_vC

```

Fonte: Elaborado pelo autor.

desenvolvimento inicial da GUI, o protocolo definitivo não inclui o instante de simulação no pacote de telemetria. O Código 21 apresenta a formação e o envio da mensagem.

O término da simulação ocorre após a execução da quantidade de passos definida na configuração. Nesse momento, o dispositivo envia a mensagem de encerramento e retorna ao estado de espera por uma nova configuração. As mensagens de resposta e encerramento seguem o formato binário definido para o protocolo, cuja estrutura é apresentada na seção correspondente deste trabalho.

Assim, o dispositivo serial virtual permitiu desenvolver e validar a comunicação entre a GUI e o simulador de forma independente do *hardware*. A implementação possibilitou testar a configuração dos parâmetros, a

Código 20 – Quantização dos sinais para representação das saídas analógicas.

```
def quantize_dac(self, voltage):
    voltage = max(0.0, min(voltage, self.dac_max_v))

    code = voltage * self.dac_levels / self.dac_max_v
    code_quantized = int(code + 0.5)

    return code_quantized * self.dac_max_v / self.dac_levels

```

Fonte: Elaborado pelo autor.

Código 21 – Formação e transmissão da telemetria do dispositivo virtual.

```
telemetry_packet = struct.pack(
    "<HIff", HEADER_DATA, k, iL_val, vC_val
)
os.write(self.master_fd, telemetry_packet)

```

Fonte: Elaborado pelo autor.

execução do modelo discreto e a transmissão das variáveis de estado, utilizando uma interface compatível com o protocolo estabelecido para o firmware do microcontrolador.